

Bakım ve İş Emri Yönetim Sistemi — Teknoloji ve Karar Rehberi

Bariş Köse

August 22, 2026

CONTENTS

Bakım ve İş Emri Yönetim Sistemi — Teknoloji ve Karar Rehberi

Giriş — neden stack değişti

Kararların dört kutusu

BÖLÜM 0 — Sistem nasıl çalışıyor

İki ayrı bilgisayar

Bir isteğin yolculuğu

Her teknoloji hangi adımda — tek tabloda

Sık karıştırılan üç çift

Tek cümlelik özet

BÖLÜM A — Hızlı eşleme tablosu

A.1 Ödevin zorunlu teknolojileri → benim karşılıklarım

A.2 Ödevde adı geçmeyen ama eklediğim parçalar

A.3 Çalışmanın bölümleri → hangi teknolojiyle karşılanıyor

BÖLÜM B — Sistemin yapması istenen on iki şey

1) Kuruma ait lokasyonlar yönetilebilmelidir

2) Lokasyonlara bağlı varlıklar yönetilebilmelidir

3) Varlıklar için arıza, bakım veya kontrol talepleri oluşturulabilmelidir

4) Oluşturulan talepler iş emrine dönüştürülebilmelidir

5) İş emirleri uygun personele atanabilmelidir

6) İş emirlerinin durumları takip edilebilmelidir

7) İş emirlerine yorum ve işlem kaydı eklenebilmelidir

- 8) İş emri geçmişi görüntülenebilmelidir
- 9) İş emirleri için SLA süreleri hesaplanabilmelidir
- 10) Yaklaşan ve geçen SLA süreleri sistem tarafından takip edilebilmelidir
- 11) Kullanıcılara sistem içi bildirim oluşturulabilmelidir
- 12) Yönetim ekranında temel operasyon istatistikleri gösterilebilmelidir

BÖLÜM C — Teknoloji kartları

C.1 NestJS

- 1. “Express’in üstünde çalışır” ne demek
- 2. NestJS’in getirdiği araçlar — istek tüneli
- 3. Projedeki yeri — iki uygulama, aynı çatı
- 4. Ödevin §14 maddesi — servis yaşam döngüsü

C.2 Next.js

Ödevin istediği üç ayrı ürün neydi

Next.js neden üçünün yerine geçiyor

Dürüst tarafı

C.3 Prisma

Prisma nerede duruyor

Prisma’nın dört kritik görevi

⚠️ Zod ile Prisma ilişkisi — yön önemli

Özet mimari akış

C.4 Zod

Zod’un rolü

Akış — Zod tam olarak nerede devreye giriyor

İki farklı denetimi karıştırmamak	
C.5 PostgreSQL	
Foreign key (yabancı anahtar)	
Unique constraint (benzersizlik kuralı)	
Transaction (işlem bütünlüğü)	
pg_trgm + GIN index ile arama	
C.6 BullMQ + Redis	
Neden gerekli	
Bileşenler	
İki iş türü — kodla	
Projedeki dört iş	
En kritik kural: idempotency (aynı iş iki kez çalışsa da sonuç değişmez)	
Dashboard güvenliği	
C.7 Testcontainers	
Neden sahte veritabanı kullanılmıyor	
Nasıl çalışıyor	
Bu projede test edilen senaryolar	
C.8 dependency-cruiser	
Çözdüğü sorun	
Nasıl çalışıyor	
Bu projede denetlenen kurallar	
Neden bu araç	
C.9 TypeScript	

Çözdüğü gerçek sorun

Bu projede neden özellikle kritik

"Strict" modu ne katıyor

Sürüm kısıtı

C.10 Docker ve Docker Compose

Çözdüğü gerçek sorun

Temel kavramlar

Bu projede Compose ne yapıyor

Çok aşamalı (multi-stage) yapı

C.11 Vitest

Testin gerçekte çözdüğü sorun

Bu projede tipik bir test

Zamana bağlı kuralları test etmek

Ödevin istediği test alanları

Koruma testi disiplini

C.12 Playwright

Diğer testlerden farkı

C.13 JWT ve argon2 — kimlik doğrulama

JWT gerçekte hangi sorunu çözüyor

Neden iki ayrı jeton

Jeton nerede saklanıyor

argon2 gerçekte hangi sorunu çözüyor

Ödevin diğer şartları

C.14 Turborepo + pnpm workspaces

Çözdüğü gerçek sorun

Turborepo ne yapıyor

C.15 nestjs-pino — Yapılandırılmış Loglama (Sistemin Gözü)

Neden düz metin değil de JSON

Correlation ID — hata ayıklamanın en değerli alanı

🚫 Log güvenliği (maskeleye / redaction)

Kritik ayrım: uygulama logu vs. denetim kaydı (audit trail)

C.16 nestjs-cls — İstek Bağlamı (Sistemin Hafızası)

Problem

Çözüm: AsyncLocalStorage (nestjs-cls)

Bu projede her şey nasıl birleşiyor

C.17 @nestjs/terminus — Sağlık kontrolü

Çözdüğü sorun

İki ucun farkı

Bu projede kim kullanıyor

C.18 Swagger / OpenAPI

Çözdüğü gerçek sorun

Bu projede doküman elle yazılmıyor

Sadece okunmaz, denenir

Neden Swagger UI, Scalar değil

C.19 TanStack Query

Hangi projede yaşıyor

Çözdüğü gerçek sorun

Ödevdeki “ortak katman” maddesi ne demek istiyor (§22)

Ortak katman pratikte neye benziyor

Bu projede özellikle işe yarayan yanı

C.20 Tailwind CSS + shadcn/ui + React Hook Form

1. Tailwind CSS — hızlı stil verme

2. shadcn/ui — hazır ve esnek tasarım parçaları

3. React Hook Form + Zod uyumu — akıllı form yönetimi

Özetle

C.21 @dbml/cli — veri modeli dokümanı (Database Markup Language)

Problem: elle yazılan doküman ilk şema değişikliğinde eskir

Çözüm: @dbml/cli ile otomatik üretim

CI sürecinde kontrol — “build’in kırmızı yanması” ne demek

Özetle

C.22 Renovate

Çözdüğü gerçek sorun

Nasıl çalışıyor

Neden Dependabot değil

Neden ödevde olmadığı hâlde eklendi

C.23 Expo — ileride mobil için

Gerçek hayat karşılığı

İkisi arasındaki ilişki

Mobil yapılmadığı hâlde neden şimdiden kararı verildi

Nasıl geliştirilir

BÖLÜM E — Kavramlar, prensipler ve desenler

E.0 Önce temel kelimeler

Sınıf ve nesne

Metot

Arayüz (interface)

Katman

Bağımlılık

Projeksiyon

SLA (Service Level Agreement — Hizmet Seviyesi Taahhüdü)

E.1 Modüler monolit + Clean Architecture (§10)

”Monolit” ne demek

Bu projede hangi modüller monolit içinde, hangileri değil

”Modüler” ne katıyor

Clean Architecture’ın tek kuralı

Monorepo, monolit ve mikroservis — üç terim, üç ayrı soru

E.2 SOLID prensipleri (§8)

S — Single Responsibility (tek sorumluluk)

O — Open/Closed (geliştirmeye açık, değiştirmeye kapalı)

L — Liskov Substitution (yerine geçebilirlik)

I — Interface Segregation (arayüz ayrımı)

D — Dependency Inversion (bağımlılığın tersine çevrilmesi)

Ödevin özellikle uyardığı dört hata

E.3 DRY prensibi (§9)

Ödevin saydığı yedi tekrar ve bu projedeki karşılığı

⚠ DRY'ın tehlikeli tarafı — her tekrar ortaklaştırılmaz

E.4 Factory Pattern ve SLA hesabı (§7)

Problem

Naif çözüm ve neden sürdürülemez

Uygulanan tasarım

Ödevin üç uyarısına verilen cevap

E.5 Durum makinesi (§6)

Geçiş tablosu

Tabloya ek olarak duran koşullu kurallar

Neden dağınık if yerine tablo

Geçiş ile geçmiş kaydının atomikliği

Geçersiz geçişin sonucu

E.6 Mapping kararı: neden bir mapping kütüphanesi yok (§13)

Mapping neyi çözer

Ölçüm: iki aday da elendi

Yerine kurulan mekanizma

Ödevin dört şartının karşılanması

Savunma cümlesi

E.7 Merkezî hata yönetimi (§19)

Problem

Çözüm: tek hata filtresi

Hata türleri ve HTTP kodları	
Cevabın içeriği ve correlation ID	
E.8 Transaction ve iyimser eşzamanlılık (§20)	
Kayıp güncelleme (lost update) problemi	
İki yaklaşım	
Uygulama	
API davranışı	
Neden bu yaklaşım	
Transaction sınırları — nerede açılıyor, nerede açılmıyor	
E.9 Veri modeli kararları (§11, §21)	
Soft delete (yumuşak silme)	
Audit alanları	
Enum'ların saklanma biçimi	
Tarih ve saat standardı	
Index kararları	
İş emri numarası üretimi	
Eşzamanlılık alanı	
E.10 Listeleme, filtreleme ve sayfalama (§17)	
Sunucu tarafı ne demek, neden zorunlu	
Sorgu yapısı	
Uygulanan kurallar	
Cevap zarfı	
Offset yerine cursor ne zaman gerekir	

E.11 Git akışı ve CI (§26, §27)

Git nedir

Bu projedeki dal akışı

CI nedir

E.12 Dokümantasyon: ADR ve AI_USAGE (§28, §29)

ADR — mimari karar kaydı

AI_USAGE.md

Zorunlu doküman listesi

E.13 Değerlendirilip seçilmeyen alternatifler

Neden Express, Fastify değil (NestJS'in altındaki HTTP katmanı)

Neden REST, GraphQL değil

Neden Repository Pattern kullanılmadı

Neden BullMQ, pg-boss değil

BÖLÜM F — Bir iş emrinin hayatı (uçtan uca akış)

Bu akışta hangi karar nerede görünüyor

KAPANIŞ

Bilinen teknik borçlar

Bu stack'in tek cümlelik özeti

Değişmeyen ilke

Bakım ve İş Emri Yönetim Sistemi — Teknoloji ve Karar Rehberi

Bu doküman şu soruyu baştan sona cevaplar: **”Kurumun istediği her maddeyi hangi teknolojiyle karşılıyorum, o teknoloji nedir, ne işe yarar ve bu projede tam olarak hangi sorunu çözüyor?”**

Her bölüm **kendi içinde yeterlidir** — bir teknolojiyi okurken başa dönmeniz gerekmez; o teknoloji orada yeniden ve tam olarak anlatılır. Kavramlar, prensipler ve tasarım desenleri Bölüm E’de ayrıca ele alınır.

Giriş — neden stack değişti

Teknik değerlendirme çalışmasında zorunlu teknolojiler .NET ailesinden verilmişti: ASP.NET Core Web API, Entity Framework Core, Hangfire, AutoMapper, FluentValidation, React + Vite. Kurum **”istediğin stack’i kullanabilirsin”** dediği için tamamını JavaScript/TypeScript ailesindeki karşılıklarıyla değiştirdim.

Değiştirirken tek bir kural uyguladım: **hiçbir yetenek düşmeyecek**. Çalışmanın sorduğu her şey — Factory Pattern, servis yaşam döngüleri, transaction, iyimser eşzamanlılık denetimi, mimari testler, DBML şema dokümanı — yeni stack’te de birebir karşılanıyor. **Araç değişti, beklenti değişmedi.**

Seçimlerimde üç ölçütüm vardı:

1. Piyasada yaygın ve aktif bakımda olması. Bu sistemi yıllarca başkaları da sürdürecektir. Bir kütüphane niş kalıyorsa veya yıllardır yeni sürüm çıkmıyorsa, projeyi devralacak geliştirici için risktir. Bu yüzden kütüphane seçimlerinde haftalık indirme sayısı ve son yayın tarihi **fiilen ölçüldü**; gerekçeler ilgili bölümlerde rakamlarıyla duruyor.

2. Gerçek üretim pratiği olması. İki seçenek arasında kalındığında ölçüt *”hangisi daha hızlı biter”* değil, **”gerçek kullanıcısı ve gerçek nöbetçi ekibi olan bir sistemde hangisi doğru olurdu”** oldu.

Somut örnek: entegrasyon testlerinde sahte bir veritabanı kullanmak çok daha hızlı kurulur ve testler saniyeler yerine milisaniyelerde koşar. Ama sahte veritabanı benzersizlik kısıtını, foreign key'i ve transaction'ı uygulamaz — yani testler yeşil yanar, sistem canlıda patlar. Bu yüzden testler **gerçek PostgreSQL konteynerine** karşı koşuyor (C.7). Aynı yaklaşım eşzamanlılık denetimi, mükerrer bildirim koruması ve hata yönetimi için de geçerli: kolay olan değil, **üretimde doğru olan** seçildi.

3. Devralınabilir olması. Projeyi ilk kez gören biri nereye bakacağını bilmeli — ve daha önemlisi, **yanlış yaptığıında sistem ona söylemeli**. Bu yüzden mimari kurallar yorum satırı olarak değil, CI'da build'i durduran denetimler olarak kuruldu (C.8, C.21).

Kararların dört kutusu

Anlatırken her kararın hangi kutuda olduğunu söyleyeceğim:

Kutu	Anlamı	Örnek
1	Ödev istedi, zaten doğrusuydu	PostgreSQL, Docker, Factory Pattern, optimistic concurrency
2	Ödev istedi, JS eşleniğini kullandım	EF Core→Prisma, Hangfire→BullMQ, FluentValidation→Zod
3	Ödev istemedi, gerçek hayat gerektirdi	Renovate, mimari testi, correlation ID, <code>/api/v1</code>
4	Ödev istedi, yapmadım — şunu tercih ettim	AutoMapper yerine Zod+Prisma <code>select</code>

⚠ 4. kutudaki her madde **ölçüyle** desteklenir (indirme sayısı, son yayın tarihi, sürüm kısıtı) — “bence daha iyi” denmez.

BÖLÜM 0 — Sistem nasıl çalışıyor

Bu bölüm zemindir: sonraki bölümlerdeki her teknoloji, aşağıdaki altı adımdan birinde çalışıyor. Hangisinde çalıştığını bilmek, o teknolojiyi anlamamanın yarısıdır.

İki ayrı bilgisayar

Her web uygulamasında iş **iki yerde** yapılır:

	İstemci (client)	Sunucu (server)
Neresi	Kullanıcının tarayıcısı / telefonu	Senin kontrolündeki, kapanmayan bilgisayar
Ne yapar	Ekranı çizer, tıklamayı alır	Karar verir, veritabanına gider
Güvenilir mi	Hayır	Evet

”İstemciye güvenilmez” ne demek: Kullanıcı tarayıcıda geliştirici araçlarını açıp gönderdiği veriyi değiştirebilir. “Ben yöneticiyim” diye veri gönderebilir. Bu yüzden *”bu kişi gerçekten yönetici mi”* kararı **her zaman sunucuda** verilir. Ekrandaki “Sil” düğmesini gizlemek bir **kolaylıktır**, güvenlik değil.

Bir isteğin yolculuğu

Kullanıcı “İş Emirleri” ekranını açtığında olanlar:

1. Tarayıcı	"bana açık iş emirlerini ver"
↓ (internet)	
2. API sunucusu	"sen kimsin? yetkin var mı?"
↓	
3. İş kuralları görebilir"	"bu kullanıcı sadece kendi lokasyonunu
↓	
4. Veritabanı	SELECT ... WHERE ...
↓	
5. API sunucusu	sonucu JSON'a çevirir, fazla alanları kırpar
↓ (internet)	
6. Tarayıcı	gelen veriyi tabloya çizer

Aşağıdaki tablo her teknolojinin bu altı adımdan hangisinde çalıştığını gösteriyor.

Her teknoloji hangi adımda — tek tabloda

Teknoloji	Hangi adım	En sade tanımı	Günlük hayattan benzetme
React	6	Veriye göre ekranı çizen kütüphane	Vitrin düzenleyicisi
Next.js	6 (+1)	React'i çalışır siteye dönüştüren framework; sayfaları da sunucuda hazırlar	Vitrini kuran ve mağazayı açan ekip
TypeScript	Hepsi	JavaScript + tip kontrolü. Hatayı çalışmadan önce yakalar	Yazım denetimi
NestJS	2–5	Sunucu tarafı uygulama çatısı	Bir ofisin organizasyon şeması
Express	2	Gelen isteği karşılayan temel katman. <i>Nest zaten bunu kullanır</i>	Santral görevlisi
Prisma	4	Kod ile veritabanı arasındaki tercüman	Tercüman
PostgreSQL	4	Verinin durduğu yer	Arşiv/dolap
Zod	2 ve 5	"Gelen veri doğru biçimde mi?" kontrolü	Form denetleyicisi
JWT	2	Giriş yapan kullanıcıya verilen dijital kimlik kartı	Otel kartı
argon2	2	Şifreyi geri çevrilemez şekilde karıştırır	Kağıt öğütücü
BullMQ	—	Sonra/düzenli yapılacak işlerin listesi	Yapılacaklar defteri
Redis	—	Çok hızlı, geçici hafıza. BullMQ'nun defteri burada durur	Not kağıdı
Docker	—	Uygulamayı ihtiyaçlarıyla birlikte tek pakete koyar	Hazır yemek kabı

Teknoloji	Hangi adım	En sade tanımı	Günlük hayattan benzetme
Vitest / Playwright	—	Testler: “hâlâ çalışıyor mu?”	Kalite kontrol
Swagger	—	API’nin kullanma kılavuzu, kendiliğinden üretilir	Kullanım kılavuzu
Expo	6	Aynı React bilgisiyle telefon uygulaması yazma aracı	—

Sık karıştırılan üç çift

- 1. Kütüphane ile framework** Kütüphaneyi **sen çağırırsın**, framework **seni çağırır**. React bir kütüphanedir — sen “şunu çiz” dersin. Next.js ve NestJS framework’tür — kuralları onlar koyar, sen boşlukları doldurursun. Framework daha kısıtlayıcıdır ama karşılığında düzen verir; on kişilik ekipte bu düzen her şeydir.
- 2. Frontend ile backend** Frontend = kullanıcının **gördüğü**. Backend = kullanıcının **göremediği** ama işi yapan. Bizde frontend Next.js, backend NestJS.
- 3. Veritabanı ile ORM** Veritabanı (PostgreSQL) verinin durduğu yer. ORM (Prisma) oraya konuşan tercüman. ORM olmadan da olur — SQL’i elle yazarsın; ama o zaman tip güvenliğini ve migration yönetimini kendin kurarsın.

Tek cümlelik özet

”Kullanıcı **Next.js** ile yazılmış ekranı açar. Ekran, **NestJS** ile yazılmış API’ye istek atar. API gelen veriyi **Zod** ile doğrular, kullanıcının yetkisini **JWT** ile kontrol eder, iş kurallarını uygular ve **Prisma** üzerinden **PostgreSQL**’e gider. Sonuç JSON olarak döner. Kullanıcıyı bekletmemesi gereken işler **BullMQ** ile kuyruğa alınır ve ayrı bir worker süreci onları yapar. Hepsi **Docker** ile paketlenir, tek komutla ayağa kalkar.”

BÖLÜM A — Hızlı eşleme tablosu

(Bu bölüm içindekiler niteliğindedir. Ayrıntılar Bölüm B ve C’de.)

A.1 Ödevin zorunlu teknolojileri → benim karşılıklarım

Çalışmada istenen	Bu projede kullandığım	Tek cümlelik gerekçe
Güncel .NET + ASP.NET Core Web API	NestJS	Node tarafında bağımlılık enjeksiyonu, modül sistemi ve yetki/doğrulama/hata katmanlarını hazır getiren yaygın çatı
C#	TypeScript (strict)	Derleme zamanında tip güvenliği
Entity Framework Core	Prisma	Şema tek dosyada okunur; migration motoru ve tip güvenli sorgu birlikte gelir
PostgreSQL	PostgreSQL	Değişmedi
FluentValidation	Zod	Tek şemadan hem doğrulama, hem TypeScript tipi, hem OpenAPI dokümanı üretilir
Hangfire	BullMQ + Redis	Gecikmeli iş, tekrarlayan iş, yeniden deneme ve yatay ölçekleme
AutoMapper veya Mapster	Zod şeması + Prisma <code>select</code>	Aday kütüphaneler ölçüldü; biri bakımsız, diğeri niş kaldı
OpenAPI + Scalar/Swagger	@nestjs/swagger → Swagger UI	Doküman Zod şemasından üretiliyor, elle güncelleme gerekmiyor
React	React	Değişmedi
JavaScript veya TypeScript	TypeScript	—
Vite	Next.js (kendi derleyicisi)	Next, Vite'in paketleyici görevini kendi içinde yapıyor
React Router	Next.js App Router	Next, yönlendirmeyi dosya yapısıyla çözüyor
xUnit / NUnit / MSTest	Vitest	JS tarafının yaygın test koşturucusu

Çalışmada istenen	Bu projede kullandığım	Tek cümlelik gerekçe
Assertion kütüphanesi	Vitest’in kendi <code>expect</code>’i	Ayrı paket gerekmiyor
PostgreSQL ile integration test	Testcontainers	Test sırasında gerçek PostgreSQL konteyneri ayağa kaldırır
Docker · Docker Compose · Git	Aynı	Değişmedi

A.2 Ödevde adı geçmeyen ama eklediğim parçalar

Eklenen	Neden eklendi
Redis	BullMQ’nun iş kuyruğu burada durur; ayrıca hız sınırı sayacı ve dağıtık kilit için kullanılıyor
argon2	Şifrelerin güvenli saklanması (§5.1) için sektörde önerilen özetleme algoritması
<code>nestjs-cls</code>	Aktif kullanıcı ve iz kimliğini katmanlara parametre geçmeden taşır — §21’in “statik yapılardan alınmasın” şartının karşılığı
<code>nestjs-pino</code>	Yapılandırılmış (JSON) log üretir; §25’in istediği biçim
<code>@nestjs/terminus</code>	<code>/health/live</code> ve <code>/health/ready</code> uçları (§25)
<code>dependency-cruiser</code>	Katman bağımlılık kurallarını CI’da denetler — §23’ün mimari testleri
<code>@dbml/cli</code>	<code>docs/database.dbml</code> dosyasını migration’ın ürettiği gerçek şemadan üretir (§11)
Turborepo + pnpm workspaces	Web, API ve worker’ı tek depoda tutar; ortak tipler tek yerde yaşar (§9 DRY)
Renovate	Bağımlılık güncellemelerini otomatik takip eder

A.3 Çalışmanın bölümleri → hangi teknolojiyle

karşılıyor

Bölüm	Ne isteniyor	Nasıl karşılanıyor
§4 Kullanıcı rolleri	Dört rol, yetkileri ayrılmış	NestJS Guard 'ları; yetki kontrolü tek yerde
§5.1 Kimlik doğrulama	JWT, yenileme jetonu, rol bazlı yetki, pasif kullanıcı engeli	<code>@nestjs/jwt</code> + argon2 + jeton döndürme ve yeniden kullanım tespiti
§5.2 Lokasyon yönetimi	Liste, detay, oluştur, güncelle, aktif/pasif	NestJS modülü + Prisma + Zod
§5.3 Varlık yönetimi	Liste, filtre, detay, durum değiştirme	Aynı yapı; filtreleme veritabanı seviyesinde
§5.4 Talep ve iş emri	Zorunlu alanlar, okunabilir iş emri numarası	Prisma şeması + veritabanı sırası (sequence)
§6 Durum yönetimi	Kontrollü geçişler, her değişiklikte geçmiş	Durum makinesi + tek transaction
§7 SLA ve Factory Pattern	Factory zorunlu, Open/Closed'a uygun	NestJS bağımlılık enjeksiyonu ile politika dizisi
§8 SOLID	Katman bağımlılıkları korunmalı	Katmanlı yapı + dependency-cruiser
§9 DRY	Tekrarlanan kural/mapping/hata olmasın	Ortak şema paketi + tek doğrulama + tek hata filtresi
§10 Proje mimarisi	Tutarlı ve gerekçeli mimari	Modüler monolit + Clean Architecture
§11 Veri tabanı tasarımı	Şema, index, audit, DBML	Prisma şeması + <code>@dbml/cli</code>
§12 Entity Framework Core	Migration, ilişki, transaction, N+1 önleme	Prisma
§13 Mapping	Entity dönülmesin, hassas alan sızmasın	Zod şeması + Prisma <code>select</code>

Bölüm	Ne isteniyor	Nasıl karşılanıyor
§14 DI ve yaşam döngüleri	Transient/Scoped/Singleton	NestJS DI
§15 Zamanlanmış görevler	Dört iş türü, idempotent, korumalı panel	BullMQ + Redis
§16 Bildirim yönetimi	Mükerrer bildirim engellenmeli	Benzersiz index + benzersiz iş anahtarı
§17 Listeleme ve sayfalama	Sunucu tarafı filtre, arama, sıralama	Prisma + <code>pg_trgm</code> + GIN index
§18 Validasyon	Hem biçim hem iş kuralı	Zod (biçim) + kurallar katmanı (iş kuralı)
§19 Hata yönetimi	Merkezî, tutarlı, tiplere ayrılmış	NestJS Exception Filter
§20 Transaction ve eşzamanlılık	Tek sınır, iyimser kilitleme	<code>prisma.\$transaction</code> + <code>version</code> kolonu
§21 Audit	Merkezî audit, saat ve kullanıcı soyutlanmış	Prisma eklentisi + <code>nestjs-cls</code> + <code>Clock</code>
§22 React uygulaması	12 ekran, korumalı rota	Next.js + TanStack Query + shadcn/ui
§23 Testler	Unit, integration, architecture	Vitest + Testcontainers + dependency-cruiser
§24 Docker	Tek komutla dört servis	Docker Compose
§25 Health check ve loglama	İki sağlık ucu, yapılandırılmış log	Terminus + <code>nestjs-pino</code>
§26–27 Git ve CI	Anlamli geçmiş, otomatik kapılar	git + GitHub Actions / GitLab CI
§28–29 Dokümantasyon	<code>AI_USAGE.md</code> + sekiz doküman	<code>README</code> + <code>docs/</code>

Bu tablodaki her satırın ayrıntısı nerede: sistemin *ne yapacağı* Bölüm B’de madde madde, *hangi araçla yapıldığı* Bölüm C’deki teknoloji kartlarında, *hangi kavram ve desene dayandığı* ise Bölüm E’de anlatılıyor. Aynı açıklama iki kez yazılmadı; her konu tek yerde durur.

BÖLÜM B — Sistemin yapması istenen on iki şey

Çalışma dosyasının 2. bölümünde sistemin yapabilmesi gerekenler on iki madde hâlinde sayılıyor. Her maddede sırayla şunu anlatıyorum: **ödev hangi teknolojiyi bekliyordu, ben yerine ne kullandım, o teknoloji nedir ve bu projede hangi somut sorunu çözüyor.**

1) Kuruma ait lokasyonlar yönetilebilmelidir

Ödevin beklediği: ASP.NET Core Web API · FluentValidation · Entity Framework Core
Benim kullandığım: NestJS · Zod · Prisma · PostgreSQL

Bu teknolojiler nedir:

- **PostgreSQL**, verinin kalıcı olarak durduğu yer. Excel'in çok daha güçlü hâli gibi düşünün: satırlar ve sütunlar var, ama kayıtlar birbirine bağlanabiliyor ve kuralları sistemin kendisi koruyor.
- **Prisma**, kod ile veritabanı arasındaki tercüman. “Aktif lokasyonları getir” diye yazıyorum, o bunu veritabanının anladığı dile çeviriyor.
- **NestJS modülü**, lokasyonla ilgili her şeyin (uç noktalar, kurallar, veri erişimi) tek klasörde toplanması demek.
- **Zod**, dışarıdan gelen verinin doğru biçimde olup olmadığını denetleyen bekçi.

Bu projede hangi sorunu çözüyor: Kurumun onlarca lokasyonu var ve bunlar zamanla değişiyor: yeni bina açılıyor, eski depo kapanıyor. Kapanan bir lokasyonu **silmek** doğru değil — geçmiş iş emirleri ona bağlı, silersek tarih bozulur. Bu yüzden silmiyoruz, **pasife alıyoruz**. Pasif lokasyonda yeni varlık veya yeni iş emri açılmıyor, ama geçmiş kayıtlar yerli yerinde duruyor.

Neden böyle seçtim: Lokasyon adının boş gönderilmesi ya da 500 karakterlik saçma bir metin girilmesi gibi durumları tek tek kontrol yazarak değil, Zod şemasıyla engelliyorum. Aynı şema hem tarayıcıdaki formu hem sunucuyu koruyor — kuralı iki kez yazmıyorum.

2) Lokasyonlara bağlı varlıklar yönetilebilmelidir

Ödev beklediği: Entity Framework Core ilişkileri **Benim kullandığım:** Prisma ilişkileri + PostgreSQL foreign key

Bu nedir: İki tablo arasında **bağ** kurmak demek. Her varlık (jeneratör, klima, hizmet aracı) bir lokasyona ait. **Foreign key** dediğimiz şey, veritabanının kendisinin koruduğu bir kural: *"olmayan bir lokasyona varlık bağlanamaz."*

Bu projede hangi sorunu çözüyor: Kullanıcı arayüzünde bir hata olsa bile veritabanı yanlış veriyi **kabul etmiyor** — koruma tek katmanda değil. Ayrıca varlıkların kritiklik seviyesi burada tutuluyor; bu bilgi ileride SLA süresini belirlerken kullanılacak (madde 9).

Neden böyle seçtim: Bu tür bağları uygulama kodunda "önce kontrol et, sonra kaydet" diye yazmak yaygın bir hatadır: iki kullanıcı aynı anda işlem yaparsa kontrol ile kayıt arasında lokasyon silinebilir. Veritabanının kendi kuralı bu açığı bırakmıyor.

3) Varlıklar için arıza, bakım veya kontrol talepleri oluşturulabilmelidir

Ödev beklediği: ASP.NET Core Web API (kapıyı açan) · FluentValidation (gelen veriyi denetleyen) · Entity Framework Core (veritabanına yazan) **Benim kullandığım:** NestJS · Zod · Prisma

Neden değiştirdim: Kurum stack'i serbest bıraktı. Üçünün de yaptığı işi JavaScript tarafında birebir karşılayan, piyasada yaygın ve aktif bakımda olan karşılıklarını seçtim. Yapılan iş aynı, sadece araçlar değişti.

Bu teknolojiler nedir:

NestJS, sunucu tarafında çalışan bir uygulama çatısı. "Çatı" demek, size hazır bir düzen vermesi demek: hangi kodun nereye yazılacağını, isteğin hangi sırayla işleneceğini o belirler. Bir ofisin organizasyon şeması gibi — kim ne yapar bellidir,

herkes kendi kafasına göre çalışmaz. Bu projede 50'den fazla uç nokta olacağı için düzen kendiliğinden korunmalı.

Uç nokta (endpoint), uygulamanın dışarıya açtığı bir kapı: *"buraya şu bilgileri gönderirsen sana şunu yaparım."* Talep oluşturma kapısına form bilgileri gelir.

Zod, dışarıdan gelen verinin doğru biçimde olup olmadığını denetleyen bekçi. "Başlık boş olamaz", "açıklama en fazla 2000 karakter", "öncelik şu dört değerden biri olmalı" gibi kuralları tek yerde tanımlarsınız.

Prisma, kod ile veritabanı arasındaki tercüman.

Bu projede hangi sorunu çözüyor: Herkes talep açabilmeli — ama her talep kabul edilmemeli. Kullanım dışı bırakılmış bir jeneratör için arıza talebi açılması anlamsız; kapalı bir lokasyon için de öyle. Burada iki farklı denetim var ve bunları karıştırmamak önemli: *"başlık boş mu"* bir **biçim** sorusudur, Zod cevaplar. *"Bu varlık için talep açılabilir mi"* bir **iş kuralı** sorusudur, ayrı bir katmanda duruyor.

Neden bu ayrımı yaptım: İş kurallarını kapının içine yazmak kolay olurdu, ama aynı talep mobil uygulamadan veya arka plan işinden geldiğinde o kural çalışmazdı. Ayrı katmanda olduğu için kural **tek yerde yaşıyor** ve testi veritabanı gerektirmeden, saniyenin altında koşuyor.

4) Oluşturulan talepler iş emrine dönüştürülebilir

Ödevin beklediği: EF Core transaction yönetimi **Benim kullandığım:** Prisma transaction (`$transaction`)

Bu nedir: Transaction, birbirine bağlı birden fazla işlemi "ya hepsi ya hiçbiri" mantığıyla yapmak demek. Yarım kalmasına izin verilmez.

Bu projede hangi sorunu çözüyor: Talep iş emrine dönüşürken aynı anda birkaç şey oluyor: iş emri kaydı açılıyor, ilk geçmiş kaydı yazılıyor, iş emri numarası üretiliyor. Hata çıksa bunların **yarısının** kaydedilmesi sistemi tutarsız bırakır — numarası olan ama geçmişi olmayan bir iş emri gibi. Transaction bunu imkânsız kılıyor.

Neden böyle seçtim: Çalışma §20’de bunu açıkça istiyor. Ayrıca her işleme gereksiz transaction açmıyorum — sadece gerçekten birbirine bağlı işlemlerde.

5) İş emirleri uygun personele atanabilmelidir

Ödevin beklediği: ASP.NET Core `[Authorize]` + rol bazlı yetkilendirme **Benim kullandığım:** NestJS Guard’ları + iş kuralı doğrulaması

Bu nedir: **Guard**, kapıdaki güvenlik görevlisi gibi: istek işleme girmeden önce *”sen kimsin, buna yetkin var mı”* diye bakar.

Bu projede hangi sorunu çözüyor: İki ayrı soru var ve karıştırılmamalı: *”Bu kişi atama yapabilir mi?”* bir **yetki** sorusudur, Guard cevaplar. *”Atanan kişi teknik personel mi ve aktif mi?”* bir **iş kuralı** sorusudur, kurallar katmanı cevaplar. Muhasebe personeline jeneratör tamiri atanamaması yetki meselesi değil, iş kuralı meselesidir.

Neden böyle seçtim: Bu ayırım yapılmazsa yetki kontrolleri iş kurallarının içine karışır ve altı ay sonra kimse hangisinin nerede olduğunu bulamaz.

6) İş emirlerinin durumları takip edilebilmelidir

Ödevin beklediği: Belirli bir teknoloji şart koşulmamış; sadece “dağınık koşul blokları olmasın” denmiş **Benim kullandığım:** Durum makinesi (state machine) — sade bir kurallar tablosu

Bu nedir: **Durum makinesi**, “hangi durumdan hangi duruma geçilebilir” sorusunun tek bir yerde yazılı hâli. Trafik ışığı gibi: kırmızıdan yeşile geçilir, kırmızıdan sarıya geçilmez.

Bu projede hangi sorunu çözüyor: İş emri “Açık → Atandı → İşlemde → Çözüldü → Kapatıldı” yolunu izliyor. Atanmamış bir işin “İşlemde” olması ya da kapatılmış bir işin tekrar açılması kabul edilemez. Bu kontroller uç noktaların içine dağıtılmış `if` blokları olarak yazılırsa, yeni bir durum eklendiğinde hepsini tek tek bulmak gerekir — biri unutulur ve açık kalır.

Neden böyle seçtim: Çalışma §6 bunu özellikle vurguluyor: *”Durum geçiş kurallarının controller içerisinde dağınık koşul bloklarıyla yönetilmesi beklenmemektedir.”* Tek tablo hâlinde tutunca hem okunuyor hem test ediliyor.

7) İş emirlerine yorum ve işlem kaydı eklenebilmelidir

Ödevin beklediği: EF Core `SaveChanges` üzerinden merkezî audit **Benim kullandığım:** Ayrı yorum tablosu + Prisma eklentisi ile otomatik audit

Bu nedir: Audit alanları, her kaydın “kim oluşturdu, ne zaman, kim güncelledi, ne zaman” bilgisi. **Prisma eklentisi** ise bu alanları elle yazmak yerine **otomatik dolduran** bir ara katman.

Bu projede hangi sorunu çözüyor: Bu bilgileri her servis içinde tek tek doldurmak hem sıkıcı hem tehlikeli — biri unutulunca o kaydın izi kaybolur. Merkezî bir yerden doldurunca unutma ihtimali ortadan kalkıyor.

Neden böyle seçtim: Çalışma §21 “audit alanlarının merkezî bir yaklaşımla yönetilmesi” istiyor. Ayrıca aktif kullanıcı bilgisini global bir değişkenden almıyorum — bu, iki kullanıcının verisinin karışmasına yol açan klasik hatadır.

8) İş emri geçmişi görüntülenebilmelidir

Ödevin beklediği: İşlem geçmişi tabloları (§11) **Benim kullandığım:** Ayrı geçmiş tabloları + transaction garantisi

Bu nedir: Her durum değişikliği ve her atama, ayrı bir tabloya **satır olarak** yazılıyor. Kaydın üstüne yazmak yerine yeni satır eklemek — defter tutmak gibi.

Bu projede hangi sorunu çözüyor: *”Bu iş emri neden 3 gün bekledi?”* sorusunun cevabı ancak geçmiş varsa verilebilir. Kurumsal bir sistemde bu bilgi denetim için gerekiyor ve yıllarca saklanıyor.

Neden böyle seçtim: Geçmiş kaydını uygulama loglarına yazmak yaygın bir hatadır — loglar birkaç hafta sonra silinir, oysa “bu kaydı kim değiştirdi” sorusu iki yıl sonra sorulur. Bu yüzden geçmiş **veritabanında tablo** ve durum değişikliğiyle **aynı transaction**

içinde yazılıyor.

9) İş emirleri için SLA süreleri hesaplanabilmelidir

Ödevin beklediği: Factory Pattern (zorunlu tutulmuş) **Benim kullandığım:** Factory Pattern + NestJS bağımlılık enjeksiyonu

Bu nedir: SLA, işin ne kadar sürede tamamlanması gerektiğine dair söz. **Factory Pattern**, “duruma göre doğru hesaplayıcıyı seçen” bir tasarım deseni. **Bağımlılık enjeksiyonu**, sınıfların ihtiyaç duyduğu parçaları kendilerinin üretmesi yerine dışarıdan verilmesi.

Bu projede hangi sorunu çözüyor: SLA süresi tek bir sayı değil; işin önceliğine, varlığın kritiklik seviyesine ve iş emri türüne göre değişiyor. Kritik bir jeneratörün arızası 4 saatte, düşük öncelikli bir boya işi 15 günde çözülmeli. Bunu tek yerde iç içe `if` bloklarıyla yazarsak, yeni bir kural geldiğinde o bloğa dokunmak zorunda kalırız — ve her dokunuş mevcut kuralları bozma riski taşır.

Neden böyle seçtim: Her SLA kuralını **ayrı bir sınıf** yaptım. Yeni kural gelince yeni sınıf yazılıyor, mevcut kod **hiç değişmiyor**. Yazılımda buna Open/Closed prensibi deniyor: geliştirmeye açık, değiştirmeye kapalı. Çalışma §7 Factory Pattern’i zorunlu tutuyor ve “*göstermelik olmasın*” diye özellikle uyarıyor.

10) Yaklaşan ve geçen SLA süreleri sistem tarafından takip edilebilmelidir

Ödevin beklediği: Hangfire (gecikmeli job + recurring job) **Benim kullandığım:** BullMQ + Redis + ayrı worker süreci

Bu nedir: Normalde uygulama **isteğe cevap verir** — biri bir şey sorar, o cevaplar. Ama burada kimse bir şey sormasa da çalışması gereken işler var. **İş kuyruğu**, “şunu 2 saat sonra yap” veya “şunu her 15 dakikada bir yap” diyebildiğiniz bir yapılacaklar defteri. **Worker**, o defteri okuyup işleri yapan ayrı bir program.

Bu projede hangi sorunu çözüyor: SLA süresi dolmadan önce uyarı gitmeli, süre aşılmıca iş emri ihlal olarak işaretilenmeli. Bunu kimse ekranı açmasa bile sistemin kendisi yapmalı — gece 3'te bile.

Neden böyle seçtim ve dikkat ettiğim nokta: Bu işlerin **iki kez çalışması** en büyük risk. Sunucu yeniden başlarsa veya iş yeniden denenirse aynı bildirim tekrar üretilebilir. Bunu iki katmanda engelliyorum: kuyrukta benzersiz iş anahtarı, veritabanında benzersiz index. Ayrıca işler yalnızca kimlik numarası alıyor, durumu **kendisi veritabanından okuyor** — iş emri bu arada kapanmışsa hiçbir şey yapmıyor.

11) Kullanıcılara sistem içi bildirim oluşturulabilmelidir

Ödevin beklediği: Bildirim tekrarlarını engelleyen yapılar (§11, §16) **Benim kullandığım:** Bildirim tablosu + benzersiz index + BullMQ

Bu nedir: Benzersiz index (unique index), veritabanına “bu ikili tekrar edemez” demenin yolu. Örneğin “şu kullanıcıya, şu olay için” bildirimi bir kez yazılabilir.

Bu projede hangi sorunu çözüyor: Aynı olay için kullanıcıya beş bildirim gitmesi, sistemin güvenilirliğini bitiren şeydir — insanlar bildirimlere bakmayı bırakır. Uygulama kodunda “önce var mı diye bak, yoksa ekle” yazmak yeterli değil: iki iş aynı anda çalışırsa ikisi de “yok” görüp ikisi de ekler. Veritabanının kendi kuralı bu yarışı kaybettirmiyor.

Neden böyle seçtim: Çalışma §15 ve §16 bunu ayrı ayrı vurguluyor. Koruma tek katmanda olsaydı, o katman atlandığında sessizce bozulurdu.

12) Yönetim ekranında temel operasyon istatistikleri gösterilebilmelidir

Ödevin beklediği: EF Core sorguları + React ekranı **Benim kullandığım:** Prisma toplama sorguları + Next.js + TanStack Query

Bu nedir: Toplama (aggregation) sorgusu, “kaç tane açık iş emri var” gibi sayı üreten sorgu. **TanStack Query**, tarayıcı tarafında veriyi getiren, saklayan ve tazeleyen kütüphane.

Bu projede hangi sorunu çözüyor: Yönetici ekranında açık iş emri sayısı, SLA ihlali sayısı, kritik iş sayısı ve personel bazında iş yükü görünüyor. Bu sayıları **veritabanına saydırıyorum** — tüm kayıtları çekip uygulamada saymıyorum. On bin kayıta bu iki yaklaşım arasında saniyelerce fark oluyor.

Neden böyle seçtim: Çalışma §17 açıkça *”bütün kayıtlar belleğe alındıktan sonra filtreleme yapılmamalıdır”* diyor; aynı ilke sayma için de geçerli. Ayrıca günlük özet, her gece çalışan bir arka plan işiyle önceden hesaplanıp saklanıyor — yönetici ekranı açtığında ağır sorgu beklemiyor.

BÖLÜM C — Teknoloji kartları

Bu bölümde her teknoloji **tek tek ve kendi içinde yeterli** biçimde anlatılıyor. Bir kartı okurken başka bölüme dönmeniz gerekmez.

C.1 NestJS

Nedir: Node.js için sunucu tarafı uygulama çatısı. Express'in üstünde çalışır. **Ne işe yarar:** API uçlarını, iş kurallarını ve yetkilendirmeyi düzenli bir yapıda barındırır. Modül sistemi, bağımlılık enjeksiyonu (DI), Guard (yetki), Interceptor (log/denetim), Pipe (doğrulama), Filter (merkezî hata) getirir. **Bu projede nerede:** Tüm API (`apps/api`) ve arka plan işçisi (`apps/worker`). **Ödevdeki karşılığı:** ASP.NET Core Web API.

1. “EXPRESS’İN ÜSTÜNDE ÇALIŞIR” NE DEMEK

Gerçek hayat: Boş bir dükkân kiralamak ile hazır kurulmuş bir zincir mağaza şubesi açmak. Boş dükkânda rafları nereye koyacağınız, kasanın nerede duracağı, deponun nasıl düzenleneceği size kalmıştır — ilk gün özgürlük, üçüncü yıl kaos. Zincir mağazada düzen önceden bellidir; yeni gelen çalışan hangi ürünün nerede olduğunu bilir.

Düz Express’in sorunu: Express size hiçbir kural koymaz. Klasör yapısını, kodları nereye yazacağınızı siz seçersiniz. Proje büyüdükçe her geliştirici kendi düzenini kurar.

NestJS çözümü: Express’in üzerine kurulmuş çelik bir iskelet. Express’in hızını arkada kullanmaya devam eder, size disiplinli bir mimari sunar.

⚠ Bu yüzden *“Express mi Nest mi”* diye bir seçim yoktur — Nest’i seçtiğinizde Express’i zaten almış olursunuz.

2. NESTJS’İN GETİRDİĞİ ARAÇLAR — İSTEK TÜNELİ

Gelen bir istek, cevaba dönene kadar sıralı bir tünelden geçer. Her katman tek bir soruya bakar:

İstek gelir

- Guard "sen kimsin, yetkin var mı?" → hayırsa 401/403
- Pipe "gönderdiğin veri geçerli mi?" → hayırsa 400
- Controller "hangi işi istiyorsun?"
- Service iş kuralları + veritabanı
- Interceptor süreyi ölç, logla, cevabı biçimlendir
- Filter yolda hata çıktıysa yakala, düzgün cevaba çevir

Cevap döner

Bu projedeki karşılıkları:

```
// Bu sınıftaki tüm uçlar /work-orders adresi altında toplanıyor
@Controller('work-orders')
// İki bekçi: önce "giriş yapmış mı", sonra "yetkisi var mı" kontrol
ediliyor.
// Bu satır sayesinde her metoda ayrı ayrı kontrol yazmak gerekmiyor.
@UseGuards(JwtAuthGuard, RolesGuard)
export class WorkOrdersController {

    // POST /work-orders/1042/assign adresine gelen istekleri bu metot
    karşılıyor
    @Post(':id/assign')
    // Yalnızca bu iki rol atama yapabilir. Teknik personel gelirse 403
    alır.
    @Roles('ADMIN', 'OPERATIONS')
    assign(
        // Adresteki "1042" metnini gerçek sayıya çeviriyor.
        // Çeviremezse (örn. "abc") istek buraya hiç ulaşmadan 400 dönüyor.
        @Param('id', ParseIntPipe) id: number,

        // İsteğin gövdesindeki JSON. Zod şemasıyla doğrulanıyor;
        // eksik veya hatalı alan varsa metot hiç çalışmıyor.
        @Body() dto: AssignWorkOrderDto,
    ) {
        // Controller'da iş kuralı YOK – işi servise devredip cevabı
        döndürüyor
        return this.service.assign(id, dto.assigneeId);
    }
}
```

Dört aracın ne yaptığı:

- **Dependency Injection:** Bir sınıfın ihtiyaç duyduğu şeyi kendisi yaratmaz (`new PrismaService()`), Nest verir. Faydası testte görülür: testte gerçek yerine sahtesini verirsiniz, sınıf farkı anlamaz.
- **Guard:** Kapıdaki güvenlik. Yetkisizse istek servise **hiç ulaşmaz**.
- **Pipe:** Gelen veriyi kontrol eder ve dönüştürür. Yukarıdaki `ParseIntPipe` , URL'den gelen `"1042"` metnini sayıya çevirir; çeviremezse 400 döner ve kodunuz hiç çalışmaz.
- **Filter:** Kodun neresinde hata çıkarsa çıksın yakalar. `try/catch` yazmayı unutsanız bile uygulama çökmez, kullanıcı düzgün bir hata mesajı alır (E.7).

3. PROJEDEKİ YERİ — İKİ UYGULAMA, AYNI ÇATI

- `apps/api` : Kullanıcıların istek attığı, hızlı cevap dönen ana sunucu.
- `apps/worker` : Zamanlanmış ve ağır işleri arka planda çözen süreç (C.6).

İkisi de NestJS olduğu için **aynı servisleri, aynı iş kurallarını ve aynı veritabanı katmanını** paylaşıyor. Worker'da bir iş emri kapatıldığında, API'den kapatıldığında kuralların **birebir aynısı** çalışıyor.

4. ÖDEVİN §14 MADDESİ — SERVİS YAŞAM DÖNGÜSÜ

Ödev, Transient / Scoped / Singleton kavramlarının doğru kullanıldığını görmek istiyor. Bunlar bellekteki nesnelerin **ne kadar yaşayacağını** belirler:

Nest	Ne demek	Bu projede
<code>DEFAULT</code> (singleton)	Uygulama boyunca tek kopya	Yapılandırma, <code>Clock</code> , SLA politikaları, factory
<code>REQUEST</code> (scoped)	Her istekte yeni , istek bitince silinir	İsteğe özel bağlam
<code>TRANSIENT</code>	Her kullanımda yeni	Kullanılmıyor — gerekçesiz kullanılmaz

⚠ **Neden hayati:** İstek bazlı veriyi singleton bir serviste tutarsanız iki kullanıcının verisi karışır (C.16'daki Ahmet/Mehmet örneği). Bu hata tek kullanıcı testte **hiç görünmez**, yük altında ortaya çıkar ve kurumsal bir sistemde yanlış kişinin verisini göstermek demektir.

Bu projede aktif kullanıcı bilgisi scoped servis yerine `nestjs-cls` ile taşınıyor (C.16) — daha güvenli ve daha performanslı.

Savunma cümlesi: *”Düz Express’in kurlsızlığı yerine kurumsal mimari araçlarını hazır sunan NestJS’i seçtim. En önemlisi, ödevin 14. maddesinde istenen servis yaşam döngülerini Node tarafında eksiksiz uygulayabilen yaygın çatı Nest olduğu için bu tercih teknik olarak zorunluydu.”*

C.2 Next.js

Nedir: React’in framework’ü. **Ne işe yarar:** React tek başına bir web sitesi değildir; “veriye göre ekranı çiz” işini yapan bir kütüphanedir. Ayakta durması için **paketleyici** ve **yönlendirici (router)** gerekir. Next ikisini de içinde barındırır. **Bu projede nerede:** Tüm arayüz (`apps/web`) — 12 ekran. **Ödevdeki karşılığı:** React + Vite + React Router.

ÖDEVİN İSTEDİĞİ ÜÇ AYRI ÜRÜN NEYDİ

Gerçek hayat: Bir mutfak kuruyorsunuz. Ocak, davlumbaz ve tezgâhı üç ayrı markadan alırsanız her biri iyi olabilir — ama ölçüleri tutturmak, birinin yeni modeli çıkınca diğerine uyup uymadığını takip etmek sizin işiniz olur. Hazır mutfak setinde bu uyumu **üretici garanti eder**.

Parça	Ne işi yapar	Onsuz ne olur
React	Arayüzü küçük parçalara (component) bölerek yazmayı sağlar. Veri değiştiğinde tüm sayfayı değil yalnızca değişen kısmı günceller	Ekran çizilemez
Vite	Kod yazarken tarayıcıyı anında günceller; yayına çıkarken kodu tarayıcının anlayacağı, sıkıştırılmış hâle çevirir	Tarayıcı TypeScript ve JSX’i anlamaz
React Router	Adres → ekran eşlemesi. <code>/is-emirleri/42</code> adresine hangi bileşenin geleceğini yönetir, sayfa yenilenmeden geçiş sağlar	Uygulama tek ekrandan ibaret kalır

NEXT.JS NEDEN ÜÇÜNÜN YERINE GEÇİYOR

Next.js zaten **React’i barındırır**; diğer ikisinin işini kendi içine gömülü mimarilerle çözer:

Vite’in yerine kendi derleyicisi. Geliştirme sunucusu ve yayın derlemesi aynı araçtan gelir.

React Router’in yerine dosya sistemi tabanlı yönlendirme. Klasör yapısının kendisi rotadır — ayrıca rota tablosu yazılmaz:

```
apps/web/app/
├─ (auth)/login/page.tsx      → /login          · giriş ekranı
├─ (protected)/
│  └─ ister
│     ├─ dashboard/page.tsx   → /dashboard
│     └─ is-emirleri/
│        └─ page.tsx          → /is-emirleri      (liste
ekranı)
│           └─ yeni/page.tsx   → /is-emirleri/yeni (yeni kayıt
formu)
│              └─ [id]/
│                 └─ page.tsx   → /is-emirleri/1042 (detay
ekranı)
│                    └─ duzenle/page.tsx → /is-emirleri/1042/duzenle
│                       └─ lokasyonlar/page.tsx → /lokasyonlar
│                          └─ bildirimler/page.tsx → /bildirimler
└─ not-found.tsx              → bulunamayan adreslerde açılan 404
ekranı

# Parantezli klasörler – (auth) ve (protected) – adreste GÖRÜNMEZ.
# Yalnızca grupta yaparlar: (protected) altındaki tüm sayfalar
# tek bir yerde yazılan oturum kontrolünden geçer.
```

★ Parantezli klasörler ((auth), (protected)) **adrese girmez**; yalnızca grupta yaparlar. Bu projede işe yarayan yanı şu: (protected) altındaki tüm sayfalar **tek bir yerde** yazılan oturum kontrolünden geçiyor. Ödevin §22’de istediği *“protected route kullanılmalıdır”* maddesi böylece her sayfaya ayrı kontrol yazmadan karşılanıyor.

Üstüne ikisinin de vermediklerini verir: sunucuda render (ilk açılış hızı), görsel optimizasyonu, yerleşik önbellekleme.

DÜRÜST TARAFI

Küçük ve tamamen istemci taraflı bir panel için Vite + React Router daha hafif ve öğrenmesi daha kolaydır. Next.js daha kuralcıdır. Bu projede Next seçildi çünkü sistem uzun ömürlü olacak ve **üç ayrı paketin sürüm uyumunu yıllarca elle takip etmek**, projeyi devralacak geliştiriciyi zorlar.

⚠ **Önemli:** React yine kullanılıyor. Next.js React'in framework'ü olduğu için ödevin *"React ile frontend geliştirme"* şartı doğrudan karşılanıyor.

C.3 Prisma

Nedir: ORM — veritabanıyla konuşmayı yapan katman. **Ne işe yarar:** Ham SQL yerine temiz kod yazmanızı sağlar, veritabanı şemasını yönetir ve tip güvenliği verir. **Bu projede nerede:** Yalnızca NestJS tarafında. Next.js Prisma'yı **hiç görmez** — veritabanına tek kapıdan girilir. **Ödevdeki karşılığı:** Entity Framework Core.

PRISMA NEREDE DURUYOR

API (NestJS) dış dünyaya veri sunan kapımızsa, **Prisma** o kapının arkasında PostgreSQL'e gidip veriyi hızlı, hatasız ve TypeScript uyumlu şekilde çekip getiren işçidir.

Prisma kullanmasaydık, API fonksiyonunun içine uzun uzun SQL kodunu elle yazıp çalıştırmak gerekirdi. Prisma sayesinde SQL yerine **temiz ve okunabilir kod** yazıyoruz; o kodu arkada veritabanının anladığı SQL'e kendisi çeviriyor.

PRISMA'NIN DÖRT KRİTİK GÖREVI

1. Tür güvenliği (type safety) `where`, `id`, `isActive` gibi alanları yazarken kod editörü otomatik tamamlamalar sunar. Yanlış bir harf yazarsanız kod anında uyarır — yani hatayı **projeyi çalıştırmadan önce**, derleme aşamasında görürsünüz.

2. SQL yazmadan veri çekmek ve ilişkileri yönetmek Karmaşık SQL sorguları yazmak zorunda kalmazsınız. Prisma, yazdığınız TypeScript kodunu arka planda en optimize SQL sorgusuna dönüştürür.

3. Şemayı tek dosyada toplamak `schema.prisma`, Prisma'nın kalbidir. Tüm veritabanı mimarisini tek yerde topladığınız, okunması kolay bir dosyadır. Üç şeyi tanımlar:

- **Datasource:** Hangi veritabanını kullandığınız ve adresi
- **Generator:** Hangi dil için kod üretileceği
- **Models:** Tablolar, kolonlar, veri tipleri ve tablolar arası ilişkiler

```
// 1) HANGİ VERİTABANI – bağlantı adresi koda yazılmıyor,
// ortam değişkeninden okunuyor (local ile canlı farklı olsun diye)
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// 2) NE ÜRETİLECEK – Prisma bu şemadan TypeScript kodu üretiyor
generator client {
  provider = "prisma-client-js"
}

// 3) TABLOLAR
model Location {
  id      Int      @id @default(autoincrement()) // birincil anahtar,
otomatik artar
  name    String   @unique                       // iki lokasyon aynı
adı alamaz
  isActive Boolean @default(true)                // pasife alınca false
olur
  assets  Asset[]  // bu lokasyondaki
varlıklar
}

model Asset {
  id      Int      @id @default(autoincrement())
  name    String
  locationId Int    // hangi lokasyona ait
  // İki tabloyu bağlayan tanım: locationId, Location tablosundaki id'yi
gösteriyor
  location Location @relation(fields: [locationId], references: [id])
}
```

★ **Devralınabilirlik açısından belirleyici olan nokta:** Entity Framework Core’da veri modelini görmek için 30 sınıf dosyası gezersiniz. Prisma’da tek dosya açarsınız, tüm model önünüzdedir.

4. Veritabanı değişikliklerini (migration) yönetmek Yeni tablo eklemek veya kolon değiştirmek gerektiğinde Docker konteynerine girip elle SQL çalıştırmazsınız. Akış şu:

1. `schema.prisma` dosyasında değişikliği yaparsınız
2. `npx prisma migrate dev` komutunu çalıştırırsınız
3. Prisma otomatik olarak `CREATE TABLE ...` gibi SQL üretir, PostgreSQL’e uygular ve bu değişikliği bir **geçmiş dosyası** olarak kaydeder

Bu geçmiş dosyaları git’e girer. Siz, DevOps ve CI **aynı SQL’i** çalıştırır; “bende tablo var sende yok” durumu oluşmaz.

⚠ ZOD İLE PRISMA İLİŞKİSİ — YÖN ÖNEMLİ

Yaygın bir yaklaşım, Prisma modellerinden **otomatik Zod şeması üretmektir** (`zod-prisma` gibi araçlar). **Bu projede bilerek bunu yapmıyoruz.**

Sebebi: o yön, API sözleşmenizi veritabanı şemanız hâline getirir. Kullanıcı tablosunda `passwordHash` varsa üretilen şemada da olur ve dışarı sızma riski doğar. Ödev §13 tam olarak bunu yasaklıyor: *“Entityler doğrudan API response olarak dönmemelidir.”*

Bizim yönümüz tersi: Şemayı `packages/contracts` içinde Zod ile yazıyoruz, Prisma’nın `select` ifadesini o şemadan türetiyoruz. Böylece veritabanı yalnızca cevabın ihtiyaç duyduğu kolonları çekiyor ve çıkışta fazla alan kırılıyor. Sözleşme veritabanına değil, **ihtiyaca** göre şekilleniyor.

ÖZET MİMARİ AKIŞ

```
[ Tarayıcı ]
  ↓
[ Next.js Frontend ]
  ↓ (API çağrısı)
[ NestJS Backend ]
  ↓ (Prisma Client)
[ Docker – PostgreSQL ]
```


Prisma burada NestJS'in veritabanıyla konuşan dili hâline gelir. Docker ise veritabanının bilgisayarınızda temiz bir kutu içinde stabil çalışmasını sağlar.

C.4 Zod

Nedir: Şema tanımlama ve doğrulama kütüphanesi. **Ne işe yarar:** Bir veri şeklini **bir kez** tanımlarsınız; o tanımdan hem çalışma anı doğrulaması, hem TypeScript tipi, hem OpenAPI dokümanı üretilir. **Bu projede nerede:** Her API girişinde doğrulama · `packages/contracts` içinde paylaşılan tipler · her API çıkışında fazla alanın kırılması · frontend form doğrulaması (aynı şema). **Ödevdeki karşılığı:** FluentValidation — ama o yalnızca doğrular; tip ve OpenAPI ayrı iştir.

ZOD'UN ROLÜ

Zod, bu mimaride ”**dışarıdan gelen verinin güvenlik muhafızı**” rolündedir.

Prisma veritabanından çıkan verinin tipini (**içeriği**) korurken; Zod, kullanıcının formdan gönderdiği veya dışarıdan gelen verinin (**dışarıyı**) kurallara uygun olup olmadığını kontrol eder.

AKIŞ — ZOD TAM OLARAK NEREDE DEVREYE GIRIYOR

⚠️ Zod ayrı bir servis veya sunucu **değildir**. NestJS'in **içindeki bir katmandır** (ValidationPipe). Yani kapıda ayrı bir bina yok, kapının içinde bir bekçi var:

```
[ Kullanıcı formu ]
  ↓
[ NestJS ]
  ├── 1. Zod katmanı → "biçim doğru mu?" (hayırsa istek burada biter)
  ├── 2. Guard       → "yetkin var mı?"
  ├── 3. İş kuralları → "bu işlem yapılabilir mi?"
  └── 4. Prisma       → veritabanına yaz
  ↓
[ Docker – PostgreSQL ]
```

Zod katmanı: E-posta gerçekten @ içeriyor mu? Başlık boş mu? Öncelik tanımlı dört değerden biri mi? Veri onaylanmazsa istek **kod içine bile girmeden** “hatalı format” cevabıyla geri döner.

Prisma katmanı: Zod’dan temiz çıkan güvenli veri Prisma’ya teslim edilir, o da SQL’e çevirip kaydeder.

İKİ FARKLI DENETİMİ KARIŞTIRMAMAK

Soru	Kim cevaplar
”Başlık boş mu, 2000 karakteri aştı mı?” — biçim	Zod
”Bu varlık için talep açılabilir mi?” — iş kuralı	Kurallar katmanı

Bu ayırım yapılmazsa iş kuralları doğrulama koduna karışır ve aynı kural mobil uygulamadan gelen istekte çalışmaz.

C.5 PostgreSQL

Nedir: İlişkisel veritabanı. **Ne işe yarar:** Veriyi tablolarda tutar; ilişkileri ve tutarlılığı **kendisi** garanti eder (foreign key, unique constraint, transaction). **Bu projede nerede:** Tüm veri. Ayrıca metin araması `pg_trgm` + GIN index ile veritabanının içinde çözülüyor — ayrı arama motoru gerekmiyor. **Ödevdeki karşılığı:** Aynı; ödev zaten PostgreSQL istiyordu.

PostgreSQL verileri Excel tabloları gibi satır ve sütunlarda tutar. En büyük gücü ise bu tablolar arasındaki ilişkileri ve kuralları **asla esnetmeden** korumasıdır.

FOREIGN KEY (YABANCI ANAHTAR)

Gerçek hayat: Nüfus müdürlüğü, üzerine kayıtlı aracı olan bir kişiyi sistemden silmenize izin vermez. Önce aracı devredersiniz, sonra kaydı kapatırsınız. Kural memurda değil, **sistemin kendisinde** olduğu için kimse “acelem var” diye atlayamaz.

Bu projede:

```

model Asset {
  id          Int          @id @default(autoincrement()) // varlığın kendi
numarası

  // Bu varlığın hangi lokasyona ait olduğunu tutan alan
  locationId Int

  // İki tabloyu birbirine bağlayan tanım.
  // 🚫 onDelete: Restrict → üzerinde varlık olan bir lokasyon SİLİNEMEZ.
  //   Koruma uygulama kodunda değil, veritabanının kendisinde duruyor.
  location    Location @relation(fields: [locationId], references: [id],
                                onDelete: Restrict)
}

```

`onDelete: Restrict` sayesinde, üzerinde varlık olan bir lokasyon silinemez. Bu koruma **uygulama kodunda değil, veritabanında** durur — arayüzde veya API’de hata olsa bile veri bozulmaz.

⚠ Bu yüzden zaten lokasyonları **silmiyoruz**, pasife alıyoruz (E.9 → soft delete). Foreign key, kazara silmeye karşı ikinci savunma hattı.

UNIQUE CONSTRAINT (BENZERSİZLİK KURALI)

Gerçek hayat: Aynı koltuğa iki bilet satılamaz. Gişe memuru dalgın olsa bile sistem ikinci bileti basmaz.

Bu projede en kritik kullanımı — mükerrer bildirim engelleme:

```

model Notification {
  id      Int    @id @default(autoincrement()) // bildirimin kendi
numarası
  userId  Int                                // bildirim kime
gidecek
  eventKey String                             // hangi olay:
"SLA_BREACH:1042"

  // 🚫 Korumanın kalbi: aynı kullanıcıya aynı olay için İKİNCİ bir
bildirim
  //   yazılamaz. Kural uygulama kodunda değil, veritabanının
kendisinde.
  @@unique([userId, eventKey])
}

```

Neden uygulama kodunda kontrol yetmiyor:

```

// 🚫 YANLIŞ YOL: önce bak, yoksa ekle

// 1) "Bu bildirim daha önce yazılmış mı?" diye soruyoruz
const varMi = await prisma.notification.findFirst({ where: { userId,
eventKey } });

// 2) Yazılmamışsa ekliyoruz
if (!varMi) await prisma.notification.create({ ... });

// ⚠️ İki arka plan işi aynı anda 1. satırı çalıştırırrsa İKİSİ DE "yok"
//   cevabı alır ve ikisi de ekler. Kullanıcı aynı olay için iki
bildirim görür.

```

İki arka plan işi aynı anda kontrol ederse ikisi de “kayıt yok” görür ve ikisi de ekler. Buna **yarış koşulu (race condition)** denir. Veritabanının kuralı bu yarış yapısal olarak kaybettirir: ikincisi hata alır, kod onu sessizce yutar.

TRANSACTION (İŞLEM BÜTÜNLÜĞÜ)

”Ya hep ya hiç” kuralıdır. Birbirine bağlı işlemler ya tamamen olur ya hiç olmaz.

Bu projedeki somut senaryo:

```
// "Ya ikisi birden ya hiçbiri" garantisi
await prisma.$transaction(async (tx) => {

  // 1) İş emrini kapalı olarak işaretle
  await tx.workOrder.update({ where: { id }, data: { status: 'CLOSED' }
});

  // 2) Aynı işlemde geçmişe de yaz: kim, nereden nereye aldı
  await tx.workOrderHistory.create({
    data: { workOrderId: id, from: 'RESOLVED', to: 'CLOSED', byUserId },
  });

  // Arada hata çıkarsa ikisi de geri alınır. Böylece "kapatılmış ama
  // kim kapattığı belli olmayan" bir kayıt asla oluşmaz.
});
```

Transaction olmasaydı ve tam ikisi arasında hata çıksaydı: iş emri “Kapatıldı” görünürdü ama **kimin, ne zaman kapattığı kaydı olmazdı**. Denetim açısından bu, kaydın hiç olmamasından kötüdür — sistem doğru görünür ama izi yoktur.

⚠ **Transaction’ın sınırı.** Bir PostgreSQL transaction’ı yalnızca **kendi veritabanındaki** işlemleri geri alabilir. Farklı bir kurumun sistemine (örneğin bir bankaya) gönderilmiş işlemi geri alamaz; orada **telafi işlemi** (iade) devreye girer ve buna *saga* denir. Bu projedeki tüm transaction senaryoları tek veritabanı içinde kaldığı için tam garanti sağlanıyor.

PG_TRGM + GIN INDEX İLE ARAMA

Problem: İş emri listesinde metin araması var. `WHERE title LIKE '%pompa%'` yazarsanız PostgreSQL **tüm tabloyu satır satır tarar**. 100 kayıta fark etmez, 500 bin kayıta ekran donar.

Index nedir: Kitabın arkasındaki dizin. Ama varsayılan index (B-tree) “şununla **başlayanları**” bulmakta hızlıdır; **ortasında geçenleri** bulamaz — yani `%pompa%` aramasında hiç işe yaramaz.

Çözüm iki parçalı:

- **pg_trgm** : Kelimeleri üçer harflik parçalara böler (“pompa” → “pom”, “omp”, “mpa”). Böylece “ortada geçiyor mu” sorusu “şu parçaları içeriyor mu” sorusuna dönüşür. Yazım hatasını da tolere eder (“jeneratör” / “jenerator”).
- **GIN index**: Bu parçaları aranabilir hâle getiren index türü.

```
-- 1) PostgreSQL'in kelime parçalama eklentisini aç (yoksa kur).
-- Bu eklenti "pompa" kelimesini "pom","omp","mpa" parçalarına
bölüyor.
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- 2) İş emri başlıkları için arama index'i oluştur.
-- GIN = bu parçaları hızlı aramaya uygun index türü.
-- Bu index olmadan "%pompa%" araması tüm tabloyu satır satır tarardı.
CREATE INDEX work_order_title_trgm_idx
ON "WorkOrder" USING GIN (title gin_trgm_ops);
```

★ **Kazanç**: Normalde bu iş için Elasticsearch gibi ayrı bir arama motoru kurulur — ayrı sunucu, ayrı bakım, ayrı maliyet. Burada PostgreSQL’in kendi yetenekleriyle çözülüyor.

Ne zaman yetmez: Alaka sıralaması ve çok dillilik gereken gerçek tam metin aramada PostgreSQL FTS veya ayrı motor gerekir — bu proje için gereksiz.

C.6 BullMQ + Redis

Nedir: Node için iş kuyruğu; verilerini Redis’te tutar. **Ne işe yarar**: **Gecikmeli** (“2 saat sonra hatırlat”) ve **tekrarlayan** (“her 15 dakikada bir tara”) işleri yönetir. Hata alırsa otomatik yeniden dener. **Bu projede nerede**: Ödevin dört zamanlanmış işi. **Ödevdeki karşılığı**: Hangfire.

NEDEN GEREKLİ

Gerçek hayat: Bir restoranda garson siparişi alıp mutfağa asıyor ve **masaya dönüyor**. Yemeğin pişmesini mutfağın önünde beklemiyor — beklerse diğer masalar hizmet alamaz.

Yazılımda: API isteğe cevap verir ve biter. Ama bazı işler kimse bir şey sormasa da çalışmalı: gece 03:00’te SLA’sı geçmiş iş emirlerini taramak gibi. Bu işleri API içinde yaparsanız, o sırada gelen kullanıcı istekleri bekler.

BİLEŞENLER

- **BullMQ:** İşleri sıraya koyar, zamanı geleni dağıtır, hata alanı yeniden dener.
- **Redis:** Diske değil doğrudan belleğe yazan çok hızlı veri deposu. Kuyruk burada durur.

⚠ Redis bu projede yalnızca kuyruk için değil, **hız sınırı sayacı** ve **dağıtık kilit** için de kullanılıyor — tek amaçlı bir bağımlılık değil.

İKİ İŞ TÜRÜ — KODLA

```
// GECİKMELİ İŞ – "şimdi değil, ileri bir saatte çalış"
await queue.add(
  'sla-reminder', // işin adı
  { workOrderId: 1042 }, // işe verilen tek bilgi:
  KİMLİK NUMARASI // (iş emrinin tamamı
  gönderilmiyor – // kuyrukta beklerken
  eskiyebilirdi)
{
  // Kaç milisaniye sonra çalışsın. Hatırlatma zamanına kalan süre.
  delay: remindAt.diffNow().milliseconds,

  // İşin benzersiz kimliği. Aynı kimlikle ikinci kez eklenirse
  // kuyruk onu kabul etmiyor – mükerrer hatırlatma olmuyor.
  jobId: `sla-reminder-1042`,

  attempts: 3, // hata alırsa 3
  kez dene
  backoff: { type: 'exponential', delay: 5000 }, // her denemede
  daha uzun bekle
},
);

// TEKRARLAYAN İŞ – kimse tetiklemese de düzenli çalışır
await queue.add('sla-breach-scan', {}, {
  // Cron biçimi: "her 15 dakikada bir"
  repeat: { pattern: '* / 15 * * * *' },
});
```

PROJEDEKİ DÖRT İŞ

İş	Türü	Ne yapar
SLA hatırlatma	Gecikmeli	Süre dolmadan önce ilgili personele uyarı üretir
SLA ihlali taraması	Tekrarlayan	Süresi geçmiş açık iş emirlerini bulur, ihlal işaretler
Günlük operasyon özeti	Tekrarlayan	Gece bir kez: açık iş sayısı, ihlaller, personel iş yükü
Arşiv adayı belirleme	Tekrarlayan	Eski kapalı kayıtları arşivlenebilir işaretler

EN KRITİK KURAL: IDEMPOTENCY (AYNI İŞ İKİ KEZ ÇALIŞSA DA SONUÇ DEĞİŞMEZ)

Gerçek hayat: Asansör düğmesine beş kez basmak asansörü beş kez çağırılmaz. Sistem “zaten çağırıldı” durumunu biliyor.

Neden risk: Sunucu yeniden başlarsa veya iş hata alıp yeniden denenirse aynı iş ikinci kez çalışır. Önlem alınmazsa kullanıcı **iki bildirim** alır.

Bu projede üç katmanlı korunuyor:


```
// Bu iş saatler önce kuyruğa bırakılmıştı. Şimdi çalışıyor.
async handleSlaBreach(workOrderId: number) {

  // 1) İş emrinin GÜNCEL hâlini veritabanından oku.
  //   Kuyruğa girerken gönderilen bilgiye güvenmiyoruz – o bilgi
  //   eskimiş olabilir.
  const wo = await this.prisma.workOrder.findUnique({ where: { id:
workOrderId } });

  // 2) Bu arada iş kapanmış veya iptal edilmişse hiçbir şey yapma,
  //   sessizce çık
  if (!wo || wo.status === 'CLOSED' || wo.status === 'CANCELLED') return;

  // Zaten ihlal olarak işaretlenmişse tekrar işaretleme (iş ikinci kez
  //   çalışmış olabilir)
  if (wo.slaBreach) return;

  // 3) İşaretleme ve bildirim BİRLİKTE yazılıyor.
  //   Bildirim tablosundaki benzersiz kural, aynı olay için ikinci
  //   bildirimin yazılmasını veritabanı seviyesinde engelliyor.
  await this.prisma.$transaction(async (tx) => {
    await tx.workOrder.update({ where: { id: wo.id }, data: {
slaBreach: true } });
    await tx.notification.create({
      data: { userId: wo.assigneeId, eventKey: `SLA_BREACH:${wo.id}` },
    });
  });
}
```

★ **1. maddedeki kural en önemlisi:** İş, parametre olarak **yalnızca kimlik numarası** alıyor; iş emrinin durumunu kendisi veritabanından okuyor. Nesnenin tamamı parametre olarak geçirilseydi, iş kuyrukta beklerken durum değişmiş olabilirdi ve iş **eskimiş veriyle** karar verirdi.

Ödev §15 bunu doğrudan söylüyor: *”Job parametresi olarak büyük nesneler yerine tanımlayıcı değerler kullanılmalıdır.”*

DASHBOARD GÜVENLİĞİ

Ödev *”Hangfire Dashboard yetkisiz erişime açık bırakılmamalıdır”* diyor. Bull Board arayüzü de aynı şekilde yalnızca yönetici rolüne açık — Guard arkasında.

C.7 Testcontainers

Nedir: Test başlarken Docker üzerinde **gerçek** bir veritabanı konteyneri ayağa kaldıran, test bitince silen kütüphane. **Bu projede nerede:** Tüm entegrasyon testleri. **Ödevdeki karşılığı:** Ödevin *"EF Core InMemory provider kullanılmamalıdır"* maddesinin olumlu karşılığı.

NEDEN SAHTE VERITABANI KULLANILMIYOR

Gerçek hayat: Uçuş simülatöründe pilot mükemmel iniş yapıyor. Ama simülatörde yan rüzgâr, buzlanma ve motor arızası modellenmemişse, o pilotun “iniş yapabildiği” **kanıtlanmış olmaz**. Test yeşil yanar, gerçek uçuşta sorun çıkar.

Yazılımda: Sahte (in-memory) veritabanı gerçek bir SQL motoru değildir — bellekteki bir sözlüktür. Şunları **uygulamaz**:

Test edilmek istenen	Sahte veritabanında
Unique constraint ihlali	Yakalanmaz — ikinci kayıt yazılır
Foreign key ihlali	Yakalanmaz
Transaction geri alma	Gerçekten geri almaz
Eşzamanlılık çakışması	Kavram yok
<code>pg_trgm</code> metin araması	Çalışmaz

Ödev §23 tam da bu beşini test etmeyi istiyor. Yani sahte veritabanıyla **testler yeşil yanar ama hiçbir şey ölçmez** — üstelik yanlış bir güven verir.

⚠ Bizim stack'teki aynı tuzak: **Prisma Client'ı mock'lamak**. Yapmıyoruz.

NASIL ÇALIŞIYOR

```
let container: StartedPostgreSqlContainer;

// beforeAll = "testler başlamadan önce bir kez çalış"
beforeAll(async () => {
  // 1) Docker'da gerçek bir PostgreSQL konteyneri başlat
  container = await new PostgreSQLContainer('postgres:18-alpine').start();

  // 2) Uygulamaya "veritabanının burada" de.
  // Adres rastgele bir porta çıkıyor, bilgisayardaki diğer
  // veritabanlarıyla çakışmıyor.
  process.env.DATABASE_URL = container.getConnectionUri();

  // 3) Tabloları oluştur – gerçek migration dosyaları çalışıyor
  execSync('npx prisma migrate deploy');
}, 60_000); // konteyner inmesi uzun sürebilir, 60 saniye süre tanı

// afterAll = "tüm testler bittikten sonra çalış"
afterAll(async () => {
  // Konteyneri kapat ve sil. Bilgisayarda hiçbir kalıntı bırakma.
  await container.stop();
});
```

Dört adım:

1. **Konteyner ayağa kalkar.** Belirtilen imaj indirilir, boş bir veritabanı başlar.
2. **Rastgele port atanır.** Bilgisayarda çalışan başka bir PostgreSQL varsa çakışma olmaz — bu projede zaten paralel bir proje çalışıyor, dokunulmuyor.
3. **Testler gerçek motora karşı koşar.** Migration'lar uygulanır; benzersizlik, foreign key ve transaction canlıdaki gibi davranır.
4. **Her şey silinir.** Test başarılı da olsa başarısız da olsa konteyner kapanır; artık veri kalmaz.

BU PROJEDE TEST EDİLEN SENARYOLAR

```
it('aynı olay için ikinci bildirim yazılamaz', async () => {
  // 1) Birinci bildirimi yaz – bu başarılı olmalı
  await createNotification(userId, 'SLA_BREACH:1042');

  // 2) Aynı bildirimi tekrar yazmayı dene – bu HATA VERMELİ
  await expect(
    createNotification(userId, 'SLA_BREACH:1042'),
  ).rejects.toThrow(/Unique constraint/);
  // Gelen hata gerçek PostgreSQL'in benzersizlik hatası.
  // Sahte bir veritabanında bu kural hiç uygulanmaz, test yeşil yanar
  // ve koruma hiç doğrulanmamış olurdu.
});
```

Bu test sahte veritabanında **her zaman geçerdi** ve mükerrer bildirim koruması hiç doğrulanmamış olurdu.

★ **Koruma testi disiplini:** Bir korumayı test eden test yazıldığında, koruma geçici olarak kaldırılıp testin **kırmızıya döndüğü gözle görülür**. Dönmüyorsa test korumayı değil başka bir şeyi ölçüyordur.

C.8 dependency-cruiser

Nedir: Kod içindeki `import` ilişkilerini kurallara göre denetleyen otomatik mimari denetim aracı. **Bu projede nerede:** Ödev §23'ün mimari testleri. **Ödevdeki karşılığı:** .NET tarafındaki NetArchTest.

ÇÖZDÜĞÜ SORUN

Gerçek hayat: Bir binada yangın kapısı var ve “sürekli kapalı tutulacak” yazıyor. Ama kapı ağır olduğu için biri altına takoz koyuyor — herkes o kapıyı kullanmaya başlıyor. Uyarı levhası duruyor, **kural fiilen ölmüş**.

Kuralı yaşatan şey levha değil, kapının **kendiliğinden kapanan mekanizmasıdır**.

Yazılımda: “Domain katmanı veritabanını tanımaz” kuralı dokümana yazılır, herkes başta uyar. Sonra biri acele eder, domain’e bir Prisma import’u koyar — çalışır, kimse fark etmez. Altı ay sonra domain katmanı Prisma’sız çalışmaz hâle gelmiştir ve mimari

kâğıt üstünde kalmıştır.

NASIL ÇALIŞIYOR

Projenin başında kural dosyası yazılır:

```
// Dosya: .dependency-cruiser.js – mimari kuralların yazılı hâli
forbidden: [
  {
    name: 'domain-altyapiya-bagimli-olamaz',
    severity: 'error', // ihlal = hata, uyarı değil
    // KİM: domain klasöründeki dosyalar
    from: { path: '^packages/domain' },
    // NEYİ ÇAĞIRAMAZ: veritabanı aracı, Nest çatısı veya API klasörü
    to: { path: '(node_modules/@prisma|@nestjs|^apps/api)' },
  },
  {
    name: 'web-veritabanina-erisemez',
    severity: 'error',
    // KİM: arayüz projesi
    from: { path: '^apps/web' },
    // NEYİ ÇAĞIRAMAZ: veritabanı aracını doğrudan
    to: { path: 'node_modules/@prisma' },
    // Sebep: veritabanına yalnızca API üzerinden gidilmeli.
  },
]
]
```

Sonra CI'da çalışır:

```
✘ domain-altyapiya-bagimli-olamaz
  packages/domain/work-order.ts → node_modules/@prisma/client
error Process exited with code 1
```

Build kırmızı yanar, o kod ana dala giremez.

BU PROJEDE DENETLENEN KURALLAR

Kural	Neden
Domain, Prisma ve NestJS'i import edemez	İş kuralları altyapıdan bağımsız kalsın (E.1)
Application, infrastructure'a doğrudan bağlanamaz	Bağımlılık yönü korunsun
<code>apps/web</code> , Prisma'yı import edemez	Veritabanına tek kapıdan girilsin
Controller, Prisma'yı doğrudan çağırılmaz	İş kuralı atlanmasın
Modüller birbirinin iç dosyasına erişemez	Modül sınırı korunsun (E.1)

NEDEN BU ARAÇ

Katman kuralları sözlü kararlar alınır veya yalnızca dokümana yazılırsa altı ay içinde çöğnenir; projeye yeni katılan biri dokümanı okumamış olabilir.

Kural ancak kapıya dönüşürse yaşar.

Ödevin §23'te istediği *"katman bağımlılıklarını doğrulayan testler"* maddesi tam olarak budur — ve bu, dokümantasyonda anlatılan değil **kod seviyesinde zorlanan** bir mimari demektir.

C.9 TypeScript

Nedir: JavaScript'in üzerine **tip kontrolü** eklenmiş hâli. **Bu projede nerede:** Her yerde — backend, frontend, testler, ortak paketler. **Ödevdeki karşılığı:** C#.

ÇÖZDÜĞÜ GERÇEK SORUN

Gerçek hayat: Bir formda “doğum tarihi” alanına biri `15/03/1990`, biri `1990-03-15`, biri `15 Mart 1990` yazıyor. Form bunu kabul ediyor çünkü hepsi “metin”. Hata, o veriyle yaş hesaplamaya çalıştığınızda — yani **çok sonra** ortaya çıkıyor.

Yazılımda: Düz JavaScript'te hatayı ancak kod **çalışırken** görürsünüz:

```
// JavaScript – hata vermiyor ama yanlış çalışıyor
const gun = isEmri.slaBitis; // Alanın gerçek adı slaDueAt. Yanlış
yazdık.
console.log(gun);           // Sonuç: undefined. Uyarı yok, çökme yok.
                             // Bu hatayı büyük ihtimalle KULLANICI
                             bulacak.
```

```
// TypeScript – aynı hatayı yazarken yakalıyor
const gun = isEmri.slaBitis;
//                               ~~~~~ Editör anında altını çiziyor:
//                               "WorkOrder tipinde slaBitis diye bir alan yok.
//                               slaDueAt demek mi istediniz?"
// Yani hata kullanıcıya değil, geliştirme sırasında sana gidiyor.
```

Fark şu: birinde hatayı **kullanıcı** bulur, diğesinde **derleyici**.

BU PROJEDE NEDEN ÖZELLİKLE KRİTİK

Sistem aynı veriyi üç yerde kullanıyor: backend, web arayüzü, ileride mobil. Bir alanın adı değiştiğinde diğerlerinin de güncellenmesi gerekiyor.

```
// packages/contracts – tek tanım
export type WorkOrderListItem = z.infer<typeof WorkOrderListItem>;
```

API’de `slaDueAt` alanı `dueAt` olarak değişirse, arayüz **derlenmiyor**. Hata ekranda `undefined` olarak değil, geliştirme sırasında kırmızı çizgi olarak çıkıyor (E.3).

”STRICT” MODU NE KATIYOR

En katı ayar açık. En çok işe yarayan kısmı **boş değer kontrolü**:

```
// strict modu açık – bu kod DERLENMİYOR
function ata(wo: WorkOrder) {
  return wo.assigneeId.toString();
  // ~~~~~ "assigneeId boş (null) olabilir" uyarısı.
  // ⚠ İş emri henüz kimseye atanmamışsa bu alan boş olur; boş bir değer
  // üzerinde işlem yapmak uygulamayı çalışma anında çökertir.
}

// Derleyici, boş olma ihtimalini ele almanı ZORUNLU kılıyor
function ata(wo: WorkOrder) {
  // Atanmamış iş emri durumunu açıkça düşünmek zorundasın
  if (wo.assigneeId === null) throw new NotAssignedError(wo.id);

  // Buradan sonrası güvenli: derleyici artık boş olmadığını biliyor
  return wo.assigneeId.toString();
}
```

★ Bu projede iş emrinin `assigneeId` alanı **atanmamışken boş**. Strict mod, “atanmamış iş emri” durumunu düşünmeyi **zorunlu kılıyor** — ödevin “*atanmamış bir iş emri işleme alınamamalıdır*” kuralı böylece derleyici tarafından hatırlatılıyor.

SÜRÜM KISITI

Projede TypeScript **6** kullanılıyor, 7 değil. Sebep tercih değil kısıt: `typescript-eslint` paketi henüz TS 7’yi desteklemiyor. TS 7’ye çıkmak lint kapısını tamamen devre dışı bırakırdı. Kısıt kalkınca yükseltilecek.

C.10 Docker ve Docker Compose

Nedir: Uygulamayı, çalışması için ihtiyaç duyduğu her şeyle birlikte tek bir pakete koyan sistem. **Ne işe yarar:** “Benim bilgisayarımda çalışıyordu” sorununu ortadan kaldırır. **Bu projede nerede:** Tüm sistemin tek komutla ayağa kalkması. **Ödevdeki karşılığı:** Aynı; ödev §24 zaten Docker ve Docker Compose istiyor.

ÇÖZDÜĞÜ GERÇEK SORUN

Bir uygulama boşlukta çalışmaz: belirli bir Node.js sürümüne, belirli bir PostgreSQL sürümüne, belirli ayarlara ihtiyaç duyar. Geliştiricinin bilgisayarında Node 24, sunucuda Node 18 varsa uygulama sunucuda patlar ve sebebini bulmak günler alabilir.

Docker bu sorunu kökten çözer: uygulamayı **ihtiyaçlarıyla birlikte** paketler. O paket nerede çalıştırılırsa çalıştırılsın içi aynıdır.

TEMEL KAVRAMLAR

Terim	Ne demek	Benzetme
Dockerfile	Paketin nasıl yapılacağıının tarifi: “Node 24 al, şu dosyaları kopyala, derle, şu portu aç”	Yemek tarifi
İmaj (image)	Tarifin çalıştırılmış hâli — donmuş, değişmez paket	Dondurulmuş yemek
Konteyner	İmajın çalışan hâli	Isıtılıp tabağa konmuş yemek
docker-compose.yml	Birden fazla konteyneri birlikte tarif eden dosya	Menü

BU PROJEDE COMPOSE NE YAPIYOR

Sistem dört ayrı parçadan oluşuyor ve hepsinin birlikte çalışması gerekiyor:

1. **PostgreSQL** — veritabanı
2. **API** — NestJS sunucusu
3. **Worker** — arka plan işlerini yapan süreç
4. **Web** — Next.js arayüzü

Compose bu dördünü tek dosyada tarif eder: hangi portlarda çalışacakları, hangi sırayla başlayacakları, birbirlerini nasıl bulacakları. Tek komutla hepsi ayağa kalkar:

```
docker compose up --build
```

Ödev §24 tam olarak bunu şart koşuyor: değerlendirmeyi yapan kişi projeyi indirip tek komutla çalıştırabilmeli.

ÇOK AŞAMALI (MULTI-STAGE) YAPI

Ödev bunu özellikle istiyor. Sebebi şu: uygulamayı **derlemek** için gereken araçlar (derleyiciler, geliştirme paketleri) **çalıştırmak** için gerekmez. Çok aşamalı yapıda önce büyük bir ortamda derleme yapılır, sonra yalnızca sonuç dosyaları küçük bir imaja taşınır. Sonuç: birkaç kat küçük imaj, daha hızlı dağıtım ve daha az güvenlik yüzeyi.

C.11 Vitest

Nedir: Test çalıştırıcı — yazdığınız testleri koştan ve sonucu raporlayan araç. **Bu projede nerede:** İş kuralları, SLA politikaları, durum geçişleri, Factory. **Ödevdeki karşılığı:** xUnit / NUnit / MSTest.

TESTİN GERÇEKTE ÇÖZDÜĞÜ SORUN

Gerçek hayat: Bir binada yangın alarmı var. Amacı yangını **söndürmek** değil; yangın çıktığında **haber vermek**. Ayda bir test edilmezse, gerçek yangında çalışıp çalışmayacağı bilinmez.

Yazılımda: Test yazmanın amacı “kod çalışıyor mu” değildir — o zaten elle bakılır. Asıl amaç: **altı ay sonra biri bu koda dokunduğunda, farkında olmadan bir şeyi bozarsa haberi olsun.**

BU PROJEDE TIPIK BİR TEST

```
describe('İş emri kapatma', () => {

  it('çözüm açıklaması boşsa kapatılamaz', () => {
    // HAZIRLIK: çözülmüş durumda bir iş emri oluştur
    const wo = workOrder({ status: 'RESOLVED' });

    // ÇALIŞTIR ve BEKLE: boş açıklamayla kapatmayı dene, hata fırlatmalı
    expect(() => wo.close('', clock)).toThrow(ResolutionNoteRequired);
  });

  it('RESOLVED olmayan iş emri kapatılamaz', () => {
    // HAZIRLIK: henüz üzerinde çalışılan bir iş emri
    const wo = workOrder({ status: 'IN_PROGRESS' });

    // Açıklama dolu olsa bile durum uygun değil → geçiş hatası
    bekleniyor
    expect(() => wo.close('Tamam', clock)).toThrow(InvalidTransition);
  });
});
// Dikkat: bu testlerde veritabanı, HTTP veya NestJS yok.
// Kural saf bir sınıfta durduğu için test milisaniyede bitiyor.
```

★ Dikkat: bu testlerde **veritabanı yok, HTTP yok, Nest yok**. İş kuralları domain katmanında saf durduğu için (E.1) test milisaniyede koşuyor. Yüzlerce kural testi birkaç saniyede bitiyor.

ZAMANA BAĞLI KURALLARI TEST ETMEK

SLA kuralları “şu an saat kaç” bilgisine bağlı. Gerçek saati beklemek imkânsız:

```
it('SLA süresi geçmiş iş emri ihlal olarak işaretlenir', () => {
  // Saati sabitliyoruz: "şu an 10:00" diyoruz.
  // Gerçek saati beklemeden zamana bağlı kuralı test edebiliyoruz.
  const clock = fixedClock('2026-08-21T10:00:00Z');

  // SLA bitişi 09:00 olan bir iş emri – yani süre bir saat önce dolmuş
  const wo = workOrder({ slaDueAt: '2026-08-21T09:00:00Z' });

  // Sonuç: ihlal edilmiş sayılmalı
  expect(isBreached(wo, clock)).toBe(true);
});
```

`Clock` soyutlaması (E.2 → D harfi) sayesinde teste *"sen şu an şu andasın"* denebiliyor. Kod içinde doğrudan `new Date()` çağırılsaydı bu test yazılamazdı.

ÖDEVİN İSTEDİĞİ TEST ALANLARI

Ödev §23 hangi alanların test edileceğini tek tek sayıyor: Factory Pattern, SLA politikaları, geçerli ve **geçersiz** durum geçişleri, atama kuralları, pasif lokasyon kuralı, mükerrer bildirim engelleme, arka plan işinin kapalı iş emrinde işlem yapmaması.

Ortak özellikleri: hepsi **iş kuralı** ve hepsi veritabanı gerektirmeden test edilebiliyor.

KORUMA TESTİ DISİPLİNİ

⚠ Bir korumayı test eden test yazıldığında, koruma **geçici olarak kaldırılıp** testin kırmızıya döndüğü gözle görülür. Dönmüyorsa test korumayı değil başka bir şeyi ölçüyordur ve size **yanlış bir güven** verir.

C.12 Playwright

Nedir: Gerçek bir tarayıcıyı otomatik olarak sürüp uygulamayı bir kullanıcı gibi kullanan test aracı. **Ne işe yarar:** "Giriş yap → iş emri oluştur → ata → kapat" gibi uçtan uca akışların gerçekten çalıştığını doğrular. **Bu projede nerede:** Kritik kullanıcı yolculukları. **Ödevdeki karşılığı:** Doğrudan istenmemiş; ekran testleri için eklendi.

DIĞER TESTLERDEN FARKI

Birim testleri tek bir kuralı ölçer. Playwright ise **parçaların birlikte çalıştığını** ölçer: arayüz doğru isteği atıyor mu, API doğru cevap veriyor mu, gelen cevap ekranda doğru görünüyor mu, yetkisiz kullanıcı doğru sayfaya yönleniyor mu.

Gerçek hayatta çoğu hata tek tek parçalarda değil, **parçaların birleştiği yerde** çıkar. Playwright tam oraya bakar.

C.13 JWT ve argon2 — kimlik doğrulama

Nedir:

- **JWT (JSON Web Token):** Giriş yapan kullanıcıya verilen, **imzalı** dijital kimlik kartı.
- **argon2:** Şifreyi geri döndürülemez şekilde karıştıran algoritma.

Bu projede nerede: Giriş, yetkilendirme, oturum yenileme. **Ödevdeki karşılığı:** §5.1 — JWT tabanlı kimlik doğrulama, yenileme jetonu, rol bazlı yetkilendirme, pasif kullanıcı engeli.

JWT GERÇEKTE HANGİ SORUNU ÇÖZÜYOR

Gerçek hayat: Otele giriş yaptınız. Resepsiyon her seferinde kimliğinizi kontrol etmiyor; size bir **oda kartı** veriyor. Kart üzerinde hangi odaya, hangi tarihe kadar girebileceğiniz kodlu. Kartı kopyalayıp “süit oda” yazamazsınız çünkü kart **sistem tarafından imzalanmış**.

Yazılımda: Kullanıcı bir kez giriş yapar; sonraki her istekte “sen kimsin” sorusunun yeniden cevaplanması gerekir. Her seferinde şifre sormak mümkün değil, şifreyi saklamak ise tehlikeli.

Jetonun içi şuna benzer:

```
{
  "sub": 42,                // kullanıcının kimlik numarası
  "role": "TECHNICIAN",    // rolü – yetki kontrolü buna bakıyor
  "exp": 1755503600        // son kullanma zamanı; geçince jeton
geçersiz
}
// 🚫 Bu içerik ŞİFRELİ DEĞİL, herkes okuyabilir – sadece imzalı.
//   Bu yüzden jetona TCKN, telefon gibi kişisel veri KONMAZ.
//   ⭐ İmza sayesinde içerik değiştirilemez: biri "role": "ADMIN" yapmaya
//   kalkarsa imza tutmaz ve sunucu jetonu reddeder.
```

Üzerinde oynanamaz: biri `"role": "ADMIN"` yapmaya kalkarsa **imza tutmaz** ve sunucu jetonu reddeder.

⚠ **Sık yapılan hata:** Jetonun içi **şifreli değildir**, yalnızca imzalıdır. Herkes içeriğini okuyabilir. Bu yüzden jetona TCKN, telefon, e-posta gibi kişisel veri **konmaz** — sadece kimlik numarası ve rol.

NEDEN İKİ AYRI JETON

Erişim jetonu (access token) kısa ömürlüdür — dakikalar. Çalınırsa zararı sınırlı kalsın diye. Ama kullanıcıya her 15 dakikada bir giriş yaptırmak da olmaz.

Bu yüzden ikinci bir **yenileme jetonu (refresh token)** verilir: süresi uzundur, tek işi yeni bir erişim jetonu almaktır.

Rotasyon (döndürme): Yenileme jetonu her kullanıldığında **değişir**. Eskisi geçersiz olur.

```
// Kullanıcı "oturumumu yenile" dedi. Tüm adımlar tek işlemde.
await prisma.$transaction(async (tx) => {

  // 1) Gelen yenileme jetonunu veritabanında ara.
  //   Jetonun kendisi değil, karıştırılmış hâli (hash) saklanıyor.
  const stored = await tx.refreshToken.findUnique({ where: { hash } });

  // 2) Böyle bir jeton hiç yoksa istek reddedilir
  if (!stored) throw new UnauthorizedException();

  // 3) Jeton var AMA daha önce kullanılıp iptal edilmiş.
  //   Bir kez kullanılmış jetonun tekrar gelmesi = kopyalanmış
  //   demektir.
  if (stored.revokedAt) {
    // ★ Güvenlik önlemi: o kullanıcının AÇIK TÜM oturumlarını kapat.
    //   Saldırgan da meşru kullanıcı da yeniden giriş yapmak zorunda
    //   kalır.
    //   Jetonun çalındığını anlamanın tek güvenilir yolu bu.
    await tx.refreshToken.updateMany({
      where: { userId: stored.userId, revokedAt: null },
      data: { revokedAt: clock.now() },
    });
    throw new UnauthorizedException('Token reuse detected');
  }

  // 4) Jeton geçerli: kullanıldığı için hemen iptal et (bir daha
  //   kullanılamaz)
  await tx.refreshToken.update({ where: { hash }, data: { revokedAt:
    clock.now() } });

  // 5) Yeni bir jeton çifti üret ve kullanıcıya ver
  return issueNewPair(stored.userId);
});
```

★ **Yeniden kullanım tespiti** bu kodun kalbi: iptal edilmiş bir jeton tekrar gelirse, jetonun kopyalandığı anlaşılır ve o kullanıcının **tüm oturumları** kapatılır. Ödev bunu doğrudan istemiyor ama gerçek sistemlerde standarttır.

İşlemin tamamı **tek transaction** içinde — ödev §20 bunu ayrıca sayıyor: *"Refresh token yenileme ve eski token'in geçersiz hâle getirilmesi."*

JETON NEREDE SAKLANIYOR

İstemci	Nerede	Neden
Web	httpOnly çerez	JavaScript okuyamaz → XSS saldırısında çalınamaz
Mobil / Swagger	<code>Authorization: Bearer</code> <code>...</code> başlığı	Çerez kavramı yok

Aynı doğrulama katmanı ikisini de kabul ediyor; API istemci tanımıyor (E.10'daki “tek API, çok istemci” ilkesi).

ARGON2 GERÇEKTE HANGİ SORUNU ÇÖZÜYOR

Gerçek hayat: Kâğıt öğütücü. Belgeyi öğütürsünüz — geri birleştirilemez. Ama aynı belgeyi tekrar öğütürseniz **aynı şeritler** çıkar. Karşılaştırma bu şekilde yapılır; belgenin kendisi hiç saklanmaz.

Yazılımda: Şifreler veritabanına **asla düz metin yazılmaz**. Veritabanı sızarsa tüm kullanıcıların şifresi ele geçer — üstelik insanlar aynı şifreyi başka yerlerde de kullandığı için zarar o sistemle sınırlı kalmaz.

```
// KAYIT SIRASINDA – kullanıcının şifresini geri döndürülemez hâle getir.
// Veritabanına yazılan şey şifre değil, bu karıştırılmış çıktı.
const hash = await argon2.hash(password, { type: argon2.argon2id });

// GİRİŞTE – şifre asla geri çözülüyor.
// Kullanıcının yazdığı şifre aynı işlemde geçiriliyor ve
// veritabanındaki çıktıyla karşılaştırılıyor. Eşleşirse giriş başarılı.
const ok = await argon2.verify(user.passwordHash, password);
```

★ **argon2 kasıtlı olarak yavaştır ve çok bellek kullanır.** Bu bir kusur değil, tasarım tercihidir: saldırgan çalınmış bir veritabanında saniyede milyonlarca şifre deneyememeli. Hızlı algoritmalar (MD5, SHA-1) tam da bu yüzden şifre için **yanlıştır** — hızlı olmaları saldırganın işine yarar.

ÖDEVİN DİĞER ŞARTLARI

Şart	Karşılığı
Pasif kullanıcı giriş yapamamalı	Guard, jetonu doğruladıktan sonra kullanıcının <code>isActive</code> alanını kontrol eder
Token süreleri yapılandırmadan yönetilmeli	<code>.env</code> içinde; kodda sabit değer yok
Yetkisiz ve yasaklı erişim doğru kod dönmeli	Kimliksiz 401 , yetkisiz 403 (E.7)
Secret'lar source control'a girmemeli	<code>.env</code> commit edilmez, <code>.env.example</code> edilir

C.14 Turborepo + pnpm workspaces

Nedir: Birden fazla uygulamayı tek depoda düzenli yönetmeyi sağlayan araçlar. **Ne işe yarar:** Web, API ve worker aynı depoda yaşar; ortak kodlar tek yerde tanımlanır; tek komutla hepsi başlar. **Bu projede nerede:** Projenin genel iskeleti. **Ödevdeki karşılığı:** Doğrudan istenmemiş; §9 (DRY) maddesinin mekanizması.

ÇÖZDÜĞÜ GERÇEK SORUN

Bu projede üç ayrı program var (web, API, worker) ve hepsi aynı veri şekillerini kullanıyor. Bunları ayrı depolarda tutsaydık, ortak tipleri paylaşmanın tek yolu her değişiklikte paket yayınlamak olurdu: sürüm yükselt → yayınla → diğer depolarda güncelle. Bu döngü unutulduğunda depolar sessizce ayrışır.

Tek depoda ise ortak şemalar `packages/contracts` içinde bir kez tanımlanır ve üç taraf da oradan okur. API'de bir alanın adı değiştiğinde **arayüz derlenmez** — hata anında ortaya çıkar.

TURBOREPO NE YAPIYOR

Komutları doğru sırayla ve paralel çalıştırır, sonuçları önbelleğe alır. Tek komutla tüm sistem geliştirme modunda başlar:

```
docker compose up -d      # PostgreSQL ve Redis
pnpm dev                  # web + api + worker aynı anda
```

Çıktılar tek ekranda etiketli görünür, biri çökerse hangisi olduğu anlaşılır.

⚠ **Monorepo ile monolit farklı şeylerdir.** Monorepo *kodun nerede durduğu*, monolit *programın nasıl çalıştığı* hakkındadır. Bu projede monorepo (tek depo) kullanılıyor ama çalışırken birden fazla süreç var.

C.15 nestjs-pino — Yapılandırılmış Loglama (Sistemin Gözü)

Modern backend altyapısı: takip edilebilirlik, bağlam ve güvenlik

Bu altyapı; kurumsal yazılımlarda işlerin aksamadan yürümesi, hataların saniyeler içinde bulunması ve veritabanı kayıtlarının güvenliği için tasarlanmıştır. İki temel kütüphane üzerine kuruludur: **nestjs-pino** (sistemin gözü/kulağı) ve **nestjs-cls** (sistemin hafızası — C.16).

Nedir: Node'un en hızlı log kütüphanesi olan **pino**'nun NestJS entegrasyonu. **Ne işe yarar:** Her log satırını **JSON olarak** üretir (düz metin olarak değil) ve isteğe ait bağlamı (correlation ID, kullanıcı, süre, durum kodu) otomatik ekler. **Bu projede nerede:** API ve worker'ın tüm log çıktısı. **Ödevdeki karşılığı:** §25 — structured logging.

NEDEN DÜZ METİN DEĞİL DE JSON

Düz metin: "Ali saat 14:30'da 1042 nolu siparişi sildi ve hata oluştu." Bu satırı yalnızca insan gözü okuyabilir. Milyonlarca log arasında arama motorları bu metni anlamlandıramaz.

JSON (yapılandırılmış): Log alanlara ayrılır:

```
{
  "level": "info",           // önem derecesi: info | warn | error
  "time": 1755500000,        // olayın zamanı
  "correlationId": "9f3c...", // bu isteğin takip numarası – tüm izler
                             bununla bulunur
  "userId": 42,              // işlemi yapan kullanıcı
  "method": "POST",          // HTTP yöntemi
  "path": "/api/v1/work-orders/1042/close", // hangi uç çağrıldı
  "statusCode": 200,         // sonuç: 200 başarılı, 4xx/5xx hata
  "durationMs": 63           // kaç milisaniye sürdü – yavaş uçları
                             bulmak için
}
```

Bu sayede DevOps ekipleri log toplayıcı araçlarda (Loki, ELK, CloudWatch) nokta atışı sorgu yapabilir:

“Son 1 saatte, 42 nolu kullanıcının attığı ve hata (500) alan tüm istekleri getir.”

CORRELATION ID — HATA AYIKLAMANIN EN DEĞERLİ ALANI

Sisteme gelen **her HTTP isteğine** veya uyanan **her worker işine** benzersiz bir takip numarası (`correlationId`) verilir.

Faydası: Bir kullanıcı “hata aldım” dediğinde ekranda bu kod yazar. Yazılımcı bu kodu arattığı an, o isteğin veritabanında, servislerde ve arka planda bıraktığı **tüm izleri** kronolojik olarak listeler — tek bir isteğe ait yüzlerce satır log. Paralel akan yüzlerce isteğin logları birbirine karışmaz.

🚫 LOG GÜVENLİĞİ (MASKELEME / REDACTION)

Loglara **asla** şifre, kredi kartı, TCKN veya token gibi kişisel veriler yazılmaz. `nestjs-pino` içinde merkezî bir süzgeç (maskeleme listesi) vardır. İstek gövdesinde bu veriler geçse bile, loga yazılmadan önce otomatik olarak sansürlenir.

KRİTİK AYRIM: UYGULAMA LOGU VS. DENETİM KAYDI (AUDIT TRAIL)

Yazılımcıların en çok karıştırdığı konulardan biri budur. İkisi tamamen farklı amaçlara hizmet eder:

Özellik	Uygulama logu (<code>nestjs-pino</code>)	Denetim kaydı (audit / veritabanı)
Örnek	<code>POST /orders/1042</code> <code>200 63ms</code>	<i>"Ahmet, 1042 nolu sipariş durumunu 'Beklemede'den 'İptal Edildi'ye çekti."</i>
Depolama yeri	Konsol çıktısı (stdout) → log toplayıcı	Doğrudan veritabanı tablosu
Saklama süresi	Günler veya haftalar (sonra silinir)	Yıllarca (silinmesi yasaktır)
Kullanıcı	Yazılımcı / DevOps (hata çözmek için)	Müfettiş, hukuk, iş birimi (denetim için)
Şema yapısı	Serbest / esnek	Sabit kolonlar (<code>createdBy</code> , <code>updatedBy</code>)

C.16 nestjs-cls — İstek Bağlamı (Sistemin Hafızası)

Nedir: Node.js'in yerleşik `AsyncLocalStorage` API'sini NestJS'e bağlayan ince bir katman. **Ne işe yarar:** Bir isteğe ait bilgileri (aktif kullanıcı, correlation ID) çağrı zincirinin her katmanına, **parametre olarak taşımadan** ulaştırır. **Bu projede nerede:** Audit alanlarının doldurulması, loglama, worker bağlamı. **Ödevdeki karşılığı:** §21 — *"sistem saati ve mevcut kullanıcı bilgisi doğrudan statik yapılardan alınmamalıdır."*

PROBLEM

Veritabanına bir veri yazarken `createdBy` (yazan kim) alanını doldurmak için, o anki kullanıcının kimlik bilgisini en üstteki controller'dan en alttaki Prisma veritabanı katmanına kadar indirmemiz gerekir.

Bunu yapmak için iki yaygın ama **yanlış** yöntem vardır:

1. Parametre olarak taşımak (hamallık). Geçtiğiniz her fonksiyona `(..., userId)` eklemek. Kod kirlenir. Yarın bir gün takibe `correlationId` veya `ipAddress` eklemek isterseniz **yüzlerce fonksiyon imzasını** değiştirmek zorunda kalırsınız.

2. Global değişkende tutmak (felaket). Node.js tek bir iş parçacığında (single thread) asenkron çalışır. Global bir değişken kullanırsanız asenkron beklemler (`await`) sırasında işlemler çakışır:

```
Ahmet sisteme girer          → Global isim "Ahmet" olur
Ahmet'in veritabanı işlemi (await) beklemeye alınır
0 sırada Mehmet sisteme girer → Global isim "Mehmet" olarak EZİLİR
Ahmet'in işlemi kaldığı yerden devam eder ve veritabanına kaydedilir
                                → Sistem veriyi MEHMET kaydetti sanır
```

⚠ Bu hata testlerde çıkmaz; sadece sistem canlıya geçip **yük altında** kaldığında ortaya çıkar.

ÇÖZÜM: ASYNCLOCALSTORAGE (NESTJS-CLS)

Node.js'in yerleşik `AsyncLocalStorage` yapısı, her asenkron çağrı zincirine özel, **birbirini asla görmeyen izole hafıza kutuları** açar. .NET dünyasındaki `AsyncLocal` veya Java'daki `ThreadLocal` yapısının aynısıdır.

Nasıl çalışır: İstek geldiği an `nestjs-cls` bir kutu açar, içine `userId` ve `correlationId` koyar. Katmanlar boyunca hiçbir fonksiyona parametre geçilmez. En alttaki Prisma veya log sistemi, veriyi bu kutudan doğrudan okur. Ahmet ve Mehmet'in kutuları asenkron dünyada asla birbirine karışmaz.

BU PROJEDE HER ŞEY NASIL BİRLEŞİYOR

Kurulan bu altyapı sayesinde projedeki dört ana unsur otomatik olarak tıkr tıkr çalışır:

1. Audit (otomatik takip)

Prisma için bir eklenti (extension) yazılmıştır. Yazılımcı servis kodunda tek bir satır bile `createdBy = user.id` yazmaz. Prisma veritabanına kayıt giderken eklenti araya girer, `nestjs-cls` hafıza kutusundan o anki aktif kullanıcıyı okur ve alanları otomatik doldurur.

2. Log

`nestjs-pino` her log satırına hafıza kutusundaki `correlationId` ve `userId` bilgilerini otomatik ekler.

3. Sistem saati (Clock servisi)

Kodun içinde doğrudan `new Date()` çağırılması **yasaktır**. Zaman bilgisi merkezî bir `Clock` servisi üzerinden okunur.

Test yazarken bu servise *"sen şu an 30 gün sonrasındasın"* denerek, zamana bağlı tüm kurallar (SLA) gerçek zamanı beklemeden saniyeler içinde test edilir.

4. Arka plan işlerinde (worker/job) bağlam

 **Kritik uyarı (§15):** Bir worker (örneğin gece çalışan robot) tetiklendiğinde ortada bir HTTP isteği, tarayıcı veya token **yoktur**. Dolayısıyla HTTP bağlamı boş (`undefined`) gelecektir. Eğer yazılımcı servis kodunu yazarken *"nasılsa HTTP isteğinden kullanıcı gelir"* diye güvenerek kod yazarsa, worker çalıştığı an sistem çöker.

Çözüm: Robot işe başladığı an **kendi bağlam kutusunu** (`nestjs-cls`) kendisi açar. İçine `userId: "SYSTEM_ROBOT"` ve kendi ürettiği `correlationId` bilgisini koyar. Böylece:

- Ortak yazılan servis kodları çökmeden çalışır
- Prisma audit alanları otomatik dolar
- Loglarda bu işi bir insanın değil **robotun** yaptığı net şekilde görünür

Ödevin §15'teki *"job içerisinde HTTP request context'e güvenilmemelidir"* maddesinin karşılığı budur.

C.17 @nestjs/terminus — sağlık kontrolü

Nedir: Uygulamanın sağlıklı olup olmadığını dışarıya bildiren uçları oluşturan kütüphane. **Bu projede nerede:** `/health/live` ve `/health/ready`. **Ödevdeki karşılığı:** §25 — bu iki uç açıkça isteniyor.

ÇÖZDÜĞÜ SORUN

Gerçek hayat: Bir mağazanın kapısında "AÇIK" tabelası var ama içeride kasa sistemi çökmüş. Kapı açık, ışıklar yanıyor — ama **alışveriş yapılamıyor**. Dışarıdan bakan biri farkı anlamaz; içeri girip denemesi gerekir.

Yazılımda: Uygulama süreci ayakta olabilir ama veritabanına bağlanamıyor olabilir. Dışarıdan bakınca “çalışıyor” görünür; gerçekte her istek hata alır.

İKİ UCUN FARKI

```
@Controller('health')
export class HealthController {

  // GET /health/live – "Uygulama süreci yaşıyor mu?"
  @Get('live')
  live() {
    // Hiçbir şeyi kontrol etmiyor, sadece cevap veriyor.
    // Cevap gelmiyorsa süreç donmuş demektir → izleme sistemi yeniden
    başlatır.
    return { status: 'ok' };
  }

  // GET /health/ready – "İstek almaya HAZIR mı?"
  @Get('ready')
  ready() {
    return this.health.check([
      // Veritabanına kısa bir sorgu atıp cevap verip vermediğine
      bakıyor.
      // 1.5 saniyede cevap gelmezse hazır değil sayılıyor.
      () => this.db.pingCheck('database', { timeout: 1500 }),
    ]);
    // Hazır değilse bu kopyaya trafik yönlendirilmiyor –
    // ama süreç yeniden başlatılmıyor da. Farkı bu.
  }
}
```

Uç	Soru	Cevap gelmezse ne olur
/health/live	Süreç yaşıyor mu	İzleme sistemi konteyneri yeniden başlatır
/health/ready	İstek alabilir mi	O kopyaya trafik yönlendirilmez

★ **Ayrımın sebebi:** İkisi aynı olsaydı, veritabanı birkaç saniyeliğine yavaşladığında izleme sistemi uygulamayı **gereksiz yere yeniden başlatırdı** — ve yeniden başlatma sırasında zaten hiç cevap veremezdi. Yani çözüm sorunu büyüttü.

`live` yalnızca sürecin donmadığını söyler, `ready` bağımlılıkları yoklar.

BU PROJEDE KİM KULLANIYOR

Bu uçlar **sizin için değil**, kurumun izleme sistemi için. DevOps ekibi konteynerleri bu uçlara bakarak yönetir:

- Uygulama açılırken veritabanı henüz gelmemişse `ready` “hayır” der, o aralıkta gelen istekler hata almaz
- `docker-compose.yml` içinde de aynı uç kullanılır: `api` servisi ayağa kalkmadan `worker` başlatılmaz

Yani bu iki uç, teslim paketinin **DevOps’la sözleşmesinin** bir parçası.

C.18 Swagger / OpenAPI

Nedir: API’nin hangi uçları olduğunu, her ucun hangi veriyi beklediğini ve ne döndürdüğünü anlatan otomatik dokümantasyon. **Bu projede nerede:** `/docs` adresinde tarayıcıdan açılabilen arayüz. **Ödevdeki karşılığı:** §3 — OpenAPI + Scalar veya Swagger.

ÇÖZDÜĞÜ GERÇEK SORUN

Gerçek hayat: Bir cihazın kullanma kılavuzu, cihazın üçüncü sürümüne göre yazılmış ama elinizde beşinci sürüm var. Kılavuzdaki düğme cihazda yok. Kılavuz **zararlı** hâle gelmiş — hiç olmamasından kötü, çünkü ona güveniyorsunuz.

Yazılımda: API dokümantasyonu elle yazıldığında **kaçınılmaz olarak eskir**. Kod değişir, doküman güncellenmez; bir süre sonra kimse dokümana güvenmez ve herkes koda bakmaya başlar.

BU PROJEDE DOKÜMAN ELLE YAZILMIYOR

Doküman **Zod şemalarından otomatik üretiliyor** — yani doğrulama kuralı ne ise dokümanda yazan da odur:


```
// TEK KAYNAK: iş emri oluşturma isteğinin kuralları
export const CreateWorkOrder = z.object({
  title: z.string().min(5).max(200), // 5-200 karakter
  priority: z.enum(['LOW', 'NORMAL', 'HIGH', 'CRITICAL']), // bu dört
  değerden biri
  assetId: z.number().int().positive(), // pozitif tam
  sayı
});

// Aynı şema iki işi birden yapıyor:
// 1) gelen isteği doğruluyor
// 2) Swagger dokümanını üretiyor
// Yani kural değişince doküman kendiliğinden güncelleniyor.
@Post()
@ApiOperation({ summary: 'Yeni iş emri oluşturur' }) // dokümandaki
açıklama
create(@Body() dto: CreateWorkOrderDto) { ... }
```

★ Kural değiştiğinde doküman **kendiliğinden** değişir. Eskime ihtimali yapısal olarak ortadan kalkıyor — E.3'teki DRY ilkesinin bir uygulaması.

SADECE OKUNMAZ, DENENİR

Swagger arayüzü uçları **tarayıcıdan çalıştırmayı** sağlar. Bu projede iki somut faydası var:

1. **Frontend yazılmadan API gösterilebilir.** Değerlendirmede *"iş emri oluşturma çalışıyor mu"* sorusuna, ekran hazır olmasa bile cevap verilebilir.
2. **Mobil geliştirici** (ileride) API'yi koda bakmadan öğrenir.

NEDEN SWAGGER UI, SCALAR DEĞİL

Ödev ikisine de izin veriyor. Swagger UI seçildi çünkü `@nestjs/swagger` ile **ekstra bağımlılık olmadan** geliyor ve piyasada herkesin tanıdığı arayüz. Scalar daha modern görünüyor ama çok daha az kullanılıyor; uzun ömürlü bir projede omurgayı az bilinen bir arayüze bağlamak devraları zorlar.

C.19 TanStack Query

Nedir: Tarayıcı tarafında sunucudan gelen veriyi yöneten kütüphane. **Ne işe yarar:** Veriyi getirir, saklar, tazeler; yükleniyor ve hata durumlarını yönetir. **Bu projede nerede:** Tüm liste ve detay ekranları. **Ödevdeki karşılığı:** §22 — *"API çağrıları ortak bir katmanda yönetilmelidir, tekrarlanan request kodlarından kaçınılmalıdır."*

HANGİ PROJEDE YAŞIYOR

Bu ayrımı netleştirmek önemli: **API ve worker** backend (arka plan) projesindedir. **TanStack Query** ise frontend (ön yüz) projesinde yazılan ortak bir katmandır.

TanStack Query'nin doğası: tarayıcının hafızasını (cache) yönetir.

ÇÖZDÜĞÜ GERÇEK SORUN

Web sitesini açan insanın ekranı donmasın, butonlar hızlı çalışsın ve aynı veri için sunucuya gereksiz yere üst üste iki kez istek gitmesin diye **önbelleğe alma** yapmaktır.

ÖDEVDEKİ "ORTAK KATMAN" MADDESİ NE DEMEK İSTİYOR (§22)

Ödevde bahsedilen *"API çağrıları ortak bir katmanda yönetilmelidir"* uyarısı tamamen **frontend** projesi içindir.

Yanlış yapılan (tekrarlayan kod): Frontend yazılımcısı sipariş listesi sayfasında, ürün listesi sayfasında ve profil sayfasında tek tek elle `fetch('/api/v1/...')` yazıp, `isLoading` (yükleniyor) durumlarını her ekranda sıfırdan elle kodlarsa hata yapar ve kod kirlenir.

Doğru yapılan (ortak katman): Frontend projesinin içinde tüm API istekleri TanStack Query ile tek bir klasörde (örneğin `/hooks` klasöründe) toplanır. Ekranlar sadece bu ortak katmanı çağırır.

ORTAK KATMAN PRATİKTE NEYE BENZİYOR

```
// Dosya: apps/web/hooks/use-work-orders.ts
// Tüm ekranlar iş emri listesini BURADAN alıyor. Tek yer.
export function useWorkOrders(filters: WorkOrderFilters) {
  return useQuery({
    // Bu verinin "adresi". Filtre değişince adres de değişir ve
    // kütüphane otomatik olarak yeni veriyi çeker.
    queryKey: ['work-orders', filters],

    // Veriyi fiilen getiren fonksiyon: API'ye istek atıyor
    queryFn: () => api.get('/work-orders', { params: filters }),

    // 30 saniye içinde aynı veri tekrar istenirse ağa çıkmıyor,
    // hafızadaki kopyayı veriyor. Ekranlar arası geçiş anında oluyor.
    staleTime: 30_000,
  });
}
```

```
// Ekran tarafı – üç satır, isLoading ve error hazır geliyor
// Ekran tarafı üç satır. Yükleniyor ve hata durumları hazır geliyor –
// her ekranda elle yazmak gerekmiyor.
const { data, isLoading, error } = useWorkOrders({ status: 'OPEN' });

if (isLoading) return <TabloIskeleti />; // veri gelene kadar
iskelet göster
if (error) return <HataKutusu error={error} />; // hata varsa mesaj
göster
return <WorkOrderTable rows={data.items} />; // veri geldi, tabloyu
çiz
```

★ `queryKey` bu yapının kalbi: aynı anahtarla iki ekran veri isterse **ağa tek istek** çıkar, ikisi de aynı sonucu alır.

BU PROJEDE ÖZELLİKLE İŞE YARAYAN YANI

Bir iş emrinin durumu değiştiğinde, liste ekranındaki verinin artık **eskidiğini bilir** ve otomatik tazeler. Kullanıcı sayfayı elle yenilemek zorunda kalmaz.

```
// Veri DEĞİŞTİREN işlemler useMutation ile yapılıyor (okuma değil,
yazma)
const kapat = useMutation({
  // Kapatma isteğini API'ye gönderen fonksiyon
  mutationFn: (id: number) => api.post(`/work-orders/${id}/close`),

  // İşlem başarılı olunca çalışıyor
  onSuccess: () => {
    // "İş emri listesi artık eski" diyoruz.
    // 0 listeyi gösteren tüm ekranlar kendiliğinden yeniden çekiyor –
    // kullanıcının sayfayı elle yenilemesi gerekmiyor.
    queryClient.invalidateQueries({ queryKey: ['work-orders'] });
  },
});
```

C.20 Tailwind CSS + shadcn/ui + React Hook Form

Bu projede nerede: 12 ekranın tamamı. **Ödevdeki karşılığı:** §22 — “*hazır UI kütüphanesi kullanılabilir.*” Ödev arayüzün ileri seviye görsel tasarıma sahip olmasını zorunlu tutmuyor, ama **kullanılabilir** olmasını istiyor. Sıfırdan bileşen yazmak zaman kaybı olurdu.

1. TAILWIND CSS — HIZLI STİL VERME

Ne işe yarar: Klasik yöntemde bir butona renk vermek veya kenarlarını yuvarlatmak için ayrı bir CSS dosyası açıp sayfalarca kod yazmanız gerekir. Tailwind CSS ise hazır takma isimler (utility class) sunar.

Nasıl çalışır: Doğrudan butonun yanına `bg-blue-500 rounded p-2` yazarsınız; buton anında mavi, köşeleri yuvarlak ve içi genişlemiş hâle gelir. Ekstra CSS dosyalarıyla uğraşmadan, tasarımı kodun içinden çok hızlı bitirmenizi sağlar.

```
// KLASİK YÖNTEM: burada sadece bir isim var.
// Bu ismin ne yaptığını görmek için ayrı bir CSS dosyası açman
gerekiyor.
<button className="birincil-buton">Kaydet</button>

// TAILWIND: stilin tamamı burada, okunabiliyor.
<button className="bg-blue-600      // arka plan: mavi
                    hover:bg-blue-700 // fare üstüne gelince koyu mavi
                    text-white        // yazı rengi beyaz
                    rounded            // köşeler yuvarlak
                    px-4 py-2">      // içeriden boşluk: yanlarda 4, üst-
altta 2
    Kaydet
</button>
```

2. SHADCN/UI — HAZIR VE ESNEK TASARIM PARÇALARI

Ne işe yarar: Bir web sitesinde ihtiyaç duyulan buton, tablo, modal (açılır pencere) veya menü gibi temel parçaları sıfırdan kodlamak büyük zaman kaybıdır. shadcn/ui bu parçaları hazır olarak verir.

Özel yanı nedir: Klasik kütüphanelerde (örneğin Material UI veya Ant Design) hazır bir butonu projenize eklediğinizde o buton kütüphaneye gömülüdür; rengini veya çalışma mantığını değiştirmek çok zordur.

shadcn/ui ise seçtiğiniz bileşenin **tüm kaynak kodunu doğrudan sizin kendi klasörünüzün içine kopyalar**. Buton artık tamamen sizin kodunuz olur. İstedığınız gibi kurcalayabilir, projenize göre özelleştirebilirsiniz.

★ Uzun ömürlü projelerde bu belirleyici bir avantajdır: bir davranışı değiştirmek için kütüphane güncellemesi beklemezsiniz.

3. REACT HOOK FORM + ZOD UYUMU — AKILLI FORM YÖNETİMİ

Bu üçlünün en kritik ve en teknik kısmı burasıdır. Büyük projelerde form yönetimi iki büyük probleme yol açar:

Problem 1 — ekranın donması (performans). Kullanıcı bir kutucuğa her harf yazdığında tüm sayfa hafızada baştan çizilirse (re-render), form donmaya başlar. React Hook Form, kullanıcı yazarken sayfayı gereksiz yere baştan çizmez; performansı çok yüksektir.

Problem 2 — “ön yüzde geçti, arka yüzde reddedildi” tutarsızlığı. En çok yaşanan hatadır. Örneğin frontend yazılımcısı fiyata **en az 5 TL** sınırı koymuştur, ama backend yazılımcısı koda **en az 10 TL** yazmıştır. Kullanıcı formu doldurur, ön yüz hata vermez ama “Gönder” dediğinde backend’den hata döner.

Çözüm: React Hook Form, projedeki Zod şemalarını doğrudan kullanabilir (aradaki bağlantıyı `@hookform/resolvers` sağlar). Backend’de doğrulamayı yapan o Zod kuralını — örneğin `price: z.number().min(10)` — alır, frontend formunun içine bağlarsınız.

```
// Sunucunun doğrulama için kullandığı şemanın AYNISINI alıyoruz.
// Aynı bir kural yazmıyoruz – tek kaynak.
import { CreateWorkOrder } from '@contracts/work-order';

const form = useForm({
  // Köprü burası: Zod şemasını React Hook Form'a tanıtan çevirici
  resolver: zodResolver(CreateWorkOrder),
});

// Form alanını kütüphaneye bağlıyoruz – yazılan her harfi o takip ediyor


```

★ Hata mesajı bile şemadan geliyor — backend hangi metni döndürüyorsa kullanıcı “Gönder”e basmadan **aynı metni** görüyor.

Böylece doğrulama kuralı **tek bir merkezden** yönetilir. Kural değiştikçe hem backend hem frontend aynı kuralı çalıştırır. Kullanıcı daha “Gönder” butonuna basmadan, backend ile birebir aynı kurallara göre hata mesajlarını ekranda görür.

ÖZETLE

Araç	Ne katıyor
Tailwind CSS	Tasarımı hızlandırır
shadcn/ui	Form, buton ve tabloları hazır verir ama kontrolü size bırakır
React Hook Form + Zod	Formların donmadan çalışmasını sağlar ve backend kurallarıyla frontend kurallarını eşitleyerek hata yapılmasını engeller

C.21 @dbml/cli — veri modeli dokümanı (Database Markup Language)

Nedir: Normalde bir veritabanında hangi tabloların olduğunu, bu tabloların içinde hangi kolonların yer aldığını ve tabloların birbirine nasıl bağlandığını (ilişkilerini) görmek için karmaşık SQL kodlarına bakmak zordur.

DBML, veritabanı şemasını insanların çok rahat okuyabileceği **sade bir metin formatına** dönüştürür. En güzel yanı ise şu: bu metni `dbdiagram.io` gibi sitelere yapıştırdığınızda, size otomatik olarak kutucuklar ve çizgilerden oluşan **görsel bir veritabanı diyagramı (ERD)** çizer.

Ne işe yarar: Veri modelinin görsel ve paylaşılabılır dokümanını üretir. **Bu projede nerede:** `docs/database.dbml`. **Ödevdeki karşılığı:** §11 — bu dosya **zorunlu**.

Üretilen dosya şuna benzer — SQL bilmeyen biri de okuyabilir:

```

Table WorkOrder {
  id          int          [pk, increment]  // pk = birincil anahtar,
otomatik artar
  number      varchar      [unique, note: 'IE-2026-000148'] // benzersiz,
insan okur
  status      varchar      [not null]        // boş bırakılamaz
  assetId     int          [not null]        // hangi varlığa ait
  slaDueAt    timestamp    // SLA bitiş zamanı (boş
olabilir)
}

// İlişki tanımı: bir varlığın birden çok iş emri olabilir.
// dbdiagram.io gibi araçlar bu satırı okuyup iki kutu arasına ok
çiziyor.
Ref: WorkOrder.assetId > Asset.id

```

PROBLEM: ELLE YAZILAN DOKÜMAN ILK ŞEMA DEĞİŞİKLİĞİNDE ESKİR

Diyelim ki projeye başlarken elle bir DBML dosyası yazdınız ve ödevi teslim ettiniz. İki gün sonra projedeki bir kural değişti ve veritabanındaki `User` tablosuna `phoneNumber` adında yeni bir kolon eklemek zorunda kaldınız.

Gittiniz, veritabanı kodlarını güncellediniz (migration attınız). Ama o yoğunlukta unutup DBML doküman dosyasını elle güncellemediniz.

Sonuç: Veritabanı ile doküman birbirinden farklı oldu — uyumsuzluk doğdu. Ödevi inceleyen değerlendirmeci veya iş yerindeki bir başka geliştirici dokümana bakarak kod yazmaya çalışırsa sistem patlar.

Ödevdeki §11 maddesi tam olarak bu uyumsuzluğu yasaklıyor.

ÇÖZÜM: @DBML/CLI İLE OTOMATİK ÜRETİM

Bu projede doküman **asla elle yazılmıyor**. Süreç şöyle işliyor:

1. Yazılımcı Prisma üzerinden veritabanında bir değişiklik yapıyor (örneğin yeni bir tablo ekliyor)
2. Değişiklik bittiği an `@dbml/cli` aracı devreye giriyor

3. Bu araç, **fiilen değişen canlı veritabanı şemasına** bakıyor ve saniyeler içinde `docs/database.dbml` dosyasını baştan, hatasız biçimde yeniden üretiyor

Kodda ne değiştiyse, dokümanda da saniyesinde o değişiyor.

CI SÜRECİNDE KONTROL — “BUILD’IN KIRMIZI YANMASI” NE DEMEK

Peki ya yazılımcı bu otomatik aracı kendi bilgisayarında çalıştırmayı unutup kodu sunucuya yüklerse? Projenin **emniyet kilidi** burada devreye giriyor.

CI (Continuous Integration / sürekli entegrasyon), kodu depoya yüklediğiniz an sunucuda otomatik çalışan bir test robotudur.

1. Robot kodu alır almaz `@dbml/cli` aracını çalıştırır ve yeni bir DBML üretir
2. Sizin yüklediğiniz DBML dosyası ile kendi ürettiği güncel dosyayı karşılaştırır
3. İki dosya arasında **tek bir karakter bile fark varsa** robot şunu der: *”Yazılımcı veritabanını değiştirmiş ama dokümanı güncellemeyi unutmuş!”*
4. Build sürecini iptal eder (kırmızı yakar) ve o kodun canlıya geçmesini engeller

```
# CI adımı – doküman güncel değilse build burada durur

# 1) Veritabanının GERÇEK yapısını dışa aktar (veriyi değil, sadece şemayı)
- run: pg_dump --schema-only "$DATABASE_URL" > /tmp/schema.sql

# 2) O yapıyı okunabilir DBML biçimine çevir
- run: npx sql2dbml /tmp/schema.sql --postgres -o /tmp/database.dbml

# 3) Yeni üretilen dosyayla depodakini karşılaştır.
# Tek karakter fark varsa diff hata koduyla çıkar ve CI kırmızı yanar.
- run: diff /tmp/database.dbml docs/database.dbml
```

`diff` komutu iki dosya arasında fark bulursa sıfırdan farklı bir kodla çıkar ve CI durur.

Yazılımcı hatasını düzelterek dokümanı güncelleyene kadar sistem geçişi izin vermez.

ÖZETLE

"Uyumludur" ifadesinin bir iddia olmaktan çıkıp **mekanizmaya** dönüşmesi şu demektir: yazılımcı sözlü olarak "*dokümanım kodla uyumlu, kontrol ettim*" demez. Sistem, doküman güncel değilse zaten kodun geçmesine izin vermeyerek bu uyumluluğu **mekanik olarak zorunlu** kılar.

C.22 Renovate

Nedir: Kullanılan paketlerin yeni sürümlerini takip edip otomatik güncelleme önerisi açan bot. **Bu projede nerede:** Depo genelinde, haftalık. **Ödevdeki karşılığı:** İstenmemiş; sürdürülebilirlik için eklendi.

ÇÖZDÜĞÜ GERÇEK SORUN

Gerçek hayat: Bir binada 40 yangın tüpü var ve her birinin dolum tarihi farklı. "Ara sıra kontrol ederiz" denince hiç kontrol edilmez; yangın çıktığında yarısının boş olduğu anlaşılır.

Yazılımda: Bir projede 40–60 paket kullanılır ve hepsi zaman zaman güncellenir. Bazı güncellemeler **güvenlik açığı kapatır**. Bunları elle takip etmek gerçekçi değildir — kimse yapmaz, iki yıl sonra proje eski ve açık bir sürümde kalır.

NASIL ÇALIŞIYOR

```
// Dosya: renovate.json – güncelleme botunun ayarları
{
  // Hazır önerilen ayar setini temel al
  "extends": ["config:recommended"],

  // Ne zaman tarasın: pazartesi sabahı, mesai başlamadan
  "schedule": ["before 6am on monday"],

  "packageRules": [
    // Küçük güncellemeleri TEK bir öneride topla (haftada bir istek)
    { "matchUpdateTypes": ["minor", "patch"], "groupName": "küçük güncellemeler" },

    // Büyük sürüm atlamaları genelde kırıcı değişiklik taşır –
    // otomatik açılmasın, önce onay beklesin
    { "matchUpdateTypes": ["major"], "dependencyDashboardApproval": true
  }
]
```

Her hafta tarar ve güncelleme için otomatik değişiklik önerisi açar.

★ **Asıl değerli kısım:** O öneri açıldığında **tüm test zinciri onun üzerinde koşar**. Güncelleme bir şeyi bozuyorsa, birleştirilmeden önce görülür. Yani Renovate sadece haber vermiyor, **güvenli olduğunu da kanıtlıyor**.

Küçük güncellemeler tek öneride gruplanıyor (haftada bir tane), büyük sürüm atlamaları ise ayrı ayrı ve **onay bekleyerek** geliyor — çünkü büyük sürüm genellikle kırıcı değişiklik taşır.

NEDEN DEPENDABOT DEĞİL

Dependabot yalnızca GitHub’da çalışır. Bu proje kurumun **GitLab**’ına taşınacak; Renovate ikisinde de çalışıyor. Araç, platform değişince taşınabilir olsun diye seçildi.

NEDEN ÖDEVDE OLMADIĞI HÂLDE EKLENDİ

"Bağımlılıkları güncel tutalım" bir niyet olarak kalırsa yapılmaz. Bu proje yıllarca sürdürülecek ve devredilecek; niyeti **otomatik bir sürece** çevirmek devralınabilirliğin parçası.

C.23 Expo — ileride mobil için

Nedir: React Native'in üstüne kurulmuş araç takımı; React bilgisiyle telefon uygulaması yazmayı sağlar. **Bu projede nerede:** Şu an kapsam dışı; **mimari buna hazır tasarlandı. Ödevdeki karşılığı:** İstenmemiş.

GERÇEK HAYAT KARŞILIĞI

Bir aracın motoru ile o motorun etrafına kurulmuş araç arasındaki fark gibi. Motor tek başına da çalışır ama sürebilmek için şanzıman, direksiyon ve gösterge paneli gerekir. React Native motordur; Expo o motoru sürülebilir hâle getiren takımdır.

İKİSİ ARASINDAKİ İLİŞKİ

React Native, React ile **gerçek native mobil uygulama** yazma framework'üdür — çıktı bir web sayfası değil, App Store veya Play Store'a yüklenebilen gerçek bir uygulamadır.

Expo bunun üstünde çalışan araç takımıdır: Xcode veya Android Studio kurmadan geliştirme, tek komutla derleme, kamera ve bildirim gibi özelliklerin hazır gelmesi, mağazaya göndermeden güncelleme.

⚠ *"Expo mu React Native mi"* bir seçim değil — Expo kullanınca React Native'i zaten kullanıyorsunuz. Nest/Express ilişkisinin aynısı (C.1).

MOBİL YAPILMADIĞI HÂLDE NEDEN ŞİMDİDEN KARARI VERİLDİ

Çünkü mobilin varlığı **backend tasarımını değiştirir**, ve bu kararlar sonradan alınırsa API'yi baştan yazmak gerekir:

Karar	Mobil olmasa	Mobil olacaksa
Sürümleme	Gerekmeyebilir — web güncellenince herkes yeni sürümü alır	Zorunlu — uygulama kullanıcının telefonunda eski sürümde kalır
Jeton taşıma	Sadece çerez yeterdi	Çerez ve <code>Authorization</code> başlığı birlikte desteklenmeli
Sayfalama	Gevşek olabilirdi	Zorunlu — mobilde 5000 kayıt uygulamayı dondurur
Cevap boyutu	Önemsiz	Kritik — mobil veri kotası

Bu yüzden API baştan `/api/v1/...` biçiminde sürümlendi ve **istemci tanımaz** şekilde tasarlandı (E.10). Mobil eklendiğinde backend'e dokunmak gerekmeyecek.

★ Ödevin kendisi mobil istemiyor. Ama kurumun mevcut sistemlerinde mobil uygulamalar var; bu sistemin de eninde sonunda mobilden tüketileceğini varsaymak gerçekçi bir kabul.

NASIL GELİŞTİRİLİR

Kod yazma yeri değişmez — aynı editör, aynı dil. Fark yalnızca çalışanı görmeye:

	Web	Mobil
Çalıştırma	<code>pnpm dev</code>	<code>npx expo start</code>
Görme	Tarayıcı	Telefonda Expo Go uygulaması, QR kod okutularak
Xcode / Android Studio	Gerekmez	Çoğu iş için gerekmez

Mobil eklendiğinde API tarafında yazılacak tek şey, jetonu başlıktan da kabul etmek — ve o zaten baştan yazılı:

```
// Jetonun nereden okunacağı. Sıra önemli: önce çerez, olmazsa başlık.  
jwtFromRequest: ExtractJwt.fromExtractors([  
  
  // WEB: tarayıcı jetonu çerezde taşıyor.  
  // Çerez httpOnly olduğu için JavaScript okuyamıyor → XSS'te  
  çalınamıyor.  
  (req) => req?.cookies?.access_token,  
  
  // MOBİL ve Swagger: çerez kavramı yok, jeton başlıkta geliyor.  
  // "Authorization: Bearer eyJhbGc..." biçiminde.  
  ExtractJwt.fromAuthHeaderAsBearerToken(),  
]),  
// Aynı doğrulama kodu iki taşıma yolunu birden tanıdığı için  
// mobil eklendiğinde sunucu tarafında değişiklik gerekmiyor.
```

BÖLÜM E — Kavramlar, prensipler ve desenler

Önceki bölümde anlatılanlar **kurulan paketlerdi**. Bu bölümdekiler ise kurulmaz; **karar verilir**. Ödevin en çok puan verdiği kısımlar da burasıdır, çünkü paket kurmak kolaydır — doğru kararı verip tutarlı uygulamak zordur.

E.0 Önce temel kelimeler

Bu bölümdeki her şey birkaç temel kavrama dayanıyor. Her birini üç adımda açıyoruz: **gerçek hayattan karşılığı** → **yazılımdaki tanımı** → **bu projede tam olarak nerede**.

SINIF VE NESNE

Gerçek hayat: Kurabiye kalıbı ile kurabiye. Kalıp tektir; ondan yüzlerce kurabiye çıkar. Her kurabiyenin şekli aynıdır ama biri çilekli, biri kakaolu olabilir.

Yazılımda: **Sınıf** kalıptır — hangi bilgileri tuttuğu ve neler yapabildiği yazar. **Nesne** o kalıptan üretilmiş tek bir örnektir.

Bu projede:

```
// SINIF = kalıp. "Bir iş emri neye benzer, ne yapabilir?" sorusunun
cevabı.
// Burada henüz gerçek bir iş emri yok – sadece tarifi var.
class WorkOrder {
    id: number; // Veritabanının verdiği benzersiz
numara
    number: string; // İnsanın okuyacağı numara: "IE-
2026-000148"
    status: Status; // Hangi aşamada: OPEN | ASSIGNED |
IN_PROGRESS ...
    assigneeId: number | null; // Atanan teknik personel. null =
henüz atanmamış
    resolutionNote: string | null; // Çözüm açıklaması. Kapatılana
kadar boş
}

// NESNE = o kalıptan üretilmiş TEK BİR gerçek kayıt.
// Veritabanındaki her satır, kod tarafında böyle bir nesneye dönüşüyor.
const isEmri = { id: 1042, number: 'IE-2026-000148', status: 'OPEN', ...
};
```

Veritabanındaki `WorkOrder` tablosunun her satırı, bu kalıptan üretilmiş bir nesnedir. 5.000 iş emri = 1 sınıf, 5.000 nesne.

METOT

Gerçek hayat: Çamaşır makinesinin düğmeleri. Makine **ne olduğunu** (marka, kapasite) taşır; düğmeler **ne yapabildiğidir** (yıkla, sık, durula).

Yazılımda: Sınıfın yapabildiği iş. Veri “ne olduğu”, metot “ne yapabildiği”.

Bu projede — ekrandaki butondan koda kadar zincir:

Kullanıcı iş emri detay ekranında **”Kapat”** butonuna basıyor. Arkada şu olur:


```
// 1. ADIM – Ekran (Next.js): kullanıcı "Kapat" butonuna bastı.
//   Tarayıcı, 1042 numaralı iş emri için API'ye istek gönderiyor.
//   Kullanıcının yazdığı çözüm açıklaması da isteğin içinde gidiyor.
POST /api/v1/work-orders/1042/close { resolutionNote: "Sigorta
değiştirildi" }

// 2. ADIM – API katmanı (NestJS controller): isteği karşılayan yer.
//   Burada iş kuralı YOK; sadece gelen bilgiyi alıp servise devrediyor.
close(id, dto) {
  return this.service.close(id, dto.resolutionNote); // işi asıl yapan
yere gönder
}

// 3. ADIM – Domain katmanı: kuralın fiilen çalıştığı yer.
//   Bu sınıf veritabanını da interneti de tanımıyor; sadece kuralı
biliyor.
class WorkOrder {
  close(note: string, clock: Clock) {

    // Kural 1: iş emri "Çözüldü" aşamasında değilse kapatılamaz
    if (this.status !== 'RESOLVED') throw new
InvalidTransition(this.status);

    // 🚫 Kural 2: çözüm açıklaması boş bırakılamaz (ödevin §6 şartı)
    if (!note?.trim()) throw new ResolutionNoteRequired();

    // Kurallar geçildi – kaydın durumunu değiştir
    this.status = 'CLOSED';
    this.closedAt = clock.now(); // saati doğrudan değil Clock
servisinden al
  }
}
```

★ Dikkat: kural (`çözüm açıklaması boşsa kapatma`) butonda değil, ekranda değil, controller'da değil — **nesnenin kendi metodunda**. Böylece iş emri mobil uygulamadan da, arka plan işinden de kapatılsa aynı kural çalışır.

ARAYÜZ (INTERFACE)

Gerçek hayat: Elektrik prizi. Priz bir **söz** verir: *"iki delik, 220 volt."* Fişi takan cihazın ütü mü, şarj aleti mi, buzdolabı mı olduğunu umursamaz. Cihaz da prizin arkasındaki kablounun bakır mı alüminyum mu olduğunu bilmez. **Söz ortaktır, arkası serbesttir.**

Yazılımda: Arayüz, “bu işi yapan her şey şu metotlara sahip olmalı” diyen bir sözleşmedir. **Nasıl** yapıldığını söylemez.

Neden kullanılır: Çağırın kodun, **hangi somut sınıfın** geldiğini bilmek zorunda kalmaması için. Böylece yenisi eklendiğinde çağırın kod değişmez.

Bu projede — SLA politikaları:

```
// SÖZLEŞME (arayüz): "SLA hesaplayan her sınıf şu iki işi yapabilmeli."
// Burada hesap YOK – sadece "ne yapabilmesi gerektiği" yazıyor.
interface SlaPolicy {
    supports(ctx: SlaContext): boolean; // Soru: bu iş emri bana uyuyor mu?
    calculate(ctx: SlaContext): SlaPlan; // Soru: süreleri hesapla ve geri ver
}

// SÖZLEŞMEYİ UYGULAYAN 1 – kritik varlıkta arıza çıktıysa 4 saat verir
class CriticalAssetBreakdownPolicy implements SlaPolicy { ... }

// SÖZLEŞMEYİ UYGULAYAN 2 – düşük öncelikli bakım işine 15 gün verir
class LowPriorityMaintenancePolicy implements SlaPolicy { ... }

// ★ Bu ikisini kullanan kod, hangisinin geldiğini BİLMEK ZORUNDA DEĞİL.
// Tek bildiği: "elimde SlaPolicy sözü veren bir şey var, calculate diyebilirim." Yeni politika eklenince çağırın kod hiç değişmiyor.
```

Factory (E.4) bu politikaları `SlaPolicy` olarak tanır — hangisinin geldiğini bilmez, sadece **sözü** bilir. Yarın yeni bir politika eklendiğinde factory'ye dokunulmaz.

★ Bu tek kavram, SOLID'in **O** ve **D** harflerinin (E.2) ve Factory deseninin (E.4) tamamının dayandığı fikirdir.

KATMAN

Gerçek hayat: Restoran. **Salon** (garson) sipariş alır, **mutfak** (aşçı) pişirir, **depo** malzeme tutar. Garson yemek pişirmez, aşçı sipariş almaz.

Kritik olan şu: **aşçı değişse garsonun işi değişmez**. Yeni aşçı aynı yemeği pişirdiği sürece salon hiçbir şey fark etmez.

Yazılımda: Sorumluluğa göre ayrılmış bölümler. Her katman kendi işini yapar ve komşusuyla belirli bir noktadan konuşur.

Bu projede dört katman var:

Katman	Ne yapar	Bu projedeki örnek
API (controller)	HTTP isteğini alır, cevabı döner	<code>POST /work-orders/1042/close</code> ucu
Application (servis)	Adımları sıraya koyar, transaction açar	"Kapat" işleminin akışı: kural çalıştır → kaydet → geçmiş yaz
Domain	Saf iş kuralları	"Kapalı iş emri güncellenemez"
Infrastructure	Dış dünya: veritabanı, kuyruk, e-posta	Prisma sorguları, BullMQ işleri

"Biri değişse diğeri etkilenmez" bu projede ne demek — somut üç örnek:

Değişiklik	Etkilenen	Etkilenmeyen	Neden
Prisma'dan başka bir ORM'e geçilse	Yalnızca infrastructure	Domain ve application	Domain, Prisma'yı hiç tanımıyor
REST yerine GraphQL eklense	Yalnızca API katmanı	Domain, application, infrastructure	Kurallar HTTP'yi bilmiyor
"Çözüm açıklaması artık 20 karakter olmalı"	Yalnızca domain	API, veritabanı	Kural tek yerde yaşıyor

🚫 **Katman ihlali neye benzer:** Controller'ın doğrudan `prisma.workOrder.update()` çağırması. Çalışır — ama o an iş kuralı atlanmış olur ve aynı işlem mobilden geldiğinde kural uygulanmaz. Bu yüzden ihlal `dependency-cruiser` (C.8) ile CI'da engelleniyor.

BAĞIMLILIK

Gerçek hayat: Kahve makineniz belirli bir kapsül markasına bağımlıysa, o marka üretimi durdurduğunda makineniz çöp olur. Bağımlılık tek yönlü bir borçtur.

Yazılımda: Bir kodun çalışmak için başka bir koda ihtiyaç duyması. A, B'ye bağımlıysa **B değiştiğinde A bozulabilir.**

Bu projede neden kritik: Mimarideki asıl soru *"kim kime bağımlı olacak"* sorusudur. Domain katmanı Prisma'ya bağımlı olsaydı, veritabanı değiştiğinde iş kuralları da bozulurdu. Bu yüzden ok **tersine çevriliyor** (E.2 → D harfi).

PROJEKSİYON

Gerçek hayat: Restoranda 60 kalemlik menü var ama garsona *"sadece tatlıları söyle"* diyorsunuz. Garson 60 kalemi baştan sona okuyup siz seçmiyorsunuz — **isteneni getiriyor.**

Yazılımda: Bir kaydın **yalnızca ihtiyaç duyulan alanlarını** veritabanından istemek.

SELECT * değil, SELECT id, title, status.

Bu projede hangi somut sorunu çözüyor:

İş emri tablosunda ~25 kolon var: açıklama metni, çözüm notu, tüm zaman damgaları, audit alanları, iç notlar. Ama **liste ekranı 7 tanesini gösteriyor:** numara, başlık, durum, öncelik, atanan kişi, SLA bitışı, lokasyon.

Sayfada 20 kayıt varsa:

	Alan sayısı	Taşınan veri
Projeksiyonsuz (SELECT *)	20 × 25 = 500 alan	Açıklama metinleri dahil — kayıt başına birkaç KB
Projeksiyonlu (select)	20 × 7 = 140 alan	Yalnızca listede görünenler

Aradaki fark yalnızca hız değil: **açıklama metni ve çözüm notu gibi uzun alanlar hiç okunmuyor.** Bu alanlar detay ekranında zaten ayrıca isteniyor.

En kritik faydası güvenlik tarafında: Şemada olmayan alan cevaba çıkamıyor.

Kullanıcı tablosunda passwordHash var; liste şemasında yok, o yüzden sorguya da girmiyor, cevaba da (E.6).

```
// Veritabanına "bu kaydın hangi alanlarını istiyorum" diye söylüyoruz.
// 🚫 true yazılmayan hiçbir kolon çekilmiyor – açıklama metni, çözüm notu,
// audit alanları ve hassas alanlar veritabanından hiç çıkmıyor.
select: { id: true, number: true, title: true, status: true,
          priority: true, assigneeId: true, slaDueAt: true }
```

Ödevin §17 maddesi bunu doğrudan istiyor: *"Listeleme işlemlerinde bütün tablo belleğe alınarak mapping yapılmamalıdır."*

SLA (SERVICE LEVEL AGREEMENT — HİZMET SEVİYESİ TAAHHÜDÜ)

Gerçek hayat: İnternet servis sağlayıcınızla sözleşmeniz der ki: *"arıza bildirimini 24 saat içinde çözeriz."* Bu bir **taahhüttür**; tutulmazsa raporlanır, gerekirse yaptırımı olur.

Yazılımda / bu projede: Bir iş emrinin **ne kadar sürede çözülmesi gerektiğine** dair kurumsal taahhüt. Üç zaman üretir:

Alan	Ne demek	Örnek (kritik jeneratör arızası)
dueAt	Bitmesi gereken an	Açılıştan 4 saat sonra
remindAt	İlk hatırlatma	Bitişe 1 saat kala
escalateAt	Üst amire yükseltme	Bitişe 30 dakika kala

Süre neye göre belirleniyor: üç girdi — iş emrinin **önceliği**, varlığın **kritiklik seviyesi**, iş emrinin **türü**.

Bu projede tam olarak nerede kullanılıyor:

1. **İş emri oluşturulurken** hesaplanır ve kaydedilir
2. **Hatırlatma işi planlanır** (BullMQ gecikmeli iş — C.6)
3. **Tarama işi** her 15 dakikada bir süresi geçenleri bulur ve ihlal işaretler
4. **Liste ekranında** "SLA ihlali" filtresi bu alana bakar
5. **Yönetim ekranında** ihlal sayısı buradan sayılır

Süre nereye yazılıyor — kod akışı:

```
// 1. ADIM – Kullanıcının formdan gönderdiği üç bilgiyi factory'ye
veriyoruz.
//   Factory bunlara bakıp HANGİ SLA kuralının geçerli olduğunu seçiyor.
const policy = this.slaFactory.resolve({
  priority: dto.priority,           // Kullanıcının seçtiği
öncelik: HIGH
  assetCriticality: asset.criticality, // Varlık kaydından okundu:
HIGH
  type: dto.type,                 // İş emri türü: BREAKDOWN
(arıza)
});

// 2. ADIM – Seçilen kural üç zamanı hesaplıyor.
const plan = policy.calculate(ctx);
// Sonuç → { dueAt: 14:00,      bitmesi gereken an
//           remindAt: 13:00,   hatırlatma zamanı
//           escalateAt: 13:30 } üst amire bildirme zamanı

// 3. ADIM – Hesaplanan zamanlar iş emriyle AYNI SATIRA yazılıyor.
//   Ayrı tabloya konsaydı, liste ekranındaki "SLA'sı yaklaşanlar"
filtresi
//   her seferinde iki tabloyu birleştirmek zorunda kalırdı.
await tx.workOrder.create({
  data: { ...dto, slaDueAt: plan.dueAt, slaRemindAt: plan.remindAt },
});

// 4. ADIM – Hatırlatmayı şimdi değil, ileri bir saatte çalışacak şekilde
//   kuyruğa bırakıyoruz. Kimse ekranı açmasa da o saatte kendisi
çalışacak.
await this.queue.add('sla-reminder', { workOrderId: created.id },
  { delay: plan.remindAt.diffNow().milliseconds, // kaç milisaniye
sonra çalışsın
  jobId: `sla-reminder-${created.id}` });      // aynı iş iki kez
sıraya girmesin
```

★ `slaDueAt` 'in iş emriyle **aynı satırda** tutulması bilinçli: liste ekranında “SLA’sı yaklaşanlar” filtresi ek bir tablo birleştirmesi (JOIN) gerektirmeden çalışıyor ve bu kolona index konabiliyor.

E.1 Modüler monolit + Clean Architecture (§10)

Ne isteniyor: Ödev mimarının tarafımızdan tasarlanmasını ve **gerekçelendirilmesini** istiyor.

”MONOLIT” NE DEMEK

Gerçek hayat: Tek binalı bir hastane. Poliklinik, laboratuvar, eczane — hepsi aynı binada, aynı kapıdan girilir. Bir bölümden diğerine geçmek koridorda yürümektir: hızlı ve sorunsuz.

Karşıtı — mikroservis: Aynı hastanenin bölümleri şehrin farklı noktalarında ayrı binalarda. Hasta laboratuvara gitmek için **dışarı çıkıp taksiye biniyor**. Trafik varsa gecikir, taksi bulunamazsa hiç gidemez.

Yazılımda:

	Monolit	Mikroservis
Kaç program	Tek	Modül başına ayrı program
Modüller nasıl konuşur	Doğrudan fonksiyon çağrısı — mikrosaniye	Ağ üzerinden — milisaniye + hata riski
Nerede çalışır	Tek süreç	Ayrı süreçler, genelde ayrı sunucular
Yayın	Tek seferde hepsi	Her servis ayrı ayrı

Sorduğun soruya net cevap — mikroservisler nasıl konuşur: Genellikle **HTTP/REST API** ile veya bir **mesaj kuyruğu** üzerinden. Ama bu trafik internetten değil, kurumun **iç ağından** geçer. Yani “internet üzerinden” değil, “ağ üzerinden” — servisler aynı veri merkezinde bile olsa aradaki çağrı bir ağ çağrısıdır ve **başarısız olabilir**. Monolitte bir fonksiyon çağrısı başarısız olmaz; mikroserviste olur ve bunu yönetmek zorundasınız.

BU PROJEDE HANGİ MODÜLLER MONOLİT İÇİNDE, HANGİLERİ DEĞİL

Bu projede modüllerin tamamı tek programda (monolit içinde):

Modül	Neden monolit içinde
Lokasyon	İş emriyle aynı transaction içinde okunuyor (pasif lokasyon kontrolü)
Varlık	Aynı — iş emri açılırken varlık durumu kontrol ediliyor
İş emri	Sistemin çekirdeği; diğer her şey buna bağlı
Bildirim	İş emri değişiklikleriyle aynı transaction içinde yazılıyor
Kullanıcı/yetki	Her istekte kontrol ediliyor, ağ gecikmesi kaldırılamaz

★ **Belirleyici ölçüt: transaction sınırı.** İki modülün verisi *”ya ikisi birden ya hiçbiri”* kuralıyla yazılıyorsa, onları ayrı programlara bölmek transaction’ı **kaybettirir**. İş emri durumu değişip geçmiş kaydı yazılmazsa sistem tutarsız kalır (E.5). Mikroserviste bunu çözmek için *saga* denen çok daha karmaşık bir mekanizma kurulur — bu projede karşılığı olmayan bir maliyettir.

Ayrı süreçte çalışan tek parça: worker.

Ama dikkat — worker bir **mikroservis değildir**. Aynı kod tabanını, aynı veritabanını ve aynı iş kurallarını kullanır; sadece **farklı bir süreç olarak** başlatılır. Ayrılma sebebi iş bölümü değil, **çalışma biçimi**: API isteğe cevap verir, worker istek olmadan çalışır (C.6).

İleride hangisi mikroservise ayrılabilir: Bildirim modülü. Sebebi: e-posta ve SMS gönderimi eklendiğinde dış servislere bağımlı hâle gelir, yavaşlar ve ayrı ölçeklenmesi gerekebilir. Sınırı **şimdiden çizili** olduğu için o gün geldiğinde ayırmak kolay olacak — modüler monolitin asıl vaadi budur.

”MODÜLER” NE KATIYOR

Tek program, ama içi net sınırlı modüllere bölünmüş. Modüller birbirinin iç koduna elini sokmaz; yalnızca **ilan edilmiş arayüzünü** çağırır.


```
src/modules/
├─ locations/      → LocationsModule (dışarı sadece LocationsService
açılır)
├─ assets/         → AssetsModule
├─ work-orders/    → WorkOrdersModule
└─ notifications/ → NotificationsModule
```

🚫 **Yasak olan:** `work-orders` modülünün `notifications` modülünün Prisma sorgusunu doğrudan çağırması. **Serbest olan:** ilan edilmiş servisini çağırması. Fark şudur: ilki değişirse ikisi birden bozulur, ikincisi sözleşme olduğu için korunur.

CLEAN ARCHITECTURE'IN TEK KURALI

Bağımlılık oku hep içe bakar.

```
infrastructure    (Prisma, HTTP, BullMQ, Redis)
  ↓ bilir
application      (use case: "iş emrini kapat")
  ↓ bilir
domain           (saf iş kuralları – hiçbir kütüphane bilmez)
```

Dış katman içeriği bilir; **iç katman dışarıyı bilmez.**

Somut karşılığı: *"Kapatılmış iş emri güncellenemez"* kuralı en içte yaşar ve Prisma'yı da NestJS'i de HTTP'yi de tanımaz.

İki faydası:

1. O kuralı test etmek için **veritabanı gerekmez** — test milisaniyede koşar
2. Yarın Prisma'dan başka bir araca geçilse **iş kuralları hiç bozulmaz**

★ **Ödevle birebir örtüşen nokta:** §8 şunu şart koşuyor: *"Domain katmanı Entity Framework Core, Hangfire veya ASP.NET Core bağımlılıklarını bilmemelidir."* Bu cümle kelimesi kelimesine Clean Architecture'ın bağımlılık kuralıdır.

Ve bu kural yorum olarak değil **test olarak** korunuyor: `dependency-cruiser` (C.8), domain katmanına Prisma import edildiği an derlemeyi durduruyor.

MONOREPO, MONOLIT VE MIKROSERVIS — ÜÇ TERİM, ÜÇ AYRI SORU

Bunlar sık karıştırılır ama **farklı sorulara** cevap verirler:

Terim	Hangi soruya cevap verir	Bu projedeki cevap
Monorepo / polyrepo	Kod nerede duruyor?	Monorepo — tek git deposu
Monolit / mikroservis	Program nasıl çalışıyor?	Modüler monolit — tek program (+ worker süreci)
Tek sunucu / çok sunucu	Nerede yayınlanıyor?	Konteynerler, kurumun sunucusunda

Bunlar bağımsız eksenlerdir. Mikroservisler **aynı monorepo içinde** de durabilir (Google böyle çalışır), ayrı depolarda da.

Karar nasıl verilir — mikroservis ne zaman haklı olur:

Koşul	Bu projede var mı
Modülün kendi ekibi ve kendi yayın takvimi var	✗ Tek geliştirici
Modül çok farklı ölçekleniyor (biri 100 kat yük alıyor)	✗ Yük dengeli
Modül farklı teknoloji gerektiriyor (ör. Python ile görüntü işleme)	✗ Hepsi TypeScript
Modüller arası tutarlılık transaction gerektirmiyor	✗ Gerektiriyor

Dördü de “hayır” olduğu için mikroservis bu projede **maliyeti olan, karşılığı olmayan** bir karar olurdu. Ödev §32 bunu doğrudan uyarıyor: *“Gereksiz teknoloji, pattern, katman veya abstraction eklemek tek başına olumlu değerlendirilmez.”*

Sunumda söylenecek cümle: *“Mikroservis düşünmedim değil — dört ölçüte baktım: ayrı ekip, ayrı ölçekleme ihtiyacı, farklı teknoloji ve transaction bağımsızlığı. Dördü de bu projede yok. Ama modül sınırlarını mikroservise ayrılabilir şekilde çizdim; bildirim modülü gün geldiğinde ayrılabilir.”*

E.2 SOLID prensipleri (§8)

Ne isteniyor: Ödev SOLID’e uyulmasını **zorunlu** tutuyor ve *”dokümantasyonda açıklanması değil, kod tabanında uygulanması”* bekleniyor.

SOLID beş prensibin baş harfidir. Her birini **gerçek hayat** → **kod** → **bu projede nerede** sırasıyla açıyoruz.

S — SINGLE RESPONSIBILITY (TEK SORUMLULUK)

Gerçek hayat: İsviçre çakısı ile mutfak bıçağı. Çakı her işi yapar ama hiçbirini iyi yapmaz; ekmek kesmeye kalkarsanız ekmek de çakı da zarar görür.

Kural: Bir sınıfın **değişmek için tek bir sebebi** olmalı.

Bu projede — yanlış ve doğru:

```
// 🚫 YANLIŞ: tek sınıf birbiriyle ilgisiz dört işi birden yapıyor
class WorkOrderService {
    create()      { /* kayıt + SLA hesabı + bildirim + e-posta hepsi burada */ }
    calculateSla(){ ... }    // SLA kuralı değişince bu dosya açılır
    sendEmail()  { ... }    // e-posta sağlayıcısı değişince de bu dosya
    açılır
    generatePdf() { ... }    // rapor biçimi değişince de bu dosya açılır
}
// Sonuç: dört farklı sebeple aynı dosyaya dokunuluyor, çakışma
kaçınılmaz.
```

Bu sınıf **dört ayrı sebeple** değişir: iş kuralı değişirse, SLA kuralı değişirse, e-posta sağlayıcısı değişirse, rapor formatı değişirse. Dört ekip aynı dosyaya dokunur.

```
// ✅ DOĞRU: her sınıfın değişmek için TEK bir sebebi var
class WorkOrderService { /* sadece iş emri akışı — kural değişirse
    burası */ }
class SlaPolicyFactory { /* sadece SLA seçimi — süre kuralı
    değişirse burası */ }
class NotificationService { /* sadece bildirim — sağlayıcı değişirse
    burası */ }
```

Ödev 8 bunu doğrudan söylüyor: *"Tek bir servis, sistemdeki bütün işlemlerden sorumlu olmamalıdır."*

O — OPEN/CLOSED (GELİŞTİRMEYE AÇIK, DEĞİŞTİRMEYE KAPALI)

Gerçek hayat: Priz çoklayıcı. Yeni bir cihaz eklemek için **duvardaki tesisata dokunmazsınız**, çoklayıcıya bir fiş daha takarsınız.

Kural: Yeni davranış **eklenerek** gelmeli, mevcut kod değiştirilerek değil.

Bu projede — SLA politikaları:

```
// 🚫 YANLIŞ: yeni bir kural gelince ÇALIŞAN fonksiyonun içini açmak
// gerekiyor
function hesapla(wo) {
  if (wo.priority === 'CRITICAL') return 4;    // kritik iş → 4 saat
  if (wo.priority === 'HIGH')      return 24;  // yüksek öncelik → 24
  saat
  // Yeni kural buraya girecek. Her giriş, üstteki iki satırı bozma riski
  taşır
  // ve bu fonksiyonun tüm testleri yeniden koşturmak zorunda.
}

// ✅ DOĞRU: yeni kural = tamamen yeni bir sınıf. Mevcut hiçbir satır
// değişmiyor.
class WeekendMaintenancePolicy implements SlaPolicy { ... }
// Modülde kayıt listesine tek satır eklenir; çalışan kod olduğu gibi
// kalır.
```

Neden bu kadar önemli: Çalışan bir kodu her açtığınızda bozma riski alırsınız. Eklemek risksizdir, değiştirmek risklidir.

L — LISKOV SUBSTITUTION (YERINE GEÇEBİLİRLİK)

Gerçek hayat: Priz sözü *"220 volt"* der. Bir cihaz o prize takılıp 380 volt beklerse **söz bozulmuştur** — cihaz yanar. Prize takılabiliyor olmak yetmez, **sözü tutmak** gerekir.

Kural: Bir arayüzü uygulayan her sınıf, sözü **bozmadan** yerine geçebilmelidir.

Bu projede:

```
// Arayüzün verdiği SÖZ: "calculate çağrılırsa MUTLAKA geçerli bir plan döner."
interface SlaPolicy {
  calculate(ctx: SlaContext): SlaPlan;
}

// 🚫 SÖZÜ BOZAN uygulama – derlenir ama çalışma anında patlar
class BrokenPolicy implements SlaPolicy {
  calculate(ctx) {
    // Kontrol işi türü ise hiçbir şey döndürmüyor. Oysa söz "plan döner"di.
    if (ctx.type === 'INSPECTION') return null;
    // ⚠️ Hata BURADA değil, bu kodu çağıran yerde görünecek: orası
    // plan.dueAt okumaya çalışıp null bulacak. Bulunması en zor hata türü.
  }
}
```

Bu sınıf derlenir ama çağırın kodu **çalışma anında** patlatır: factory'yi kullanan servis `plan.dueAt` okumaya çalışır ve `null` bulur. Üstelik hata, politikada değil **onu kullanan yerde** görünür — bulunması zor bir hatadır.

Doğrusu: uymuyorsa `supports()` zaten `false` dönmeli; `calculate()` çağrılmışsa mutlaka plan üretmeli.

I — INTERFACE SEGREGATION (ARAYÜZ AYRIMI)

Gerçek hayat: Otel odasındaki 30 düğmeli kumanda. Siz sadece ışığı açmak istiyorsunuz ama önünüzde perde, klima, müzik ve “temizlik istemiyorum” düğmeleri var. **Kullanmadığınız şeyler yolunuza çıkıyor.**

Kural: Bir arayüz, uygulayanına **kullanmayacağı şeyleri dayatmamalı.**

Bu projede:

```
// 🚫 YANLIŞ: sekiz işi birden dayatan dev arayüz.
// Sadece okuma yapan bir rapor servisi bunu uygulamak zorunda kalırsa,
// kullanmayacağı yedi metodu boş bırakmak veya hata fırlatmak durumunda kalır.
interface IWorkOrderRepository {
  find(); create(); update(); delete();
  archive(); export(); recalculateSla(); sendReminder();
}
```

Sadece okuma yapan bir rapor servisi bu arayüzü uygulamak zorunda kalırsa, kullanmayacağı yedi metodu boş bırakmak (veya hata fırlatmak) durumunda kalır.

```
// ✅ DOĞRU: okuma ve yazma ayrı sözleşmeler.
// Rapor servisi yalnızca Reader ister; testte sahtesini yazmak tek metot demek.
interface WorkOrderReader { findMany(f: Filter): Promise<WorkOrder[]>; }
interface WorkOrderWriter { save(wo: WorkOrder): Promise<void>; }
```

Rapor servisi yalnızca `WorkOrderReader` ister — testte sahtesini yazmak da tek metot yazmak demektir.

D — DEPENDENCY INVERSION (BAĞIMLILIĞIN TERSİNE ÇEVİRİLMESİ)

Bu, en çok karıştırılan harftir. **Somut örneklerle açalım.**

Gerçek hayat: Bir ev inşa ediyorsunuz. İki seçenek var:

- **Bağımlı yol:** Duvara *“yalnızca X marka klima takılabilir”* şeklinde özel bir yuva açıyorsunuz. X marka üretimi durdurduğunda **duvarı kırmanız** gerekiyor.
- **Tersine çevrilmiş yol:** Duvara **standart priz** koyuyorsunuz. Hangi marka klima olursa olsun takılır; marka değişince duvara dokunmuyorsunuz.

Burada **duvar (üst katman) markayı değil, prizi (sözleşmeyi) tanıyor**. İşte “tersine çevirme” budur: üst katman alt katmanın **somut hâline** değil, **sözleşmesine** bağlanır.

Yazılımda — bu projedeki gerçek durum:

Domain katmanında bir kural var: *"Aynı iş emri için aynı gün ikinci bir SLA hatırlatma bildirimi üretilemez."* Bu kuralı uygulamak için mevcut bildirimlere **bakması** gerekiyor — yani veriye ihtiyacı var.

```
// 🚫 YANLIŞ — iş kuralı sınıfı doğrudan veritabanı aracını tanıyor
import { PrismaClient } from '@prisma/client'; // ← altyapı, domain'e sızdı

class NotificationRule {
  // ⚠ Bu sınıf artık Prisma olmadan çalışamaz — testi için bile veritabanı kurmak gerekir
  constructor(private prisma: PrismaClient) {}

  async canNotify(workOrderId: number) {
    // Veritabanına gidip "bu bildirim daha önce yazılmış mı" diye bakıyor
    const existing = await this.prisma.notification.findFirst({ ... });
    return !existing; // yazılmamışsa bildirim üretilabilir
  }
}
```

Bunun üç somut zararı var:

1. Bu kuralı test etmek için **veritabanı kurmak** gerekir — test saniyeler sürer
2. Prisma'dan başka bir araca geçilse **iş kuralı da değişir**
3. `dependency-cruiser` (C.8) bu import'u görür ve **build'i kırmızı yakar**

```
// ✅ DOĞRU – iş kuralı, veritabanını değil yalnızca bir SÖZÜ tanıyor

// 1) SÖZLEŞME domain katmanında yazılıyor: "bana şunu sorabilen bir şey
//    lazım."
//    Nasıl cevaplanacağı burada yazmıyor – sadece sorunun kendisi
//    tanımlı.
interface NotificationChecker {
  wasNotifiedToday(workOrderId: number, kind: NotificationKind):
  Promise<boolean>;
}

// 2) İş kuralı yalnızca bu söze bağlanıyor.
//    Prisma'yı, SQL'i, veritabanının varlığını bile bilmiyor.
class NotificationRule {
  constructor(private checker: NotificationChecker) {} // dışarıdan
  verilecek

  async canNotify(workOrderId: number) {
    // "Bugün bu bildirim gitti mi?" diye soruyor. Cevabı kimin verdiği
    // önemsiz.
    return !(await this.checker.wasNotifiedToday(workOrderId,
    'SLA_REMINDER'));
  }
}

// 3) SÖZÜ TUTAN sınıf altyapı katmanında. Prisma ancak burada görünüyor.
//    Testte bunun yerine sahte bir sınıf verilir, iş kuralı farkı
//    anlamaz.
class PrismaNotificationChecker implements NotificationChecker {
  constructor(private prisma: PrismaClient) {}

  wasNotifiedToday(workOrderId, kind) {
    // Sorunun veritabanı karşılığı: böyle bir kayıt var mı?
    return this.prisma.notification.findFirst({ ... }).then(Boolean);
  }
}
```

★ **"Tersine çevrilen" tam olarak ne:** Normalde bağımlılık oku **üstten alta** akar (kural → veritabanı). Burada **arayüzün tanımlandığı yer** domain katmanıdır; infrastructure onu *uygulamak zorundadır*. Yani ok tersine döner: **altyapı, domain'e bağımlı hâle gelir**.

Bu projede nerede karşılaşılabilecek:

Domain'in ihtiyacı	Sözleşme	Uygulayan
Şu an saat kaç	<code>Clock</code>	Gerçekte sistem saati, testte sabit saat
İşlemi yapan kim	<code>CurrentUser</code>	<code>nestjs-cls</code> (C.16)
Bu bildirim bugün gitti mi	<code>NotificationChecker</code>	Prisma
İş emri numarası üret	<code>WorkOrderNumberGenerator</code>	Veritabanı sayacı

Testte kazanç somut: `Clock` sözleşmesine testte *"sen şu an 30 gün sonrasındasın"* dersiniz ve SLA ihlali kurallarını **gerçek zamanı beklemeden** doğrularsınız. `new Date()` doğrudan çağrılıydı bu test yazılamazdı.

ÖDEVİN ÖZELLİKLE UYARDIĞI DÖRT HATA

Ödev §8'de dört somut hata sayıyor; hepsi bu projede engelleniyor:

Ödevin uyarısı	Bu projedeki karşılığı
<i>"Controller'da iş kuralı bulunmamalı"</i>	Kurallar domain'de; controller yalnızca isteği alıp cevabı döner (E.0 → metot)
<i>"Tek servis her işten sorumlu olmamalı"</i>	S harfi — modül başına ayrı servis
<i>"Geniş arayüzlerde toplanmamalı"</i>	I harfi — <code>Reader</code> / <code>Writer</code> ayrımı
<i>"Yeni kural mevcut kodu büyük ölçüde değiştirmemeli"</i>	O harfi — Factory deseni (E.4)

E.3 DRY prensibi (§9)

Ne demek: *Don't Repeat Yourself* — "kendini tekrar etme". Aynı bilgi veya kural sistemde **tek bir yerde** yaşamalı.

Gerçek hayat: Bir kurumda telefon rehberi hem muhasebede hem insan kaynaklarında ayrı ayrı tutuluyor. Biri güncelleniyor, diğeri güncellenmiyor. Bir süre sonra **hangisinin doğru olduğu bilinmiyor** — ve genellikle insanlar önce baktıklarına inanıyor.

Asıl zarar “fazla iş” değil; **iki farklı doğrunun oluşması.**

ÖDEVİN SAYDIĞI YEDİ TEKRAR VE BU PROJEDEKİ KARŞILIĞI

Ödev §9’da yedi somut tekrar türü sayıyor. Her birini bu projedeki gerçek örneğiyle açıyoruz.

1. Aynı doğrulamanın farklı uçlarda tekrar yazılması

Somut örnek: İş emri başlığı **en az 5, en fazla 200 karakter** olmalı.

Bu kural en az dört yerde gerekir: yeni iş emri formu (tarayıcı), iş emri düzenleme formu (tarayıcı), oluşturma ucu (sunucu), güncelleme ucu (sunucu).

```
// 🚫 YANLIŞ: aynı kural dört ayrı dosyada elle yazılmış

// Tarayıcı – yeni iş emri formu
if (title.length < 5) setError('Başlık çok kısa');

// Tarayıcı – düzenleme formu. Kural aynı ama HATA METNİ farklı yazılmış
if (title.length < 5) setError('Başlık en az 5 karakter olmalı');

// Sunucu – oluşturma ucu
if (dto.title.length < 5) throw new BadRequest();

// Sunucu – güncelleme ucu. Burada sınır 3 yazılmış!
// Yani kullanıcı düzenleme ekranından 4 karakterlik başlık
kaydedebiliyor,
// oluşturma ekranından kaydedemiyor. Aynı sistem iki farklı davranıyor.
if (dto.title.length < 3) throw new BadRequest();
```

Son satır tam da olan şeydir: kural bir yerde güncellenir, diğerleri geride kalır. Sonuç: kullanıcı düzenleme ekranından 4 karakterlik başlık kaydedebilir, oluşturma ekranından kaydedemez. **Aynı sistem iki farklı davranır.**

```
// ✅ DOĞRU: kural tek dosyada tanımlı, üç yer birden bunu kullanıyor
// Dosya: packages/contracts/work-order.ts

export const CreateWorkOrder = z.object({
  // Başlık: metin olmalı, en az 5 en fazla 200 karakter.
  // Hata mesajı da burada – tarayıcı ve sunucu AYNI metni gösteriyor.
  title: z.string().min(5, 'Başlık en az 5 karakter olmalı').max(200),

  // Öncelik: yalnızca bu dört değerden biri olabilir, başkası reddedilir
  priority: z.enum(['LOW', 'NORMAL', 'HIGH', 'CRITICAL']),

  // Varlık numarası: tam sayı ve pozitif olmalı (0 veya eksi kabul edilmez)
  assetId: z.number().int().positive(),
});
```

Bu tek tanım **üç yeri birden** besliyor:

Kullanılan	Nasıl
Tarayıcıdaki form	React Hook Form + <code>@hookform/resolvers</code> (C.20)
Sunucudaki uç	NestJS doğrulama katmanı
API dokümanı	Swagger, aynı şemadan üretiliyor (C.18)

★ Kural değiştiğinde **tek satır** değişiyor ve üçü birden güncelleniyor. Hata mesajı bile aynı.

2. Durum geçiş kontrollerinin farklı servislerde tekrarı

"Kapalı iş emri güncellenemez" kuralı; güncelleme ucunda, atama ucunda, yorum ekleme ucunda ve arka plan işinde ayrı ayrı yazılırsa biri mutlaka unutulur.

Çözüm: Tek geçiş tablosu (E.5). Her değişiklik oradan geçer.

3. Aynı mapping'in farklı sınıflarda bulunması

İş emri listesinin hangi alanları döndüreceği iki ayrı yerde yazılırsa, birine yeni alan eklenip diğerine eklenmediğinde ekranlar farklı veri gösterir.

Çözüm: Response şekli tek Zod şemasından türetiliyor (E.6).

4. Aynı hata yapısının uçlarda tekrar oluşturulması

Her uç kendi hata biçimini üretirse, arayüz her ucun hatasını ayrı ele almak zorunda kalır.

Çözüm: Tek merkezî hata filtresi (E.7).

5. Audit alanlarının her işlemde ayrı doldurulması

`createdBy`, `updatedBy` alanlarını her serviste elle yazmak — biri unutulunca o kaydın izi kaybolur.

Çözüm: Prisma eklentisi merkezî doldurur (C.16).

6. Filtreleme ve sayfalama kodunun kopyalanması

İş emri, varlık ve lokasyon listelerinin üçünde de sayfalama var. Kod kopyalanırsa, azami sayfa boyutu sınırı birinde unutulur ve orası açık kalır.

Çözüm: Ortak sayfalama yardımcısı (E.10).

7. Bildirim kurallarının farklı işlerde tekrarı

Dört arka plan işinin dördü de bildirim üretiyor. Mükerrer engelleme mantığı dördüne ayrı yazılırsa, birinde eksik kalır ve kullanıcı çift bildirim alır.

Çözüm: Tek bildirim üretici; işler onu çağırır (C.6).

⚠ DRY'IN TEHLİKELİ TARAFI — HER TEKRAR ORTAKLAŞTIRILMAZ

Ödev şunu ekliyor: *"DRY prensibi uygulanırken gereksiz ve anlamsız abstraction oluşturulmamalıdır."*

Ayırt edici soru: İki kod aynı sebeple mi değişecek?

Durum	Ortaklaştırılır mı
Başlık kuralı hem formda hem sunucuda — aynı kural	✓ Evet
İş emri başlığı <code>min(5)</code> , lokasyon adı da <code>min(5)</code> — tesadüfen aynı sayı	✗ Hayır

İkincisini ortaklaştırırsanız, yarın lokasyon adı sınırı 3'e indiğinde iş emri başlığı da inmek zorunda kalır — ya da ortak fonksiyona parametre eklersiniz ve ortaklaştırmanın anlamı kalmaz.

Kural: Benzer **görünen** değil, aynı **sebeple değişen** kod ortaklaştırılır. Aksi hâlde olmayan bir bağımlılık icat edilmiş olur.

E.4 Factory Pattern ve SLA hesabı (§7)

Ne isteniyor: Ödev Factory Pattern kullanımını **zorunlu** tutuyor, SLA politikasının belirlenmesinde kullanılmasını istiyor ve *"göstermelik bir sınıf olmamalı"*, *"Service Locator olarak kullanılmamalı"*, *"Open/Closed ile uyumlu olmalı"* diye özellikle uyarıyor.

PROBLEM

SLA (Service Level Agreement), işin tamamlanması gereken süredir. Tek bir sayı değil; üç girdiye göre değişir:

- İş emrinin **önceliği**: düşük · normal · yüksek · kritik
- Varlığın **kritiklik seviyesi**
- İş emrinin **türü**: arıza · bakım · kontrol · kurulum

Her kombinasyon için üç ayrı zaman hesaplanmalı: **bitiş**, **ilk hatırlatma** ve **escalation** (yükseltme — süre aşılmadan önce üst amire haber verme).

NAİF ÇÖZÜM VE NEDEN SÜRDÜRÜLEMEZ

```
// 🚫 Bu yaklaşım kullanılmadı – neden kullanılmadığı aşağıda
function hesapla(wo: WorkOrder) {
  // Kritik öncelikli iş + kritik varlık → 4 saat
  if (wo.priority === 'CRITICAL' && wo.asset.criticality === 'HIGH')
    return 4;
  // Kritik öncelikli ama varlık kritik değil → 8 saat
  if (wo.priority === 'CRITICAL') return 8;
  // Yüksek öncelikli arıza → 24 saat
  if (wo.priority === 'HIGH' && wo.type === 'BREAKDOWN') return 24;

  // ⚠ Her yeni kural bu bloğun İÇİNE yazılmak zorunda; yani çalışan
  kodu
  // her seferinde açıp değiştiriyorsun ve mevcut kuralları bozma
  riski
  // alıyorsun.
}
```

İlk gün çalışır. Altıncı ayda sorun çıkar: her yeni kural bu bloğu **değiştirmeyi** gerektirir ve her değişiklik çalışan kuralları bozma riskidir. Ödev §7 bunu doğrudan işaret ediyor: *”Yeni bir SLA politikası eklendiğinde mevcut kodda mümkün olduğunca az değişiklik yapılmalıdır.”*

UYGULANAN TASARIM

Her kural kendi sınıfında, ortak bir **arayüz** (E.0) altında:

```
// SÖZLEŞME: SLA hesaplayan her sınıf şu iki soruyu cevaplayabilmeli
interface SlaPolicy {
  supports(ctx: SlaContext): boolean;      // "bu iş emri bana uyuyor mu?"
  calculate(ctx: SlaContext): SlaPlan;     // bitiş + hatırlatma + yükseltme zamanı
}

// TEK BİR KURAL, kendi sınıfında. Başka kuralı bilmiyor, etkilemiyor.
@Injectable()
export class CriticalAssetBreakdownPolicy implements SlaPolicy {

  // Bu kural yalnızca "kritik varlıkta arıza" durumunda devreye giriyor
  supports(ctx: SlaContext) {
    return ctx.assetCriticality === 'HIGH' && ctx.type === 'BREAKDOWN';
  }

  // Uyduysa süreleri hesaplıyor
  calculate(ctx: SlaContext): SlaPlan {
    // Şu andan itibaren 4 saat. Saati Clock servisinden alıyoruz ki
    // testte "sen şu an şu andasın" diyebilelim.
    const due = this.clock.now().plus({ hours: 4 });

    return {
      dueAt:      due,                      // bitmesi gereken an
      remindAt:  due.minus({ hours: 1 }),   // 1 saat kala hatırlat
      escalateAt: due.minus({ minutes: 30 }), // 30 dakika kala üst
    };
  }
}
```

Factory, politikaları **enjeksiyonla** alır ve ilk uyanı seçer:

```

@Injectable()
export class SlaPolicyFactory {
  // ★ Politikaların listesi DIŞARIDAN veriliyor – factory hiçbir yerden
  // kendisi arama yapmıyor. Bu ayırım, ödevin yasakladığı "Service
  Locator"
  // kullanımından kaçınmanın tam karşılığı.
  constructor(@Inject(SLA_POLICIES) private readonly policies:
    SlaPolicy[]) {}

  resolve(ctx: SlaContext): SlaPolicy {
    // Listeyi sırayla gez, "bu iş emri sana uyar mı?" diye sor, ilk
    uyanı al
    const match = this.policies.find(p => p.supports(ctx));

    // Hiçbiri uymadıysa sessizce varsayılan bir süre UYDURMUYORUZ.
    // Hata fırlatıyoruz ki eksik kural fark edilsin.
    if (!match) throw new NoSlaPolicyError(ctx);

    return match;
  }
}

```

Kayıt, modülde tek satır:

```

// Politikaların kaydedildiği tek yer. Yeni kural eklemek = bu listeye
bir
// sınıf adı yazmak. Factory'ye ve diğer politikalara DOKUNULMUYOR.
// ★ Open/Closed prensibinin somut karşılığı burası.
// ⚠ Sıra önemli: özel kurallar üstte, genel kural altta olmalı. Genel
kural
// öne geçerse özel olanlara hiç sıra gelmez ve hata sessiz kalır.
{ provide: SLA_POLICIES, useFactory: (...p: SlaPolicy[]) => p,
  inject: [CriticalAssetBreakdownPolicy, HighPriorityPolicy,
    DefaultPolicy] }

```

Yeni kural eklemek: yeni sınıf yaz, `inject` listesine ekle. Factory'ye, mevcut politikalara ve çağırılan koda **dokunulmaz**. SOLID'in O harfi (E.2) budur.

ÖDEVİN ÜÇ UYARISINA VERİLEN CEVAP

”**Service Locator olmamalı.**” Service Locator, sınıfın ihtiyacını konteynerden **kendisinin aramasıdır** (`container.get('policy')`). Bağımlılıklar imzada görünmez, test edilmesi zorlaşır. Burada factory hiçbir şey aramaz; politika dizisi **dışarıdan verilir** ve constructor imzasında görünür.

”**Göstermelik olmamalı.**” Factory gerçek bir seçim yapıyor ve sistemin başka hiçbir yerinde SLA seçme koşulu bulunmuyor — ödevin ifadesiyle *”SLA seçme koşulları sistemin farklı bölümlerine dağılmamalıdır.”*

”**Testlerle doğrulanmalı.**” Politikalar NestJS olmadan doğrudan `new` edilerek test edilebiliyor; factory testi ise “hangi bağlamda hangi politika seçiliyor” sorusunu doğruluyor. Sıra önemli olduğu için politika sırası da ayrı bir testle sabitlendi — aksi hâlde daha genel bir politika özel olanın önüne geçerse hata sessiz kalırdı.

E.5 Durum makinesi (§6)

Ne isteniyor: Durumların kontrolsüz değiştirilememesi, geçersiz geçişlerin **backend tarafında** engellenmesi, her değişiklikte geçmiş kaydı ve *”controller içerisinde dağınık koşul bloklarıyla yönetilmemesi.”*

GEÇİŞ TABLOSU

Durum makinesi, izin verilen geçişlerin tek bir yerde tanımlanmasıdır:

```
// Bu tablo şunu söylüyor: "soldaki durumdayken YALNIZCA sağdakilere
geçilebilir."
// Tabloda olmayan her geçiş otomatik olarak reddediliyor.
const TRANSITIONS: Record<Status, Status[]> = {

  // Yeni açılmış iş emri: ya birine atanır ya iptal edilir
  OPEN:      ['ASSIGNED', 'CANCELLED'],

  // Atanmış iş: çalışmaya başlanır, geri alınır (OPEN) veya iptal edilir
  ASSIGNED:  ['IN_PROGRESS', 'OPEN', 'CANCELLED'],

  // Üzerinde çalışılıyor: parça beklemeye alınır, çözülür veya iptal
  edilir
  IN_PROGRESS: ['WAITING_PART', 'RESOLVED', 'CANCELLED'],

  // Parça bekliyor: parça gelince işe devam, gelmezse iptal
  WAITING_PART: ['IN_PROGRESS', 'CANCELLED'],

  // Çözüldü: kapatılır – ya da sorun tekrarlarsa yeniden işleme alınır
  RESOLVED:   ['CLOSED', 'IN_PROGRESS'],

  // Kapatıldı: SON DURAK. Buradan çıkış yok, liste boş.
  CLOSED:     [],

  // İptal edildi: SON DURAK. Buradan da çıkış yok.
  CANCELLED:  [],
};
```

CLOSED ve **CANCELLED** **terminal** durumlardır — çıkışı olmayan düğümler. Ödevin *"kapatılmış iş emri normal güncelleme işlemleriyle değiştirilememelidir"* ve *"iptal edilmiş iş emri üzerinde işlem yapılamamalıdır"* maddeleri tablodan doğrudan okunur.

TABLOYA EK OLARAK DURAN KOŞULLU KURALLAR

Bazı kurallar sadece “hangi durumdan hangisine” sorusuyla ifade edilemez; geçişe **koşul** eklenir:

Geçiş	Ek koşul
OPEN → ASSIGNED	Atanan kullanıcı aktif ve rolü teknik personel olmalı
ASSIGNED → IN_PROGRESS	İş emri atanmış olmalı (atanmamış iş işleme alınamaz)
IN_PROGRESS → RESOLVED	Çözüm açıklaması zorunlu
herhangi → CANCELLED	İptal gerekçesi zorunlu

NEDEN DAĞINIK IF YERINE TABLO

Kontroller uç noktalara dağıtıldığında, yeni bir durum eklendiğinde hepsinin tek tek bulunması gerekir. Biri atlanır ve **sessizce açık kalır** — üstelik bu açık ancak o yoldan geçildiğinde fark edilir.

Tabloda toplandığında: yeni durum eklemek tabloya satır eklemektir, kontrol kodu değişmez. Ayrıca tablo okunabilir olduğu için iş birimine gösterilip doğrulatabilir.

Geçiş kontrolü domain katmanında yaşar; Prisma'yı, HTTP'yi ve NestJS'i bilmez (E.1). Bu yüzden testleri veritabanı gerektirmez.

GEÇİŞ İLE GEÇMİŞ KAYDININ ATOMIKLIĞI

Ödev §6 şunu şart koyuyor: *"Durum değişikliği ile geçmiş kaydı aynı işlem bütünlüğü içerisinde gerçekleştirilmelidir."*

```
// $transaction = "ya ikisi birden olur ya hiçbiri".
// Arada elektrik kesilse bile yarım kayıt kalmaz.
await prisma.$transaction(async (tx) => {

  // 1) İş emrinin durumunu değiştir.
  //   "where" içindeki version, başkası bu arada değiştirdiyse
  yakalamak için.
  const updated = await tx.workOrder.update({
    where: { id, version },
    data: { status: next, version: { increment: 1 } }, // sürümü bir
    artır
  });

  // 2) Aynı işlemde geçmiş satırını da yaz: kim, ne zaman, nereden
  nereye.
  //   Bu olmadan durum değişir ama "kim değiştirdi" kaydı olmaz.
  await tx.workOrderHistory.create({
    data: { workOrderId: id, from: current, to: next, byUserId, at:
    clock.now() },
  });
});
```

Geçiş yazılıp geçmiş yazılmazsa sistem doğru görünür ama **izi olmaz** — denetim açısından kaydın hiç olmamasından kötüdür.

GEÇERSİZ GEÇİŞİN SONUCU

Geçersiz bir geçiş denendiğinde domain katmanı tipli bir hata fırlatır; merkezî filtre bunu **409 Conflict** cevabına çevirir (E.7). Uç noktada `try/catch` bulunmaz.

E.6 Mapping kararı: neden bir mapping kütüphanesi yok (§13)

Ne isteniyor: Ödev AutoMapper veya Mapster kullanımını zorunlu tutuyor. Stack'i serbest bırakılan bu projede **karşılığı kullanılmayan tek maddedir**, bu yüzden ayrıca gerekçelendiriliyor.

MAPPING NEYİ ÇÖZER

Veritabanı kaydı (entity) ile dışarı dönen cevap (DTO) aynı şey değildir. Entity’de `passwordHash`, `deletedAt`, iç notlar gibi alanlar bulunur; hiçbiri dışarı çıkmamalıdır. Mapping, entity’yi DTO’ya dönüştürme işidir. C# tarafında bu dönüşüm elle yazıldığında çok kod tutar; AutoMapper bunu refleksiyonla otomatikleştirir.

ÖLÇÜM: İKİ ADAY DA ELENDİ

Aday	Haftalık indirme	Son yayın	Elenme sebebi
<code>class-transformer</code>	~12M	2022-12 , hâlâ 0.x	Üç yılı aşkın süredir yeni sürüm yok
<code>@autmapper/core</code>	~106K	Güncel	Niş — devralacak geliştirici tanımayabilir

İkisi de **dekoratörlü sınıf** gerektiriyor. Prisma düz nesne (POJO) döndürdüğü için, yalnızca mapper’ı beslemek amacıyla her model bir de sınıf olarak yazılmalıydı. Ödev \$12 tam bundan sakınmayı istiyor: *”Pattern kullanmış görünmek amacıyla eklenen yapılar olumlu değerlendirilmeyecektir.”*

YERINE KURULAN MEKANIZMA

Şema **tek kaynak**, `select` ondan türetiliyor, çıkış şemadan geçiyor:

```
// Dosya: packages/contracts – liste ekranının cevabı BURADA tanımlı.
// Bu şema hem sunucuyu hem tarayıcıyı besliyor, tek kaynak.
export const WorkOrderListItem = z.object({
  id: z.number(), // kaydın numarası
  number: z.string(), // "IE-2026-000148"
  title: z.string(), // iş emri başlığı
  status: z.enum(STATUSES), // yalnızca tanımlı
    durumlardan biri
  priority: z.enum(PRIORITIES), // yalnızca tanımlı
    önceliklerden biri
  assigneeName: z.string().nullable(), // atanmamışsa boş olabilir
  slaDueAt: z.date(), // SLA bitiş zamanı
  // 🚫 Burada olmayan hiçbir alan dışarı çıkamaz – şifre özeti, iç not,
  audit
});

// Yukarıdaki şemadan TypeScript tipi otomatik üretiliyor.
// Yani tipi ayrıca elle yazmıyoruz; şema değişince tip de değişiyor.
export type WorkOrderListItem = z.infer<typeof WorkOrderListItem>;
```

```
// Veritabanı sorgusu. select listesi yukarıdaki şemadan üretiliyor,
// yani veritabanı 25 kolonun tamamını değil yalnızca 7 tanesini çekiyor.
const rows = await prisma.workOrder.findMany({
  where, // filtreler (durum, lokasyon,
    tarih...)
  skip, take, // sayfalama: kaçınıcı sayfa,
    kaç kayıt
  select: selectFrom(WorkOrderListItem), // → { id: true, number: true,
    ... }
});

// ★ SON KONTROL: veri şemadan geçiriliyor. Şemada olmayan bir alan
// yanlışlıkla sorguya girmiş olsa bile burada kırılıyor.
// Koruma bir alışkanlık değil, mekanizma.
return z.array(WorkOrderListItem).parse(rows);
```

Üçüncü adım kritiktir: `passwordHash` yanlışlıkla `select`’e girse bile cevaba **çıkamaz**. Koruma konvansiyon değil, mekanizmadır.

ÖDEVİN DÖRT ŞARTININ KARŞILANMASI

Ödevin şartı	Karşılığı
Entity doğrudan response dönülmemeli	Cevap her zaman şemadan geçer
Mapping içinde iş kuralı çalıştırılmamalı	Şema yalnızca şekil doğrular
Hassas bilgi response'a taşınmamalı	Şemada olmayan alan kırpılır
Listelerde bütün tablo belleğe alınmamalı	<code>select</code> ile DB seviyesinde projeksiyon
Mapping konfigürasyonu test edilmeli	Şema round-trip testleri

SAVUNMA CÜMLESİ

"AutoMapper'ın üç işi — projeksiyon, dönüşüm ve hassas alan gizleme — karşılanıyor. Fark şu: hatalar çalışma anında değil derleme zamanında yakalanıyor. AutoMapper C#'ta gereklidir çünkü elle class→class dönüşüm çok kod tutar; Prisma'da `select` zaten projeksiyonun kendisidir. Ayrıca ölçtüm: aday kütüphanelerden biri üç yıldır bakımsız, diğeri niş."

E.7 Merkezî hata yönetimi (§19)

Ne isteniyor: Merkezî hata yönetimi, uçlarda tekrar eden `try/catch` bulunmaması, standart hata biçimi ve hata türlerinin ayrıştırılması.

PROBLEM

Gerçek hayat: Bir hastanede her poliklinik kendi hasta yönlendirme kâğıdını kendi tasarlıyor. Birinde "kat 3", diğesinde "3. kat", üçüncüsünde "üst kat" yazıyor. Hasta her kapıda yeniden anlamaya çalışıyor. Danışma tek bir standart koyduğunda sorun bitiyor.

Yazılımda: Her uçta `try/catch` yazılırsa üç sorun çıkar:

1. Aynı kod defalarca tekrarlanır (DRY ihlali — E.3)
2. Biri unutulur, o uçta uygulama **çöker**
3. Hata biçimi uçtan uca farklılaşır; arayüz her ucun hatasını ayrı ele almak zorunda kalır

ÇÖZÜM: TEK HATA FİLTRESİ

```
// 🚫 YANLIŞ: hata yakalama kodu her uç noktada yeniden yazılıyor
@Post('/:id/close')
async close(@Param('id') id: number, @Body() dto: CloseDto) {
  try {
    return await this.service.close(id, dto.note);
  } catch (e) {
    // Her hata türünü tek tek HTTP koduna çevirmek zorundasın
    if (e instanceof InvalidTransition) throw new
    ConflictException(e.message);
    if (e instanceof NotFound)      throw new NotFoundException();
    throw new InternalServerErrorException();
    // ...ve bu bloğu 40 uç noktada tekrarlıyorsun.
    // Birinde unutursan orada uygulama çöküyor.
  }
}

// ✅ DOĞRU: uç noktada hata yakalama YOK – sadece işi devrediyor
@Post('/:id/close')
close(@Param('id') id: number, @Body() dto: CloseDto) {
  // Servis hata fırlatırsa aşağıdaki merkezî filtre otomatik yakalıyor
  return this.service.close(id, dto.note);
}
```

Filtre tek yerde, tüm uygulamayı kapsar:


```

// @Catch() = "uygulamanın HER YERİNDEKİ hatayı yakala" demek.
// Tek bir yerde duruyor, 40 uç noktanın hepsini birden koruyor.
@Catch()
export class DomainExceptionFilter implements ExceptionFilter {
  catch(err: unknown, host: ArgumentsHost) {
    const res = host.switchToHttp().getResponse();

    // İsteğin başında üretilen takip numarasını al (C.16).
    // Aynı numara hem log'a hem kullanıcıya gidecek.
    const correlationId = this.cls.get('correlationId');

    // Hata türüne bakıp uygun HTTP kodunu belirle: 400 mü 409 mu 500 mü?
    const { status, code } = this.map(err);

    // Hatayı JSON olarak logla – sonradan aranabilir olsun diye (C.15)
    this.logger.error({ err, correlationId, code });

    // Kullanıcıya dönen cevap. Biçim her uçta AYNI, arayüz tek yerde ele
    alıyor.
    res.status(status).json({
      type:      `https://api.example/errors/${code}`,    // hatanın
dokümanı
      title:     this.title(code),                        // kısa başlık
      status,    // HTTP kodu
      code,      // uygulama hata
      kodu
      correlationId, // kullanıcı
ekranda bunu görür
      // Doğrulama hatasıysa hangi alanın hatalı olduğu da gönderilir
      errors:    err instanceof ValidationError ? err.fields : undefined,
    });
  }
}

```

HATA TÜRLERİ VE HTTP KODLARI

Durum	Kod	Anlamı
Doğrulama hatası	400	Gönderilen veri biçimsel olarak hatalı
Kimlik doğrulanmadı	401	Giriş yapılmamış veya jeton geçersiz
Yetkisiz işlem	403	Giriş var ama bu işleme yetki yok
Kayıt bulunamadı	404	—
Çakışma	409	Geçersiz durum geçişi veya eş zamanlı değişiklik (E.8)
İş kuralı ihlali	422	Biçim doğru ama kural izin vermiyor
Beklenmeyen hata	500	—

⚠ **401 ile 403 sık karıştırılır.** 401 *"seni tanımıyorum"*, 403 *"seni tanıyorum ama bu işi yapamazsın"* demektir. Muhasebe personeli iş emri atamaya kalkarsa **403** alır — çünkü kim olduğu belli, yetkisi yok.

400 ile 422 farkı: *"başlık boş"* biçim hatasıdır → 400. *"pasif lokasyonda iş emri açılmaz"* iş kuralı ihlalidir → 422. Biçim doğru, kural izin vermiyor.

CEVABIN İÇERİĞİ VE CORRELATION ID

Standart biçim kullanılıyor (RFC 9457 — "Problem Details"). En kritik alan **correlation ID**: kullanıcı ekranda bir kod görüyor, o kodu söylediğinde loglarda isteğin tüm yolculuğu tek aramayla bulunuyor (C.15).

🚫 **Canlıda hata detayı (stack trace) kullanıcıya gönderilmez** — sistemin iç yapısı hakkında bilgi sızdırır. Ödev §19 bunu ayrıca belirtiyor.

E.8 Transaction ve iyimser eşzamanlılık (§20)

Transaction'ın tanımı C.5'te. Burada ödevin ikinci şartı ele alınıyor: **aynı kaydın eş zamanlı güncellenmesi**.

KAYIP GÜNCELLEME (LOST UPDATE) PROBLEMİ

İki operasyon sorumlusu aynı iş emrini aynı anda açtı:

```
t0 A okur: öncelik = NORMAL
t0 B okur: öncelik = NORMAL
t1 A yazar: öncelik = HIGH
t2 B yazar: öncelik = LOW      ← A'nın değişikliği izsiz kayboldu
```

Hiçbir önlem yoksa son yazan kazanır ve **A hata görmez** — değişikliğinin uygulandığını sanır. Bu, veri kaybının en sinsi türüdür; log'a bile düşmez.

İKİ YAKLAŞIM

Kötümser kilitleme (pessimistic). Kayıt okunurken satır kilitlenir (`SELECT ... FOR UPDATE`), diğerleri bekler. Kısa ömürlü ve sıcak çekişmeli işlemlerde doğrudur. Web formunda **yanlıştır**: kullanıcı sayfayı açık bırakıp öğle yemeğine giderse satır saatlerce kilitli kalır.

İyimser eşzamanlılık (optimistic) — bu projede kullanılan. Kilit yok; çakışma **yazma anında tespit** edilir.

UYGULAMA

Her mutasyona uğrayan tabloda bir `version` sütunu var:

```
model WorkOrder {
    id      Int @id @default(autoincrement()) // birincil anahtar,
    otomatik artıyor
    version Int @default(0)                  // her güncellemede 1
    artacak sayaç
    // Bu sayaç sayesinde "ben okuduktan sonra başkası değiştirmiş mi?"
    // sorusunu cevaplayabiliyoruz.
}
```

Güncelleme, okunan sürümü **koşul olarak** taşır:

```
// Güncelleme koşullu: "bu kaydı değiştir AMA sürümü hâlâ okuduğum
sürümse."
const result = await tx.workOrder.updateMany({
  where: { id, version }, // ekranı açtığında
  okunan sürüm
  data: { priority, version: { increment: 1 } }, // değiştir ve sürümü
  artır
});

// ☆ Bu arada başkası kaydı değiştirmişse sürüm artmış olur, koşul
  tutmaz
// ve HİÇBİR satır güncellenmez. Etkilenen satır sayısı 0 gelir –
// çakışmanın kanıtı bu.
if (result.count === 0) throw new ConcurrencyConflictError(id);
```

Üretilen SQL kabaca:

```
-- Yukarıdaki kodun veritabanına gönderdiği gerçek komut:
-- "Şu kaydın önceliğini değiştir ve sürümünü bir artır,
-- AMA yalnızca sürümü hâlâ benim okuduğum değerse."
UPDATE "WorkOrder" SET priority = $1, version = version + 1
WHERE id = $2 AND version = $3;
-- ☆ Kontrol ve yazma TEK komutta. Ayrı yapılsaydı ikisinin arasına
  başka bir
-- güncelleme girebilir ve kontrolün kendisi yarış koşulu üretti.
```

Başkası araya girmişse `version` artmıştır, `WHERE` tutmaz, **etkilenen satır sayısı 0** olur. Bu, çakışmanın kanıtıdır.

☆ **Kritik ayrıntı:** kontrol ile yazma **tek bir ifadede** yapılıyor. “Önce oku, karşılaştı, sonra yaz” üç ayrı adım olsaydı adımların arasına başka bir güncelleme girebilirdi — yani kontrolün kendisi yarış koşulu üretti. `updateMany` + `count` kullanılmasının sebebi budur; `update` bulunamadığında `P2025` fırlatır ve bu hata “kayıt yok” ile “sürüm eskimiş” durumlarını ayırt ettirmez.

API DAVRANIŞI

Çakışmada **409 Conflict** döner ve cevap gövdesinde çakışan alan ile güncel sürüm bulunur. Arayüz kullanıcıya “*bu kayıt siz bakarken değişti*” uyarısını gösterip yeniden yüklemeyi önerir.

NEDEN BU YAKLAŞIM

Aynı iş emrine iki kişinin **aynı anda** dokunması nadirdir. Nadir bir durum için tüm kullanıcıları kilit kuyruğunda bekletmek yanlış takas olur; bunun yerine o nadir durumda kullanıcı bilgilendirilir. Ödev §20 zaten *”optimistic concurrency yaklaşımı uygulanmalıdır”* diyor.

TRANSACTION SINIRLARI — NEREDE AÇILIYOR, NEREDE AÇILMIYOR

Ödev *”her işlem için gereksiz manuel transaction açılmamalıdır”* diye uyarıyor. Bu projede transaction yalnızca **birden fazla yazmanın atomik olması gereken** yerlerde açılıyor:

İşlem	Transaction	Gerekçe
Durum değişikliği + geçmiş kaydı	✓	İkisi ayrılırsa iz kaybolur
Atama + atama geçmişi	✓	Aynı
SLA ihlali işaretleme + bildirim	✓	İhlal yazılıp bildirim üretilmemesi
Refresh token yenileme + eskisini iptal	✓	İkisi ayrılırsa iki geçerli token kalır
Tek kayıt güncelleme	✗	Zaten tek ifade, kendiliğinden atomik
Listeleme / okuma	✗	Yazma yok

E.9 Veri modeli kararları (§11, §21)

Ödev, veri tabanı tasarımının tarafımızdan yapılmasını ve **kararların dokümente edilmesini** istiyor. Aşağıdakiler o kararlar ve gerekçeleri.

SOFT DELETE (YUMUŞAK SILME)

Gerçek hayat: Bir kurumda emekli olan personelin kaydı **silinmez**; “pasif” işaretlenir. Silinseydi, o kişinin yıllar önce imzaladığı belgeler sahipsiz kalırdı.

Yazılımda: Kaydı gerçekten silmek yerine “silindi” olarak işaretlemek.

```
model Location {
  id          Int          @id @default(autoincrement()) // otomatik artan
  numara
  name        String
  // Soft delete: kayıt gerçekten silinmiyor, buraya silinme tarihi
  // yazılıyor.
  // null = kayıt aktif
  // dolu = silinmiş sayılıyor ama veritabanında duruyor
  deletedAt DateTime?
  // Her sorguda "silinmemiş olanlar" diye filtrelendiği için index
  // gerekli
  @@index([deletedAt])
}
```

Bu projede neden gerekli: Bir lokasyon kapandı diye silinirse, o lokasyona bağlı geçmiş iş emirleri sahipsiz kalır. *“Bu iş emri hangi binadaydı?”* sorusu cevapsız kalır — denetim açısından kabul edilemez.

Nerede kullanılıyor: Lokasyon, varlık, kullanıcı — yani **başka kayıtların referans verdiği** tablolarda. İş emri geçmiş gibi zaten değişmeyen kayıtlarda gerek yok.

⚠ Bedeli: Her sorguya “silinmemiş olanlar” koşulu eklemek gerekir. Unutulursa silinmiş kayıtlar listede görünür. Bu yüzden koşul tek tek yazılmıyor, merkezî bir filtreyle uygulanıyor:

```
// Her listeleme sorgusunun ilk koşulu: silinmiş kayıtları getirme.
// Kullanıcının seçtiği filtreler bunun ÜSTÜNE ekleniyor.
// ⚠ Bu koşul bir sorguda unutulursa silinmiş kayıtlar listede görünür –
// hata vermez, sessizce yanlış veri gösterir.
const where = { deletedAt: null, ...userFilters };
```

AUDIT ALANLARI

Her kayıta dört alan: **kim oluşturdu, ne zaman, kim güncelledi, ne zaman.**

```

model WorkOrder {
  createdAt DateTime @default(now()) // kayıt açıldığı an, veritabanı
  doldurur
  createdBy Int // kim açtı – Prisma eklentisi
  doldurur
  updatedAt DateTime @updatedAt // son değişiklik anı, otomatik
  güncellenir
  updatedBy Int? // kim güncelledi. İlk kayıta boş
  olabilir
  // Bu dört alan servis kodunda ELLE doldurulmuyor; merkezî bir eklenti
  // aktif kullanıcıyı okuyup kendisi yazıyor (C.16). Unutma ihtimali
  yok.
}

```

Bunlar **elle doldurulmuyor** — Prisma eklentisi `nestjs-cls` bağlamından okuyup otomatik dolduruyor (C.16). Sebep: elle doldurulan bir alan er geç unutulur ve o kaydın izi kaybolur.

ENUM'LARIN SAKLANMA BIÇIMI

Enum, sınırlı sayıda seçenekten oluşan alan: durum, öncelik, iş emri türü.

İki seçenek var: **sayı** olarak saklamak (0, 1, 2) veya **metin** olarak (`OPEN` , `ASSIGNED`). Bu projede **metin** tercih edildi.

```

-- Enum'u SAYI olarak saklarsak:
-- Veritabanına bakan kişi 3'ün ne demek olduğunu bilemez, kod dosyasını
açar.
SELECT status FROM "WorkOrder"; -- sonuç: 3

-- Enum'u METİN olarak saklarsak:
-- Veritabanına bakan herkes ne olduğunu doğrudan anlar.
SELECT status FROM "WorkOrder"; -- sonuç: 'RESOLVED'

```

Neden: Veritabanına doğrudan bakan biri (DevOps, raporcu, denetçi) sayıyı anlamaz. Ayrıca araya yeni bir değer eklendiğinde sayılar kayar ve **eski kayıtlar yanlış anlam kazanır** — metinde bu risk yok.

Bedeli: Birkaç bayt daha fazla yer. Bu ölçekte önemsiz.

TARİH VE SAAT STANDARDI

Tüm zamanlar veritabanında **UTC** (evrensel zaman) olarak saklanıyor; yalnızca kullanıcıya gösterilirken yerel saate çevriliyor.

Neden: Yaz saati uygulaması değiştiğinde veya sistem başka bir sunucuya taşındığında yerel saatle saklanmış veriler **kayar**. SLA hesabı saat farkına duyarlı olduğu için bu gerçek bir risk: bir iş emrinin “süresi geçti mi” sorusunun cevabı sunucunun saat dilimine göre değişmez.

INDEX KARARLARI

Index nedir: Kitabın arkasındaki dizin — aranan satırı tüm tabloyu okumadan bulmayı sağlar.

⚠ **Index bedava değildir:** her yazma işleminde güncellenmesi gerekir. Bu yüzden “her kolona index” yanlış bir yaklaşımdır; index **fiilen kullanılan filtrele** göre konur.

Index	Neden
<code>status</code>	Liste ekranının en sık filtresi
<code>assigneeId</code>	Teknik personel “bana atananlar” ekranı
<code>slaDueAt</code>	“SLA’sı yaklaşanlar” filtresi + arka plan taraması
<code>(locationId, status)</code> birleşik	İkisi birlikte filtrelenebilir
<code>title</code> GIN + <code>pg_trgm</code>	Metin araması (C.5)
<code>deletedAt</code>	Her sorguda kontrol ediliyor

★ **Birleşik (composite) index’te sıra önemlidir.** `(locationId, status)` index’i, yalnızca `locationId` ile yapılan aramada da çalışır; ama yalnızca `status` ile yapılan aramada **çalışmaz**. Bu yüzden sık kullanılan alan başa konur.

İŞ EMRİ NUMARASI ÜRETİMİ

Ödev, iş emrinin **benzersiz ve okunabilir** bir numarası olmasını ve üretim yönteminin tarafımızdan tasarlanmasını istiyor.

Biçim: `IE-2026-000148` → sabit ön ek + yıl + sıfır dolgulu sıra numarası.

Neden bu biçim: Telefonda söylenebilir, gözle sıralanabilir, hangi yıla ait olduğu okunur. Kurumsal sistemlerde numara insanlar arasında konuşulan bir şeydir — `a3f9c2b1-...` gibi bir kimlik bu işi görmez.

Nasıl üretiliyor:

```
-- Veritabanının kendi sayacı. Her çağrıldığında bir sonraki sayıyı verir
-- ve iki istek aynı anda gelse bile ikisine AYNI sayıyı vermez.
CREATE SEQUENCE work_order_seq_2026 START 1;
```

```
// Veritabanı sayacından sıradaki numarayı iste (örn. 148)
const [{ nextval }] = await tx.$queryRaw`SELECT
nextval('work_order_seq_2026')`;

// İnsanın okuyacağı biçime çevir: başına sıfır ekleyip ön ek ve yıl koy
// 148 → "IE-2026-000148"
const number = `IE-2026-${String(nextval).padStart(6, '0')}`;
```

🚫 **Neden uygulama içinde “son numarayı bul, bir artır” yapılmıyor:**

```
// 🚫 YANLIŞ YOL: "son numarayı bul, bir artır"
// Son kaydı oku
const son = await prisma.workOrder.findFirst({ orderBy: { id: 'desc' }
});
// Bir artır
const number = `IE-2026-${son.seq + 1}`;

// ⚠️ İki talep aynı anda gelirse ikisi de AYNI son kaydı okur ve aynı
//   numarayı üretir. Buna yarış koşulu denir; tek kullanıcı testi
//   asla görünmez, yük altında ortaya çıkar.
```

Veritabanı sayacı (sequence) bu yarış **yapısal olarak** engeller: iki eş zamanlı çağrı asla aynı sayıyı almaz. Üstelik transaction geri alınsa bile sayaç geri dönmez — yani numara “boşluklu” olabilir ama **asla tekrarlanmaz**.

Numaranın kendisi ayrıca `@unique` işaretli; ikinci savunma hattı.

EŞZAMANLILIK ALANI

Her mutasyona uğrayan tabloda `version` sütunu var — gerekçesi ve çalışma biçimi E.8’de.

E.10 Listeleme, filtreleme ve sayfalama (§17)

Ne isteniyor: İş emri listesinde sunucu tarafı filtreleme, sıralama ve sayfalama; *”bütün kayıtlar belleğe alındıktan sonra filtreleme yapılmamalıdır.”*

SUNUCU TARAFI NE DEMEK, NEDEN ZORUNLU

İki yaklaşım var:

	İstemci tarafı	Sunucu tarafı
Nasıl	Tüm kayıtlar tarayıcıya gider, filtre orada	Filtre + sayfa bilgisi sunucuya gider, DB yalnızca o sayfayı döner
200 kayıтта	Çalışır	Çalışır
200.000 kayıтта	Tarayıcı donar, ağ tıkanır	Fark etmez
Güvenlik	 Kullanıcının görmemesi gereken kayıtlar da tarayıcıya iner	Yetki filtresi sorguya girer

Güvenlik maddesi tek başına belirleyicidir: istemci tarafı filtreleme, yetki kontrolünü tarayıcıya devretmek demektir.

SORGU YAPISI

```
// Kullanıcının ekrandan seçtiği filtreler burada tek bir koşula
dönüşüyor.
// "&&" kalıbı şu demek: kullanıcı o filtreyi seçmediyse koşula hiç
eklenmiyor.
const where: Prisma.WorkOrderWhereInput = {
  deletedAt: null, // silinmişler hiç
gelmesin

  ...(status && { status }), // durum
seçildiyse ekle
  ...(assignee && { assigneeId: assignee }), // personel
seçildiyse ekle
  ...(locationId && { locationId }), // lokasyon
seçildiyse ekle
  ...(slaBreach != undefined && { slaBreach }), // "SLA ihlali"
kutususu

  // Tarih aralığı: başlangıç girildiyse iki tarih arasını filtrele
  ...(createdFrom && { createdAt: { gte: createdFrom, lte: createdTo }
}),

  // Arama kutusuna yazı girildiyse: başlıkta VEYA numarada ara.
  // mode:'insensitive' = büyük/küçük harf farkı gözetme
  ...(q && { OR: [
    { title: { contains: q, mode: 'insensitive' } },
    { number: { contains: q, mode: 'insensitive' } },
  ]}),
};

// İki sorgu birlikte çalışıyor: hem o sayfanın kayıtları hem toplam
sayı.
// Aynı işlem içinde oldukları için arada yeni kayıt eklenip sayı ile
// liste tutarsız hâle gelemiyor.
const [rows, total] = await prisma.$transaction([
  prisma.workOrder.findMany({ where, orderBy, skip, take, select }), //
sayfanın kayıtları
  prisma.workOrder.count({ where }), //
kaç kayıt var
]);
```

Kayıtlar ve toplam sayı **aynı sorgu bağlamında** alınır; ayrı çalıştırılırsalardı arada bir kayıt eklenip toplam ile sayfa tutarsız olabilirdi.

UYGULANAN KURALLAR

- **Varsayılan ve azami sayfa boyutu.** `?limit=999999` ile sunucu yorulamaz; değer azamiye kırılır.
- **Sıralama alanları beyaz listede.** Kullanıcının gönderdiği alan adı doğrudan `orderBy` 'a konmaz; izin verilen alanlar arasında değilse istek reddedilir. Aksi hâlde indexlenmemiş bir kolonla tam tablo taraması tetiklenebilir.
- **Filtreler URL'de taşınır.** Arayüz tarafında filtre durumu sorgu parametrelerine yazılır; böylece liste **paylaşılabilir ve yer imine eklenebilir** olur (ödev §22 bunu ayrıca istiyor).
- **Sayfalama kodu ortak.** Her uç için ayrı yazılmaz (DRY — E.3).

CEVAP ZARFI

Ödev §17 dönen bilgileri tek tek sayıyor:

```
{
  "items": [ ... ],           // bu sayfadaki kayıtlar (en fazla pageSize
  // kadar)
  "page": 2,                  // kaçınıcı sayfadayız
  "pageSize": 20,             // sayfa başına kaç kayıt
  "totalCount": 431,          // filtreye uyan TOPLAM kayıt sayısı
  "totalPages": 22,           // toplam kaç sayfa var
  "hasNext": true,            // sonraki sayfa var mı – arayüz butonu buna
  // bakıyor
  "hasPrevious": true         // önceki sayfa var mı
}
```

OFFSET YERINE CURSOR NE ZAMAN GEREKİR

`skip` / `take` (offset) yaklaşımı, veritabanının atlanan satırları **saymasını** gerektirir; derin sayfalarda (örneğin 5000. sayfa) yavaşlar. Bu ekranda kullanıcı filtreleyerek daralttığı için derin sayfalama beklenmiyor, bu yüzden offset tercih edildi. Sürekli akan bir listede (bildirimler) cursor tabanlı sayfalama daha doğrudur ve gerekirse oraya uygulanır.

E.11 Git akışı ve CI (§26, §27)

GIT NEDİR

Git, kodun değişiklik geçmişini tutan programdır. Bilgisayarda çalışır, internet gerektirmez. Her kayıt (**commit**) bir fotoğraf gibidir: o an kodun hâli ve neden değiştiği yazılıdır.

GitHub / GitLab ise bu geçmişin bir kopyasını internette veya kurumun kendi sunucusunda tutan **barındırma servisleridir**. Git'in vermediklerini eklerler: kod inceleme ekranı, değişiklik önerisi (Pull Request / Merge Request), otomatik test çalıştırma.

Benzetme: Git = belgeyi yazan ve sürümlerini tutan program. GitHub/GitLab = belgeyi sakladığın, paylaştığın ve üzerine yorum aldığın yer.

BU PROJEDEKİ DAL AKIŞI

```
main'den yeni dal aç → çalış, commit at → uzağa gönder
→ değişiklik önerisi aç → testler yeşil → main'e birleştir → dal
silinir
```

- **main** her zaman **çalışır durumdadır** ve doğrudan commit edilmez
- Her commit **tek bir işi** anlatır; ilgisiz değişiklikler aynı commit'te birleştirilmez
- Ödev §26 "projenin tamamının tek commit ile gönderilmemesi" ve "commit geçmişinin geliştirme sürecini göstermesi" şartını koyuyor — yani geçmişin kendisi de teslimin parçası

🚫 **Gizli değerler (şifreler, anahtarlar) asla commit edilmez.** `.env` dosyası gönderilmez; yalnızca hangi değişkenlerin gerektiğini gösteren `.env.example` gönderilir.

CI NEDİR

CI (Continuous Integration — sürekli entegrasyon), her değişiklik önerisinde testlerin **otomatik** çalışmasıdır.

Neden gerekli: “bende çalışıyordu” cümlesini ortadan kaldırır. Kod, kimsenin bilgisayarına bağlı olmayan temiz bir ortamda derlenir ve test edilir.

Bu projedeki zincir: bağımlılıkları kur → tip kontrolü → birim testler → entegrasyon testleri → mimari testler → arayüz derlemesi → Docker imajı. Herhangi biri kırmızı yanarsa değişiklik birleştirilemez.

Taşınabilirlik notu: Adımların kendisi proje içindeki bir komut dosyasında yaşar; CI dosyaları yalnızca o komutu çağırır. Böylece hem GitHub Actions hem GitLab CI aynı işi yapar ve platform değiştiğinde yeniden yazılmaz.

E.12 Dokümantasyon: ADR ve AI_USAGE (§28, §29)

ADR — MIMARI KARAR KAYDI

Nedir: *Architecture Decision Record*. Önemli bir teknik kararın **neden** alındığını yazan kısa belge. Ödev en az üç tane istiyor.

Gerçek hayat: Bir binada kolonun ortada durduğunu görürsünüz ve “burası kapı olsaydı daha iyiydi” dersiniz. Projeyi çizen mühendis o kolonu oraya **zemin etüdü** yüzünden koymuştur — ama bunu kimse yazmamışsa, biri gelip kolonu kaldırır.

Yazılımda: Kod *ne* yapıldığını gösterir, **neden** yapıldığını göstermez. Bir yıl sonra biri “burası neden böyle?” diye sorduğunda cevap ya ADR’dedir ya da kaybolmuştur. Kaybolduğunda o karar genellikle bozulur ve aynı duvara yeniden toslanır.

Biçimi:

ADR-004 – Mapping kütüphanesi kullanılmaması

Durum

<!-- Karar hangi aşamada: Önerildi / Kabul edildi / Geçersiz kılındı -->
Kabul edildi – 2026-08-21

Bağlam

<!-- Hangi sorunla karşılaşıldı, hangi kısıtlar vardı -->
Ödev AutoMapper veya Mapster kullanımını zorunlu tutuyor. JS tarafındaki adaylar ölçüldü: class-transformer (12M/hafta ama son yayın 2022), @autmapper/core (bakımda ama 106K/hafta).

Karar

<!-- Ne yapılmasına karar verildi -->
Mapping kütüphanesi kullanılmayacak. Response şekli Zod şemasında tanımlanacak,
Prisma select ondan türetilecek, çıkışta schema.parse() ile fazla alan kırılacak.

Sonuçlar

<!-- Artılar VE eksiler birlikte. Eksileri yazmak kararın düşünüldüğünün kanıtı -->
+ Hatalar derleme zamanında yakalanıyor
+ Hassas alan sızması yapısal olarak engelleniyor
– Ödevin harfi harfine istediği araç kullanılmadı; sunumda gerekçelendirilecek

Bu projedeki ADR’ler: mimari yaklaşım seçimi · ayrı backend kararı · Repository Pattern kullanılmaması · mapping yaklaşımı · eşzamanlılık yaklaşımı · enum saklama biçimi.

★ **En değerli kısmı “Sonuçlar” bölümündeki eksi maddeler.** Bir kararın bedelini yazmak, kararın **düşünülmüş** olduğunun kanıtıdır. Yalnızca artıları yazan ADR, karar kaydı değil savunma metnidir.

AI_USAGE.MD

Ne isteniyor: Ödev §28 yapay zekâ kullanımını serbest bırakıyor ancak **belgelenmesini** şart koşuyor: hangi araçlar, hangi aşamalarda, üretilen kodda yapılan önemli değişiklikler, **aracın yanlış veya eksik ürettiği en az bir örnek**, çıktıların nasıl doğrulandığı.

Ödevin kendi ifadesiyle: *”Agent böyle oluşturdu’ açıklaması teknik kararların sorumluluğunu ortadan kaldırmaz.”*

Neden bu dosya aslında bir avantaj: Aracın hatasını **fark edebilmek**, aracı kullanabilmekten daha değerli bir yetkinliktir. Dürüstçe yazılmış bir AI_USAGE dosyası, kodu sahiplendiğinizi gösterir.

Bu projede yazılacak gerçek örneklerden biri:

Araç, veri modeli dokümanını üretmek için `prisma-dbml-generator` paketini önerdi. Paket kontrol edildiğinde son yayınının 2024 başında olduğu ve Prisma 7 ile çalışmayacağı görüldü. Yerine `@dbml/cli` ile migration’ın ürettiği gerçek şemadan üretme yöntemi kuruldu ve CI’a bir tutarlılık kontrolü eklendi.

Bu tür bir kayıt iki şeyi birden gösteriyor: aracın önerisinin **doğrulandığını** ve yerine konan çözümün **daha sağlam** olduğunu.

ZORUNLU DOKÜMAN LİSTESİ

Ödev §29 şunları zorunlu tutuyor:

Dosya	İçeriği
<code>README.md</code>	Kurulum, Docker'lı ve Docker'sız çalıştırma, migration, testler, ortam değişkenleri, geliştirme kullanıcıları, bilinen eksikler, varsayımlar
<code>AI_USAGE.md</code>	Yukarıdaki içerik
<code>docs/database.dbml</code>	Veri modeli (C.21 — otomatik üretiliyor)
<code>docs/database-decisions.md</code>	E.9'daki kararlar
<code>docs/architecture.md</code>	Katmanlar, bağımlılık yönleri, istek yaşam döngüsü, transaction sınırları
<code>docs/api.md</code>	Uç listesi ve sözleşmeler
<code>docs/testing.md</code>	Test stratejisi ve nasıl koşulacağı
<code>docs/lifecycle.md</code>	Servis yaşam döngüsü tablosu (C.1 §4)
<code>docs/background-jobs.md</code>	Dört iş, zamanlamaları, idempotency yaklaşımı
<code>docs/decisions/</code>	ADR'ler

★ **README'nin en çok atlanan iki maddesi:** *"bilinen eksikler"* ve *"varsayımlar"*. Ödev ikisini de açıkça istiyor. Bunları yazmak zayıflık değil, **projeye hâkimiyet** göstergesidir — neyin eksik olduğunu bilmek, eksik olmadığını iddia etmekten daha güvenilirdir.

E.13 Değerlendirilip seçilmeyen alternatifler

Teknik incelemede en sık gelen sorulardan biri *"şunu düşündün mü?"* olur. Aşağıdakiler düşünüldü, ölçüldü ve **bilerek** seçilmedi.

NEDEN EXPRESS, FASTIFY DEĞİL (NESTJS'İN ALTINDAKİ HTTP KATMANI)

NestJS bir HTTP sunucusu değil; altına bir adaptör takılıyor. İki seçenek var: **Express** (varsayılan) ve **Fastify** (daha hızlı).

Darboğaz = zincirin en yavaş halkası. Yalnızca onu hızlandırmanın anlamı var; diğerlerini hızlandırmak toplam süreyi değiştirmez.

Bir iş emri listesi isteğinin süresi kabaca şöyle dağılıyor:

Aşama	Süre
HTTP katmanı (Express/Fastify)	~0.3 ms
Doğrulama + yetki kontrolü	~0.5 ms
Veritabanı sorgusu	20–80 ms
Cevabı JSON'a çevirme	~1 ms

Fastify HTTP katmanında yaklaşık iki kat hızlı: 0.3 ms → 0.15 ms. **60 ms'lik bir istekte kazanç %0.25** — ölçüm aleti zor görür.

Aynı emeği doğru bir index'e harcamak 80 ms'yi 5 ms'ye indiriyor: **16 kat**.

★ **Savunma cümlesi:** *"Fastify daha hızlı, doğru. Ama isteğimizin süresinin %95'i veritabanında geçiyor; HTTP katmanını iki katına çıkarmak toplamda ölçülemeyen bir kazanç veriyor. Emeği index'lere harcadım. Nest'te adaptör tek satır — ölçüp gerekirse geçeriz, bu karar bizi bağlamıyor."*

Ayrıca yaygınlık farkı büyük: Nest'in Express adaptörü haftada ~6.6M, Fastify adaptörü ~1.3M indiriliyor.

NEDEN REST, GRAPHQL DEĞİL

Önce yaygın bir yanılgıyı düzeltelim: *"REST tüm veriyi getirir, GraphQL sadece isteneni"* — bu, **kötü tasarlanmış REST'in belirtisi**, kuralı değil.

Bu projedeki REST zaten sadece gerekeni döndürüyor: liste ekranı için 7 alan, detay ekranı için 25 alan, ve veritabanından da yalnızca o alanlar çekiliyor (E.6). Yani mobil veri kotası argümanı burada geçerliliğini kaybediyor.

	REST	GraphQL
HTTP ön belleği	Kendiliğinden çalışır (CDN, tarayıcı, proxy)	Çalışmaz; kendi cache katmanını kurarsın
İzleme	Her uç ayrı adres — hangisi yavaş, hangisi hata veriyor doğrudan görünür	Tek adres (/graphql); izleme aracı hepsini aynı görür
Yetkilendirme	Uç bazında	Alan bazında — çok daha karmaşık
N+1 sorgu riski	Düşük	Yüksek; ek çözüm (DataLoader) gerekir
Nest'te yaygınlık	Varsayılan	@nestjs/graphql ~846K/hafta ile azınlıkta

★ **Belirleyici olan izleme maddesi:** Bu sistemi canlıda sen izlemeyeceksin, kurumun DevOps ekibi izleyecek. GraphQL'de *"hangi uç yavaşladı"* sorusu izleme aracında görünmez; kurumsal ortamda bu ciddi bir eksiktir.

GraphQL'in gerçekten kazandığı durum: *senin kontrol etmediğin* çok sayıda istemcinin farklı alan kombinasyonları istemesi. Buradaki istemciler kendi web'in ve kendi mobilin — ikisini de sen yazıyorsun, ihtiyaçlarını biliyorsun.

⚠ Gün gelir de başka kurumlar API'yi tüketmeye başlarsa, REST'in üstüne GraphQL katmanı **eklenebilir**. Bugünden ödenmesi gereken bir bedel değil.

NEDEN REPOSITORY PATTERN KULLANILMADI

Ödev §12 bunu **zorunlu tutmuyor**, ama kullanılırsa gerekçe istiyor — kullanılmaması da açıklanmalı.

Repository Pattern nedir: Veritabanı erişimini bir ara katmanın arkasına saklamak. Amacı, iş kodunun veritabanı aracını doğrudan tanımaması.

Neden bu projede eklenmedi:

1. **Prisma zaten o soyutlamadır.** Prisma Client, SQL'i saklayan ve tip güvenli bir arayüz sunan katmanın kendisi. Üstüne ikinci bir katman koymak, aynı işi iki kez yapmak olur.
2. **select ve include yeteneklerini kısıtlar.** Repository arkasına saklanınca, her ekranın ihtiyaç duyduğu farklı alan kümesi için ya ayrı metot yazılır ya da her şey döndürülür. İkisi de E.6'daki projeksiyon kazancını yok eder.
3. **Ödevin uyardığı tuzağa götürür:** *"Her entity için birbirinin aynısı generic CRUD servisleri oluşturulmamalıdır."* Repository katmanı çoğu projede tam olarak buna dönüşür.

Peki iş kuralları veritabanına nasıl bağımsız kalıyor: Repository ile değil, **bağımlılığın tersine çevrilmesiyle** (E.2 → D harfi). Domain katmanı ihtiyaç duyduğu soruyu kendi arayüzü olarak tanımlıyor; onu Prisma ile cevaplayan sınıf altyapı katmanında duruyor.

★ **Savunma cümlesi:** *"Repository eklemedim çünkü Prisma'nın kendisi o soyutlama. İkinci bir katman `select` yeteneklerini kısıtlar ve ödevin uyardığı generic CRUD tuzağına götürür. Domain'in bağımsızlığını Repository ile değil, bağımlılığı tersine çevirerek sağladım — ve bunu `dependency-cruiser` testiyle zorunlu kıldım."*

NEDEN BULLMQ, PG-BOSS DEĞİL

`pg-boss` işleri Redis yerine **PostgreSQL'de** tutuyor. Ödevin *"Hangfire verileri PostgreSQL üzerinde saklanmalıdır"* maddesine daha yakındı ve dördüncü bir konteyner gerektirmiyordu.

Seçilmeme sebebi **yaygınlık**: BullMQ ~7.9M indirme/hafta, pg-boss ~1.35M. Uzun ömürlü ve devredilecek bir kurum sisteminde daha az bilinen aracı seçmek, projeyi devralacak geliştiriciyi zorlar.

⚠ Redis'in eklenmesi tek amaçlı değil: iş kuyruğunun yanında **hız sınırı sayacı** ve **dağıtık kilit** için de kullanılıyor. Yani dördüncü konteynerin karşılığı yalnızca kuyruk değil.

BÖLÜM F — Bir iş emrinin hayatı (uçtan uca akış)

Önceki bölümler parçaları tek tek anlattı. Bu bölüm hepsini **tek bir isteğin yolculuğunda** birleştiriyor: bir talep açıldığı andan iş emri kapandığı ana kadar hangi katman devreye giriyor, hangi kural nerede çalışıyor.

 *Kod yazıldıkça gerçek dosya adları ve satır numaralarıyla doldurulacak.*

1. **Talep açılıyor** — Kullanıcı formu doldurur → Next form doğrulaması (`packages/contracts` şeması) → API'ye gider → **aynı şema** sunucuda tekrar doğrular (*istemciye asla güvenilmez*)
 2. **İş kuralları** — Lokasyon pasif mi? Varlık kullanım dışı mı? Bu kontroller `packages/domain` içinde, veritabanı bilmeden
 3. **SLA hesabı** — `SlaPolicyFactory` devreye girer (§6.1)
 4. **Kayıt** — İş emri + ilk geçmiş kaydı **tek transaction** içinde
 5. **Job planlanıyor** — BullMQ'ya gecikmeli hatırlatma bırakılır
 6. **Atama** — Yalnızca teknik personele; atama + geçmiş kaydı yine tek transaction
 7. **Durum değişikliği** — Durum makinesi geçerli mi diye bakar; geçersizse domain hatası → merkezî filtre → **409**
 8. **Eş zamanlı güncelleme** — İki kişi aynı anda değiştirirse `version` kolonu çakışmayı yakalar → **409 Conflict**
 9. **SLA aşımı** — Worker tarar, işaretler, bildirim üretir — **iki kez çalışsa da tek bildirim** (unique index + job anahtarı)
 10. **Kapanış** — Çözüm açıklaması zorunlu; kapalı kayıt normal güncellemeyle değiştirilemez
-

Bu akışta hangi karar nerede görünüyor

Adım	İlgili bölüm
Aynı şemanın iki tarafta doğrulaması	E.3 — DRY
İş kurallarının veritabanı bilmemesi	E.1 — Clean Architecture
SLA politikasının seçilmesi	E.4 — Factory Pattern
İş emri + geçmiş kaydının tek işlemde yazılması	E.5 ve E.8 — transaction
Gecikmeli hatırlatma işi	C.6 — BullMQ
Geçersiz durum geçişinin engellenmesi	E.5 — durum makinesi
Eş zamanlı güncellemenin yakalanması	E.8 — iyimser eşzamanlılık
Mükerrer bildirimin engellenmesi	C.5 ve C.6 — benzersiz index + iş anahtarı
Hataların 409'a çevrilmesi	E.7 — merkezî hata yönetimi

★ **Sunumda en çok işe yarayacak bölüm budur.** *"Mimariyi anlat"* dendiğinde katman isimlerini saymak yerine bu yolculuğu anlatmak, mimariyi **anlatmadan göstermek** demektir.

KAPANIŞ

Bilinen teknik borçlar

Ödev §31 bunu açıkça soruyor ve **sorulmadan söylenmesi** olumlu değerlendirilir: neyin eksik olduğunu bilmek, eksik olmadığını iddia etmekten daha güvenilirdir.

🕒 *Proje ilerledikçe doldurulacak.*

Bu stack'in tek cümlelik özeti

Kullanıcı **Next.js** ile yazılmış ekranı açar. Ekran, **NestJS** ile yazılmış API'ye istek atar. API gelen veriyi **Zod** ile doğrular, yetkiyi **JWT** ile kontrol eder, iş kurallarını uygular ve **Prisma** üzerinden **PostgreSQL**'e gider. Sonuç JSON olarak döner. Kullanıcıyı bekletmemesi gereken işler **BullMQ** ile kuyruğa alınır; ayrı bir **worker** süreci onları yapar. Tümü **Docker** ile paketlenir ve tek komutla ayağa kalkar.

Değişmeyen ilke

Stack baştan sona değişti, ancak ödevin sorduğu hiçbir yetenek düşmedi:

Ödevin sorduğu yetenek	Karşılandığı yer
Factory Pattern	SLA politikaları + NestJS bağımlılık enjeksiyonu
Servis yaşam döngüleri	NestJS DI (<code>DEFAULT</code> / <code>REQUEST</code> / <code>TRANSIENT</code>)
Transaction	<code>prisma.\$transaction</code>
İyimser eşzamanlılık	<code>version</code> kolonu + koşullu güncelleme
Mimari testler	dependency-cruiser
DBML şema dokümanı	<code>@dbml/cli</code> + CI kontrolü
Gerçek veritabanıyla entegrasyon testi	Testcontainers
Zamanlanmış görevler	BullMQ + worker
Merkezî hata yönetimi	NestJS Exception Filter
Merkezî audit	Prisma eklentisi + istek bağlamı

Araç değişti, beklenti değişmedi.